CS608

Software Verification and Validation

Topic 75-88

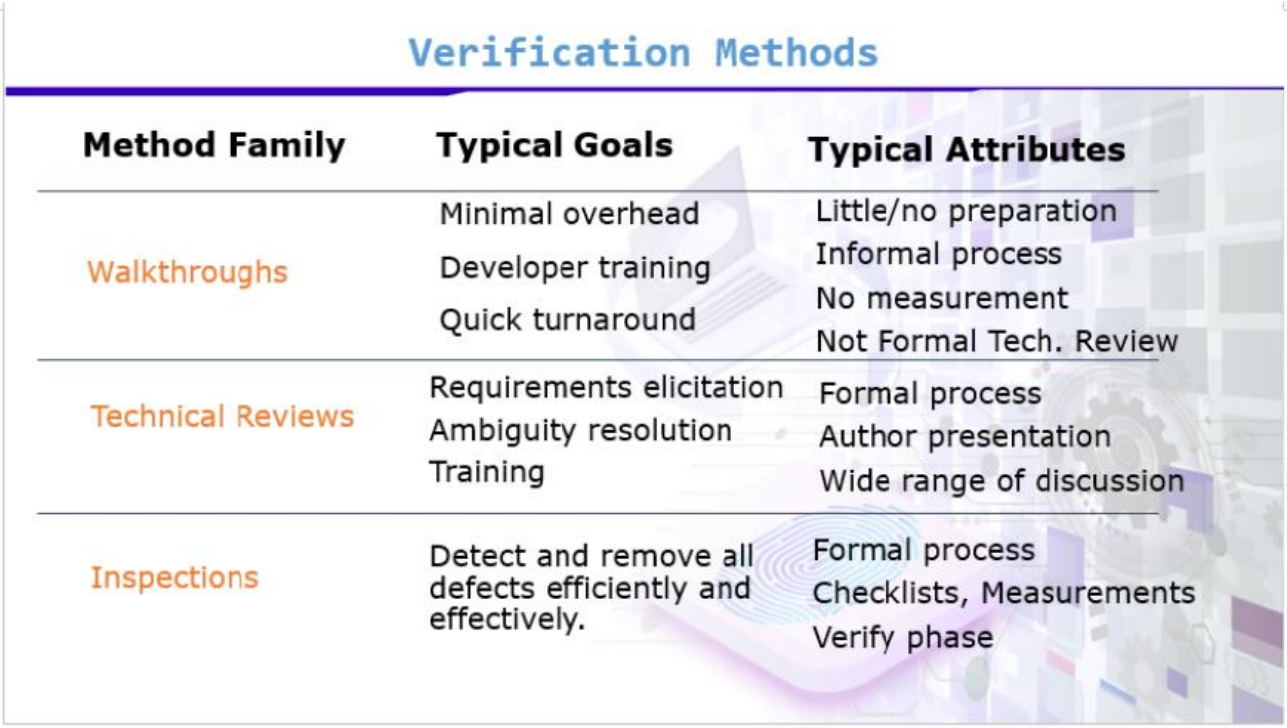
Quiz 3 Jan 8- Jan 10 Fall 2024

Introduction to Software Verification

Verification is process of assessing a software product or system during a particular phase to determine if it meets requirements or conditions that were specified at the beginning of that phase. It is a static process and it includes substantiation of artifacts such as SRS, SDS, design and code / program. In verification testing, we can find the bugs early in the development phase of the product and also we can verify that the inputs follow the outputs or not.

We mainly divide verification methods are as follows:

- **Peer Reviews**: A peer review is where the team members review one another's work to make sure that there are no mistakes in the documentation itself.
- **Walkthroughs**: The code once written is reviewed to identify any syntactic errors. This is more casual in nature and is performed by the individual developer on the code developed by oneself.
- **Inspections**: This is a more formal set up. Subject matter experts and developers check the code to make sure it is in accordance with the business and functional goals targeted by the software.



We make use of checklists which represent a set of time-tested means of making sure nothing is forgotten in any process is a checklist. A verification checklist will help manage all of the details that must be attended to in an artifact. Applications are SRS verification, design verification, code verification etc. Each step in adopted application development life-cycle requires verification before an artifact becomes a milestone and goes to the next stage.

Walk-Through

Walk-through is an informal and manual review technique to find the defects, bugs and different kind of ambiguities and problems in the documents. Walk-through are used for software verification to review documents with peers, managers, and fellow team members who are guided by the author of the document to gather feedback and reach a consensus. A walk-through can be pre-planned or organized based on the needs. This review becomes more beneficial for those people who are away from software sphere and through this meeting they get a good insight about the product that is to be developed.

In walk-through select members of the teams from different backgrounds in order to have a diverse point of view and thus, provide different dimensions to a common objective. It is pertinent to mention that this is not a formal process and it is used to find out if there are any misunderstandings or deviations from established standards both informal and defined that can be corrected or avoided. This is helpful in avoiding any manifestation of faults from mistakes made in the earlier stages such as requirements specifications, design and development and even testing process itself.

Defects/issues are noted down so that they can be fixed and they may be tracked to closure. Main purpose of walk-through is to enable learning about the content of the document under review to help team member's gain an understanding of the content of the document and also to find defects. Quality of results depend upon personal brilliance of the reviewer.

There are many objectives of the walk-through. Some of them are as follows;

- To improve the product overall
- To detect the hidden and unseen bugs
- To evaluate the test cases and documents.
- Understanding of the review object
- To discuss the alternative of design
- To discuss the alternative of implementations
- Mutual learning

In walkthrough meeting in led by the _____ and attended by the other members of the team.

a. reviewer of document
b. author of the document under review
c. tester of the document
d. project manager

• Peer Review

- An author asks a peer to read, comment, and critique a work artifact. If the work artifact is code, the reviewer will read the code, and may even develop and run some unit tests to check that the code works as advertised. Peer review or testing is a way of evaluating work performed by a co-worker. Review process does not involve management participation
- In software development, it stands for cross-checking the code written by developers. In review process, various things come to surface such as future casualties, technical content, quality, specifications of the software, etc. This deep overview of a software gives an idea

In a review Faults present those situations that may impact desired esults.

that how will the software run and is shortcomings that are likely to occur in the near future. It also provides an idea that this software is even worth launching. Sometimes, the software has fewer benefits and greater number of disadvantages falling, in this condition the software is discarded or a better development of better software is proposed.

- Peer review can be characterized as a method which can be review of documents (called artifacts) code, pair-programming, or a technical review by an expert who takes help from formal checklists to conduct this review as a process.
- Peer review requires authors to present their work, involves a team of 2 to 7 professionals; where the report optional to author and it presents issues in the shape a few faults. At the end of this process, there are questions such as what types of issues are and how their redressal can be planned, how these issues or faults can impact desired results and how these undesired outcomes can be avoided.

Inspection for Verification

- M. E. Fagan of IBM defined inspections in 1976. During inspection the documents are prepared and checked thoroughly by the reviewers before the meeting. It involves peers to examine the product. A separate preparation is carried out during which the product is examined and the defects are found. The defects found are documented in a logging list or issue log; a formal follow-up is carried out by the moderator applying exit criteria.
- Inspection is most formal and driven by checklists and rules. This review process makes use of entry and exit criteria. Inspections requires formal presenter in the shape of moderator (not author). Inspections can find about 60% of all defects. Inspections are technical, not management. Inspections results can assess/improve quality of work product, software development process and review process.
- Inspections reduce total project cost, but have their own execution costs. Upstream defect removal is 10-100 times cheaper. Reviews disseminate domain knowledge, development skills, and corporate culture.

mcqs

Fagan Inspection

steps of fagan inspection

The main idea behind this Fagan inspection methodology is to find out defects and devise a solution for the same. This inspection strategy follows a defined set of rules/process adhering to which one is able to attain the desired goal, to deal with bugs.

Author is the person that creates a product that should be inspected whereas moderator is the main person in the process. Team carrying out inspection creates the inspection plan and plays the role of the coordinator of the whole working group. Besides, team performs the duty of chairman in a session where issues about recording of errors are discussed.

Fagan inspection also identifies a reader whose role is only to read to the other members of inspection commission the text of documents that will be inspected. In order to formally record the process, we identify recorder as a role who documents all the errors that were found by the working group. A software tester can play a role of recorder. He/she also compiles a list of issues that are not discussed during the testing session. These issues can be not only about the found errors. Sometimes the recorder should transfer information from the members of the working group to the other project members and to the members of working group that for some reason were not present at the inspection session.

names imp

A formal inspection method consists of the following activities.

1. **Planning:** The moderator begins with planning the inspection activities, that is, forming the inspection team, gathering the necessary resources required during the inspection process, and distributes the necessary documents among the inspection team prior beginning the inspection process. Every document required for inspection should be clear, for instance the code should execute cleanly, and the listings (for defects) should be numbered.
2. **Overview (1-to-n meeting):** This part of the inspection process is concerned with discussions regarding the important features of the product. This process may be skipped if the key points are known to the participants in advance.
3. **Preparation (individual inspection):** Participants are expected to examine the work product before beginning with the inspection process, to be able to raise issues or to see if any defect is detected. In short one needs to prepare himself in advance for the inspection meeting so that whatever is to be discussed, they are able to provide their individual feedback.
4. **Inspection (n-to-n meeting):** This is the phase where the moderator begins leading the team, that is, the inspection process begins. The team is led by the moderator and the reader. If the moderator feels that the time allotted to the inspection meeting is insufficient, then he can reschedule the meeting at a later date.
5. **Casual Analysis:** This method deals with identifying the causes that leads to the occurrence of defects, thus allowing us to figure out ways to take preventive measures to avoid such defects to arise further.
6. **Rework:** The author shoulders the responsibility of solving issues or defects. The author must at least prepare an approximate estimation to deal with defects.
7. **Follow-up:** By the time the necessary rework is done, the moderator follows up with the author to check whether changes have been made correctly or not. The product must pass the inspection followed by this process. The defects should have resolved by now and the unresolved issues must be documented in a defect tracking system.

Fagan inspection in practice:

-
- Widely used in industry
- Evaluation studies
- Variations and other inspections

Example of checklist for SRS document inspection

Importance of checklists

Checklists represent industrial best practices as templates and provide a means of use of preexisting knowledge base and lessons learnt. It helps in conducting inspections / audits in an efficient and appropriate manner.

General (SRS requirements) Checklist

- Functional overview of system provided?
- Software and hardware environments been specified?
- Assumptions that affect implementation stated?
- Has every definition, acronym and abbreviation been defined in glossary, if they are correct?
- Are requirements, interfaces, constraints, definitions, etc. listed in appropriate sections?

Functional Requirements Checklist

- Have all requirements communicated by customer been specified?
- Are all inputs to a function sufficient to perform the required function?
- Are undesired events / inputs considered and their required responses specified?
- Is all use case descriptions provided?
- Are all misuse cases documented?

By and large, checklist for requirements verification must check if the document is:

- Complete, consistent, correct
- Precise, unambiguous, clear
- Relevant, testable, understandable
- Traceable, feasible, prioritized
- Free from design details

Examples of checklist for code Inspection

Code checklists are language specific. Following aspects are inspected in code via checklist.

Code formatting:

- Are proper alignments followed (left margin)?
- Are code block starting point/ending point easily identifiable?
- Are naming conventions (Pascal, CamelCase, Hungarian notation) followed?

Coding best practices:

- Are all similar values grouped under enumerated data types?
- Are comments properly given?
- Are comments provided for what is done rather why is done?
- Are there nested if/else statement in the code?
- Is there a separation of concerns through implementation of any framework

Similarly we will inspect structures, variables, comments, loops and branches.

Code Review Example

In order to review code, we need

- Moderator
- Team members
- Authors
- Checklists
- Reporting forms
- Artifact for inspection/review (Code)

The author presents code and moderator conducts planning, overview, preparation and inspection with two team members.

Checklist for code review:

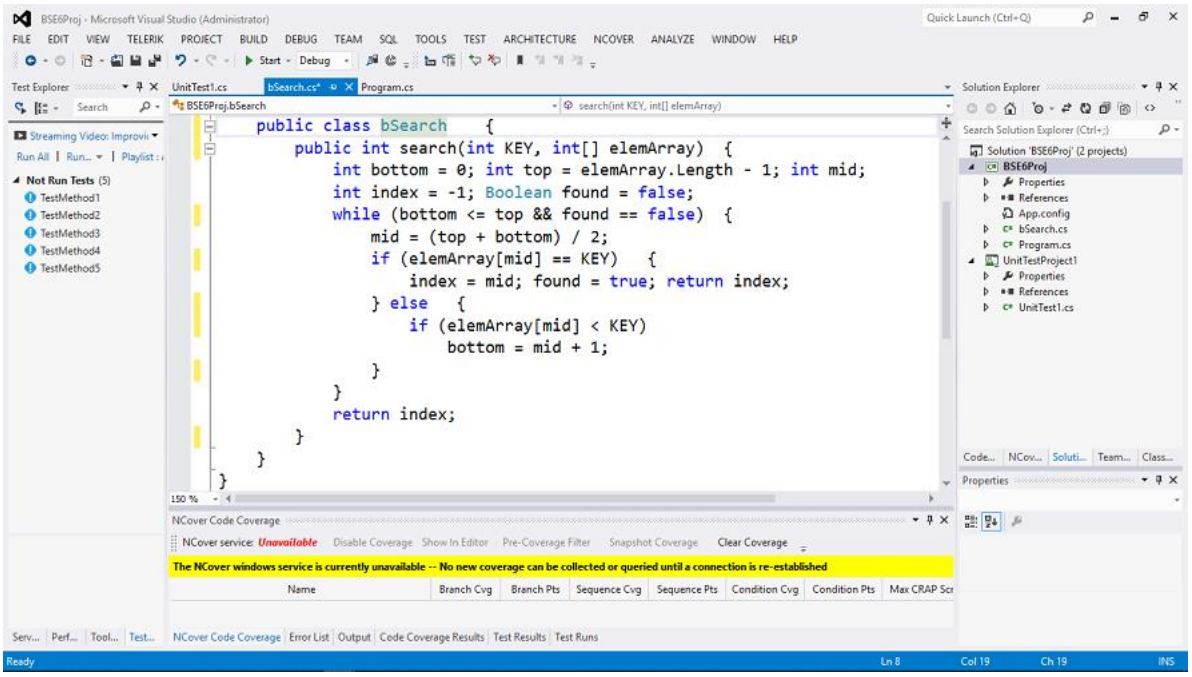
▪ Follows coding conventions. ▪ Names are simple and if possible short and are spelt correctly. ▪ Names contain units where applicable. ▪ Enums are used instead of int constants where applicable. ▪ All class, variable, and method modifiers are correct. ▪ There is no commented out code. ▪ There is no dead code. ▪ Debugging code is absent. ▪ No stack traces are printed. ▪ Variables not used with null values.	▪ Code is not repeated or duplicated. ▪ There is an else block for every if clause even if it is empty. ▪ No complex negatively named or long Boolean expressions. ▪ No empty blocks of code. ▪ Constructors don't accept null values. ▪ Arrays checked for out of bound. ▪ Catch clauses are appropriately used. ▪ Exceptions not ignored, if caught. ▪ Files/Sockets/Cursors and other resources are properly closed.

Issues classification:

Issues are classified in three major categories.

1. **Severe**: Defects that may cause incorrect results or behavior
2. **Moderate**: Defects that may affect limited areas of functionality that can either be worked around or ignored.
3. **Minor**: Defects that can be overlooked with no loss of functionality.

Code example:



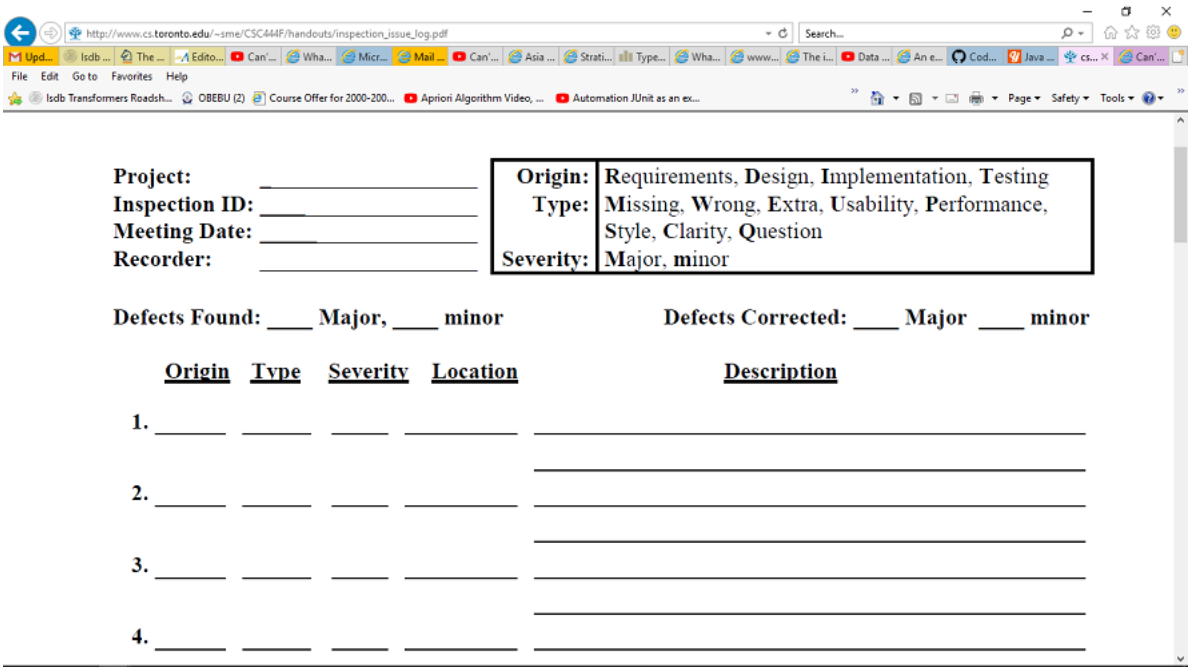
Following issues are found in above code:

- **Severe:** Try-catch block not implemented
- **Moderate:**

1. Constructor not present
2. Missing class, function and variable comments.

- **Minor:** No naming convention followed e.g., Hungarian notation is not followed for variable names, function names etc.

Example Issue Log Form:



Project:		**Origin:**	Requirements, Design, Implementation, Testing
Inspection ID:		**Type:**	Missing, Wrong, Extra, Usability, Performance, Style, Clarity, Question
Meeting Date:		**Severity:**	Major, minor
Recorder:			

Defects Found: ____ **Major,** ____ **minor** **Defects Corrected:** ____ **Major** ____ **minor**

	Origin	Type	Severity	Location	Description
1.					
2.					
3.					
4.					

After identifying issues in code via code review checklist the next step is "Review meeting" in which consolidated and comprehensive listing of issues is prepared and shared with key stakeholders. After "Review meeting", authors are given time to rework and then moderator (or representative) verifies if the corrections are made.

Importance of test planning:

While doing software testing through QA team in a software house, we need to do planning to determine effort needed to validate quality of SUT and to engage key stakeholders such as developers, business managers, customers etc. The output of test planning is **"Test Plan document"**. Test planning is essential to ensure that testing is a traceable process. Test planning make efficient use of resources and it delivers quantifiable results.

Steps involved in Test planning:

1. Analyze product and design test strategy
2. Define test objectives
3. Define test criteria (suspension/exit criteria)
4. Resource planning
5. Plan test environment
6. Determine test deliverables

The above steps are discussed as follows:

- Product analysis:

- Product family i.e., website, web service, client-server application, component etc.

- Strategy ingredients:

- Test management plan
- Test plan
- Scope(in and out), key risks, key issues, logistics (when, who, how)

- Test Criteria:
    - Suspension criteria: e.g.,
        - more than X number of crashes,
        - Y number of test cases reported to have failed
    - Exit Criteria denotes successful completion of a test phase e.g.,
        - Run rate: number test cases executed/total test cases, has to be 100%
        - Pass rate: numbers test cases passed / test cases executed

What is test case?

A test case undergoes a series of steps from its creation to being deleted or deprecated. A test case is a process rather than a single activity i.e. designing a scenario, preparing for execution and evaluating status till the test closure.

Steps involved in test case process:

A test case activity can be divided into fundamental test process with following steps explained below.

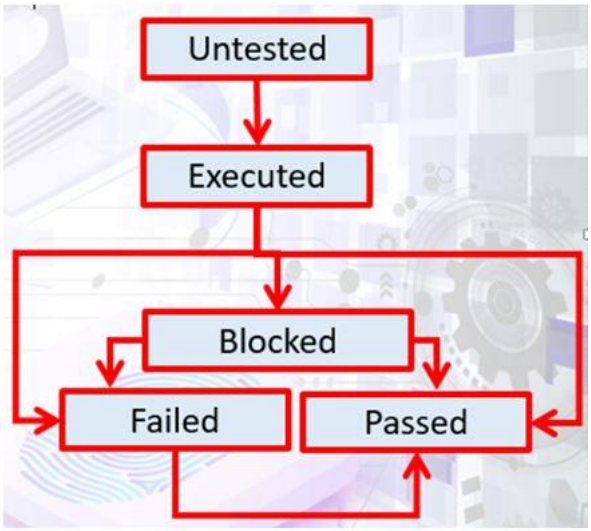
1. **Planning and Control:** This step determines scope and risks and identifies objectives of testing.
2. **Analysis and Design:** This step is used to review the information we need in order to start the test analysis and create our own test cases.
3. **Implementation and Execution:** It represents set of conditions under which a tester will determine whether an application is working correctly or not and test case is pass or fail.
4. **Evaluating exit criteria and reporting:** This step sets the criteria for each test level against which we will measure the “enough testing”.
5. **Test Closure**: These activities are done when software is delivered to QA testing companies. The testing can be closed when all the information has been gathered which is needed for the testing.

Test case life cycle:

1. Created with an ID and other details
2. Maintained in the repository for use
3. Reviewed periodically or upon a release:
   - Either kept as it is
   - Or updated and kept for use
   - Or deleted depending on situation



State transition of a test case:



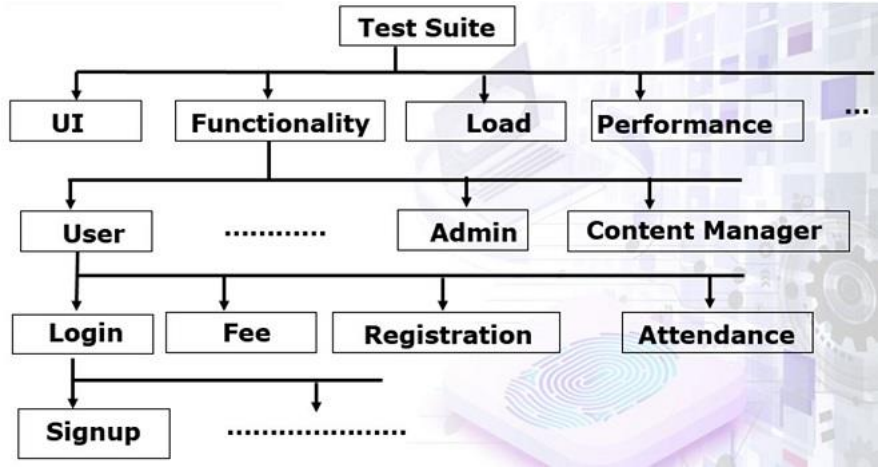
Test Suit Structure

What is test suite?

Test suit is a set of test cases organized in a hierarchal manner for a certain software release. There are two important characteristics of a test suite:

1.
    1. Test cases in a test suite relate to a certain release of the system.
    2. Test cases are organized in a hierarchical manner.

Example diagram of test suit structure:



Mutation Score = 100 * D / (N - E)

D = Dead mutants
N = Number of mutants
E = Number of equivalent mutants

What is traceability?

Traceability is a mechanism for tracking each software feature from SRS to Test docs (Test cases, Test suites). We need to make sure each requirement is covered in functional specification and test cases. Traceability helps in establishing a measure of completeness.

Example diagram for traceability:

SRS Section	SDS Section	Test case	Notes
1.1. Validation of user login credentials.	4.1. login validation.	6.1.4. User login with proper credentials.	
		6.1.5. User login with invalid username.	
		6.1.6. User login with invalid password.	
1.2. Validation of credit card information.	7.2.4. Credit card information verification.	10.1.1. Valid credit card information input.	
		10.1.2. Invalid credit card number.	
		10.1.3. Invalid credit card name.	
		...	

Software Testing Lifecycle Phases

What is software testing lifecycle?

Software testing life cycle (STLC) is defined as a sequence of activities conducted to perform Software Testing. Contrary to popular belief, software testing is not a just a single activity. It consists of a series of activities carried out methodologically to help certify a particular software product.

Phases of software testing lifecycle:

There are six phases of STLC which are as follows:

1.
    1. System under test (SUT) analysis
    2. Test Planning
    3. Test case development
    4. Test environment setup
    5. Test Execution
    6. Test Cycle closure

Test Result Reporting

Once testing activity (cycle) is complete, we need to share the results.

Inputs for test result reporting: Test plan document is an input for test result reporting which further includes

-
    - Test Design Specification
    - Test Case Specification
    - Test Input Data
    - Test Output Data
    - Test Procedure Specification

Outputs for test result reporting: Test incident report is the output for test result reporting which further includes

There are __5_OR 3_ outputs of test result reporting.

-
    - Test Log
    - Test Summary Report
    - Test Metrics
    - Coverage Analysis
    - Program Quality Estimate

. **Sample test result report:**

Overall progress of the QA cycle - 013	On time
Total number of test cases	227
Test cycle duration	5 days
Progress for 20/04/19	
Number of test cases planned	20
Number of test cases executed	18
Number of test cases executed overall	78
Number of defects encountered	2
Number of defect encountered so far	10
Number of critical defects- still open	3
Overall status	
Number of test cases planned	100
Number of test cases executed	78
Pass Percentage of the defects	98%
Defects density	2.5 per day
Critical defects percentage	20%

Sample test report form:

Consolidated test report form is the output from bug reporting database. We conduct a meeting in which development representative, testing team representative and project management discusses each of the rows in test execution report (test status report).

Module	Scenarios	Sub Levels	Complexity	Date	Status	Defect ID	Severity	Status
Login	Successful login	Signup	Simple	17/3/19	Fail	#4573 - Session data doesn't persist on different tabs	1- High	Open
		Forgot password	Moderate		Blocked	1028		
		Sign in	Simple		Pass			

Sample bug log form:

BUG LOG FORM

5655	Widget not visible on UI	1. Login to the site using the admin credentials 2. Click on E-course management. 3. Select e-course 4. Share e-course using social media widgets Expected result: The widgets should be visible on each e-course page Actual Result: Widget not visible	1- High	Open	Tester X	Dev representative

Bug Tracking Databases

We cannot use documents or spreadsheets for recording the test execution results. At the simplest, we have small scale bug tracking, open source, software having a backend in some DBMS. On the professional side, we have test management software e.g.,

-

- Microsoft Test Manager (MTM)
- JIRA
- IBM Rational Test Manager

These systems allow testers to report faults and issues. After reporting, they allow developers to view and fix these faults. These systems allow management to review and analyze the progress.

Example of open source bug tracking tool:

Figure 1 shows main interface of an open source bug tracking software available as "BTSys". A user can login into the software and track the bugs. For tracking, user can add bugs and messages. After processing the software will track bugs and display results and generate bug tracking report.

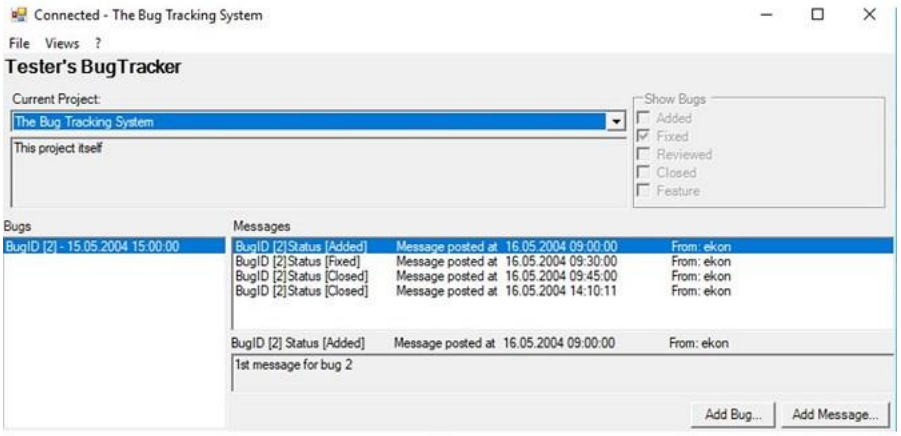


Figure 1

Test Data Management

How to manage test data?

While testing, we input data to make test cases execution possible. We keep track of test cases in a traceability matrix and know which test cases are related to which part of SRS, SDS etc. Once test cases are executed, we also collect test execution data, how many test cases were executed, and how many passed, failed, suspended or blocked. Therefore, we have test oracles, test execution resulting data in the shape of test execution logs and traceability matrix. Once testing is complete, we maintain clean-up information. Another important aspect is the storage of automated test cases, bug logs, screenshots etc. We keep record of all of these to infer valuable information such as:

- When to enforce exit
- Product quality metric
- Expected completion time

We emphasize on aspects of test data management considering automated testing tools providing support for test case recording and execution, bug log recording and analysis of results.

Test Planning with MTM

How to perform test planning with MTM?

We undertake steps involved in Software Testing Lifecycle (STLC) from system under test (SUT) analysis, test planning till test cycle closure. Application analysis is the first step in test planning. In application analysis we have a web based system under test that includes:

- Learning management system offering online lectures, assessments and practice sessions
- Teachers can plan their schedule.
- Students can access online content for learning
- Campus management system to provide support for related activities

In test planning, we develop test plan, test suites and test cases. For this purpose, we make use of Microsoft Test Manager (MTM). In MTM, we will perform following steps:

1. Create a new test plan.
2. Inside test plan, create a new test suit.
3. Inside a test suit, create a new test case.
4. To create a test case we add a test requirement and add a test case to test that requirement.
5. A complete web based system test planning implementation is performed in your course lecture video. For further understanding regarding test planning steps in MTM, you can watch this video link: *https://www.youtube.com/watch?v=12nbJ_MDirM*

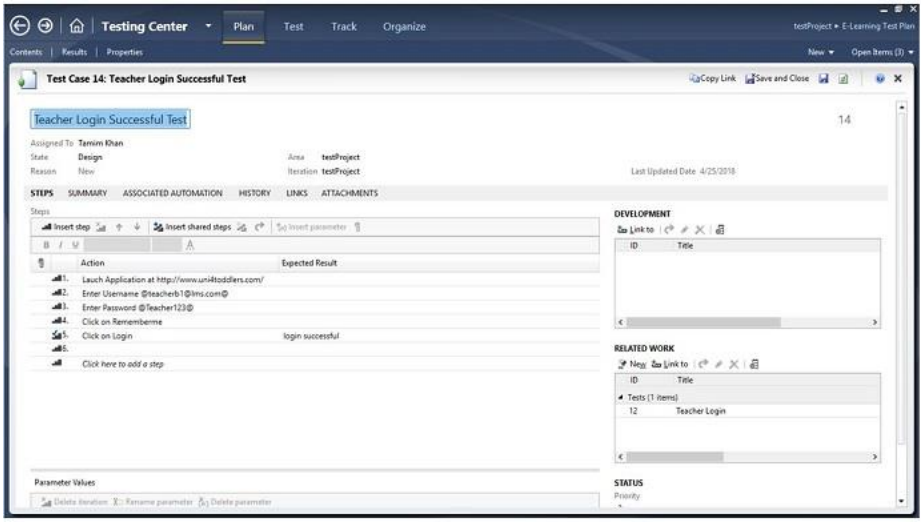
Test Case Development with MTM

We undertake steps involved in Software Testing Lifecycle (STLC) as follows:

1.
    1. System under test (SUT) analysis
    2. Test planning
    3. Test case development
    4. Test environment setup
    5. Test execution
    6. Test cycle closure

A test case is developed in MTM by following certain steps which are as follows:

1. A test case is developed for some requirement. So, after adding requirement in MTM we can add a new test case for that particular requirement using requirement interface in MTM. Inside requirement interface, press “New” button to add a new test case.
2. Once a test case is created, select that particular test case to open, update and manage.
3. Opening a test case will give you an interface containing details of test case as shown below



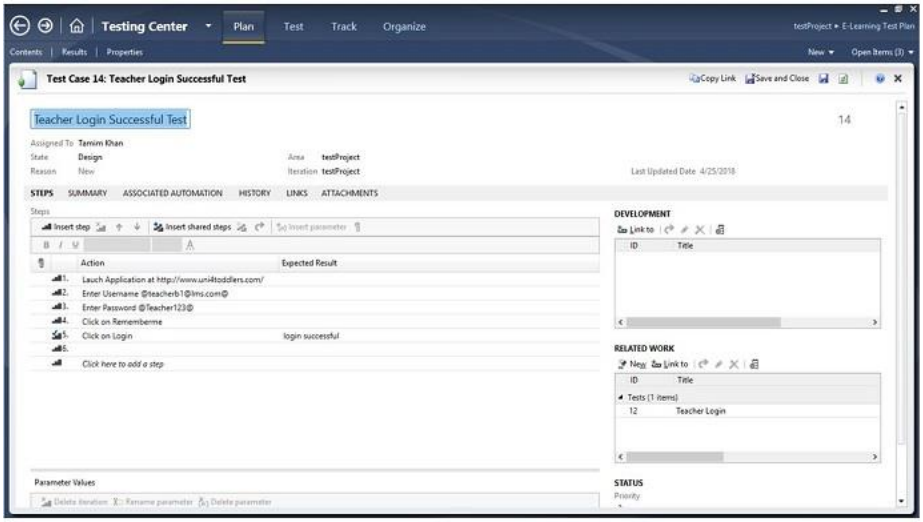
Test Case Development with MTM

We undertake steps involved in Software Testing Lifecycle (STLC) as follows:

1.
    1. System under test (SUT) analysis
    2. Test planning
    3. Test case development
    4. Test environment setup
    5. Test execution
    6. Test cycle closure

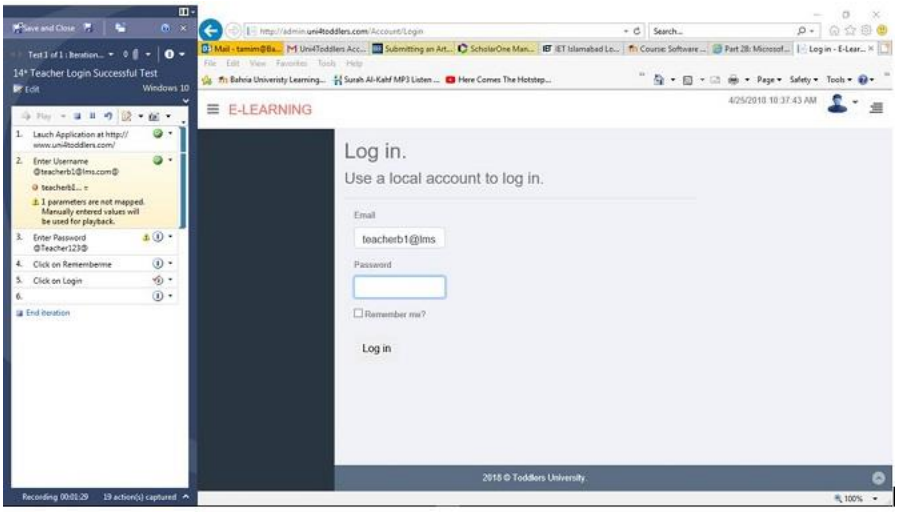
A test case is developed in MTM by following certain steps which are as follows:

1. A test case is developed for some requirement. So, after adding requirement in MTM we can add a new test case for that particular requirement using requirement interface in MTM. Inside requirement interface, press “New” button to add a new test case.
2. Once a test case is created, select that particular test case to open, update and manage.
3. Opening a test case will give you an interface containing details of test case as shown below



Test Execution Recording With MTM

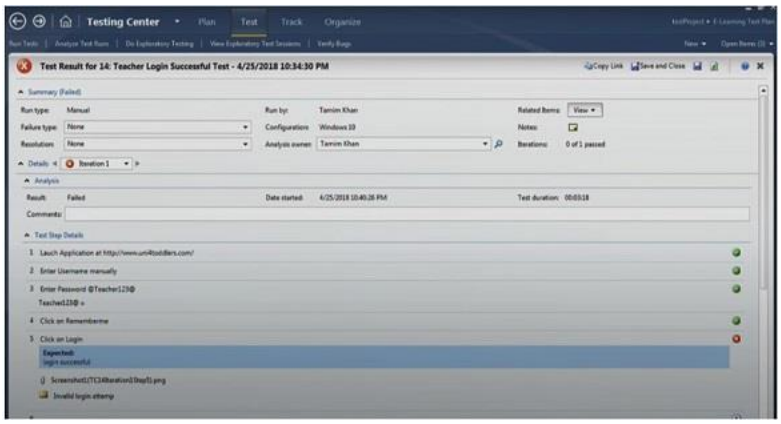
Once a test case is developed, the next step is to execute and record a test case. As test execution begins, an execution window will appear on left side and the web page under test will launch. The test execution window will display each step with its execution status. It also has the options to pause and resume recording and to capture any part of testing in form of screen shot. A test case can be updated while it is in execution mode. To update a test case, the recording will be paused and after updating the test case the recording can be resumed. In case a test case fails, MTM will record the results in form of screen shot which help developers to figure out cause of failure. Once the test case is fixed by developers, the tester will re-run that test case instead of executing it because it is already recorded. The test execution interface is shown below:



Test Execution Reporting

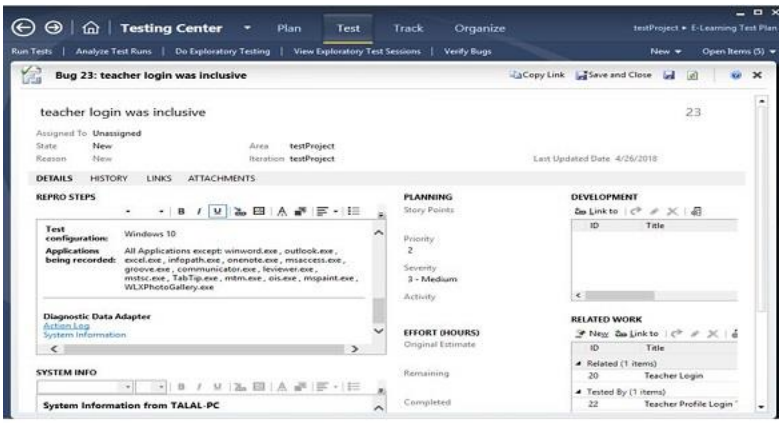
A test suit can have multiple test cases. Once these test cases are executed, test execution report can be generated consisting of execution results as pass/fail with execution steps of each test case

and reasons for test case failure. In case a test case fails, test execution reporting helps developers to track that particular test case. Text execution reporting interface in MTM is shown below:



Bug Logs with MTM

Bug logs in MTM shows reported bugs with details including title, bug status, created by and many other filters on the basis of which a developer can identify and track bugs reported at different times by different testers. Each reported bug also has detailed summary and action log shown in the figure below:



MTM provides the feature to apply different queries on bug logs to check the status of bugs. There are different views of bug logs to check if the bugs are fixed. View a bug log as “created by me” means bugs that are reported by particular tester. Similarly view as “Assigned to me” means bugs that are assigned to particular tester for correction.

Test Cycle Closure with MTM

Test cycle closure is the final step of the software testing lifecycle (STLC). Before closing a testing cycle, the tester should analyze test runs, number of bugs fixed, number of open test runs, etc. After analyzing the data, the tester will verify if all bugs are fixed, all issues raised are handled and all the requirements are tested. After verification, the test cycle can be closed.

Regression Testing

Introduction to regression testing:

Regression testing usually refers to testing activities during the software maintenance phase. This testing activity occurs after software changes.

Objective of regression testing:

The major objective of regression testing is to retest changed components (or parts) and check the affected parts (or components) of the software after changes.

Why does software evolve?

Software evolves because of requirements or design changes due to the addition of new functionality, modification in existing functionality, design changes, bug fixing, and maintenance. We need to validate if the changes have not caused any regression in the rest of the system.

Levels of regression testing:

Regression testing can be performed at the unit level, re-integration level, functional level, and system level.

Regression testing implementation via an example:

Let P be a procedure or program, let P' be a modified version of P, and let T be a test suite for P. A typical regression test proceeds as follows:

1.
    1. Select *T' ≤ T, a set of test cases to execute on P'*.
    2. Test *P'* with *T'*, establishing *P'*"s correctness concerning *T'*
    3. If necessary, create *T''*, a set of new functional or structural test cases for *P'*
    4. Test *P' with T'' establishing P''s correctness concerning T''.*
    5. Create *T'''*, a new test suite and test execution profile for *P'*, from *T*, *T'*, and *T''*

Each step in the regression testing procedure maps to a certain testing step. In the above example, the first step is the regression test selection problem. The second and fourth steps address the test suite execution problem. The third step addresses the coverage identification problem. The fifth step addresses the test suite maintenance problem, the problem of updating and storing test information.

Regression Testing Requirement

Why do we need regression testing?

Regression testing is required to verify and validate if changes in one part of the system have not impacted non-changed parts of the system. Also, the cost of finding change should not exceed rerunning the whole of the test suit.

Who performs software regression?

Regression testing at the unit level or integration level is performed by developers. Test engineers perform regression testing at the function level and QA and test engineers perform regression testing at the system level.

Regression testing artifacts:

Different artifacts are needed to perform software regression testing. Major artifacts include software change information (change notes), updated software requirements and design specifications, user manuals, software regression testing process, strategies, methods, and criteria.

How Changes Effect SUT

Identify software changes:

To determine how changes affect the system under test (SUT), firstly we need to identify different types of changes which include specification changes, design specification changes, implementation (or source code) changes, user manual changes, environment or technology changes, and test changes.

Impacts of software changes:

Change in one part of the software system may affect other parts. So, we need to identify the impacts of software change which includes REQ impacts, design impacts, implementation impacts, user impacts, and test impacts.

Classification of test suit after requirements change:

When change occurs in a software system, we need to select a subset of test cases from the existing test suite to perform testing after the change. Three types of test cases exist in a test suit which includes

1. Re-testable test case
2. Reusable test case
3. Obsolete test case

Major regression testing problems:

-
- How to use a systematic method or tool to identify changed software parts?
- How to use a systematic method or tool to identify software change impacts?
- How to use a systematic method or tool to identify affected software test cases?
- How to reduce the re-test suites?
- How to select the test cases in a test suite?
- How to minimize re-testing efforts, and achieve adequate testing coverage?

Effects of Requirement Change in Regression Testing

- Effects of changes in requirements and activities of a software system that may induce changes such as adding new function features, changing current function features, deleting existing function features, system architecture, and design changes. Similarly, changes in components include changing in component interactions, adding new components or subsystems, updating and deleting existing components.

Effect of Maintenance in Regression Testing

- **Effects of maintenance:**
- While maintaining a software system, change in any part of the system is possible. It can be a change in state-based behaviors, component interfaces, database design, GUI design, function design, change in the low-level design document, and change in algorithm logic and component structure. Maintenance also leads to changes in components, internal data types and names, internal structures such as class relationships, control flow, and data flow, internal functions, component interface, call signatures, message interactions, protocol messages, and formats. Change in software technology and/or language is also possible in the maintenance phase.

Fault Seeding
Calculating confidence C

C = 1 if n > N and
C = S/(S - N + 1) if n < N

Where

S is the number of seeded faults,

N is the total number of non-seeded (indigenous) faults which can be found

We calculate N by

N = S*n / s

Where n is actual number of faults detected during testing and s is the number of seeded faults detected during testing.

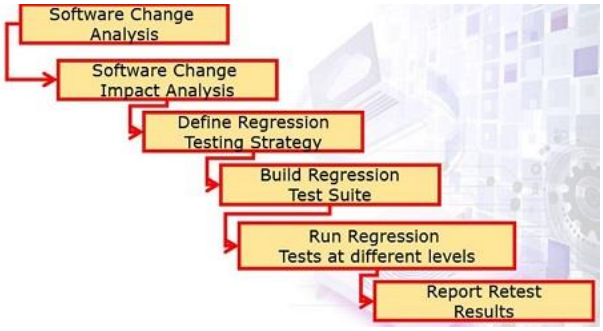
C = 1 since N=0, n=2 => n > N

Software Verification and Validation

Impact of Bug Fixing in Regression Testing

Bug fixing example:

Developers do bug fixing and they may induce further bugs into the system. For example, developer X needed to make a change in the backend. He was required to add a column to one of the tables he was working with. The development manager approved that change and forgot to disseminate information to the rest of the team. Developer Y had tested his code concerning that particular table while developer X's code is working, the code of developer Y has stopped working without his knowledge. As soon as developer Y handles the effect of change impacting his work, his code would again start working.



Regression testing method:

Methods of regression testing work based on change identification and test selection through test case minimization and test case prioritization.

Methods in Regression Testing

We study a couple of methods for regression test selection. Test case selection for regression testing requires prior knowledge of application type. Methods include:

-
- Re-test all (one extreme)
- Modification based selection methods
- Coverage based selection methods
- Random retesting (another extreme)

Why regression testing is application-specific?

Imagine you are working on a nuclear power plant, medical equipment, banking application, or attendance system. Your application domain will suggest the regression testing technique you will select for that particular application. The techniques can be “retest all” or “selective retesting”.

Regression testing examples by different researchers:

Extended finite state machine considers control and data dependence to construct a state dependence graph which is used to identify the effect of modification (Chen et al, 2006).

Model-based regression testing uses data dependence analysis whereas an extended finite state machine is used for dependence analysis in object-oriented systems for reducing regression test suites (Korel et al., 2002).

UML designs are considered where pre-and post-conditions are given as object constraint language (OCL) constraints and the changes in the model are analyzed by analyzing their XML representation (Briand et al. 2009)

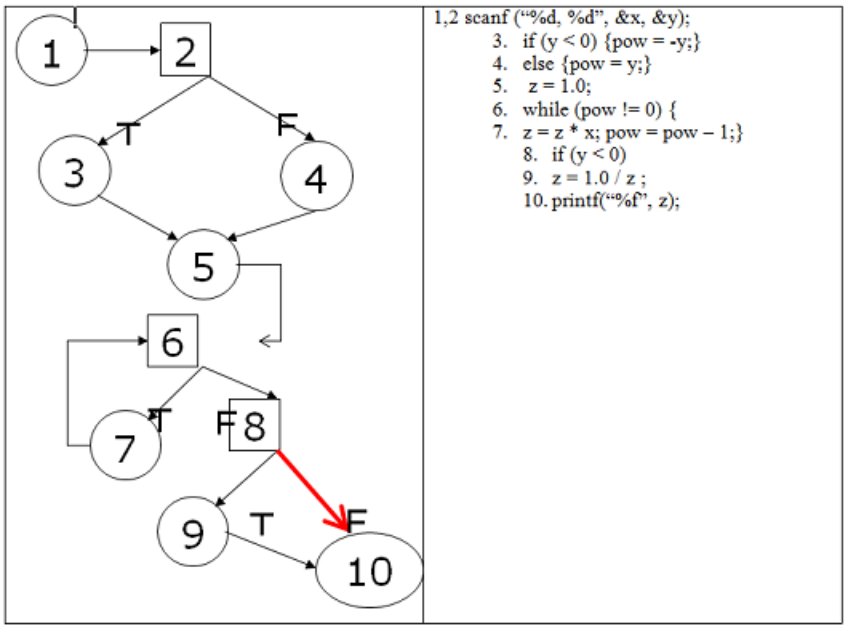
Analyze visual contracts to identify the impact of change and select test cases into subsets and perform coverage analysis to see if the resulting test suite retains coverage. Analyze the information in terms of visual contracts where our focus is how the change affects the model. We also need to know which of the code artifacts were accessed during an update. Our focus is different as we consider one web service and not compositions (Khan TA et al, 2012)

Research conclusion:

Evaluate the change in the model through visual contracts and do not consider control dependence. The analysis is based on visual contracts analysis and not on extended finite state machines and the application domain is web services. Consider visual contracts to represent pre and post-conditions formalizing them as rules in a TAGTS for analysis and propose a regression testing approach for web services.

Role of Test Case Prioritization for Regression Testing

Test case prioritization helps us order previously run test cases according to our requirements. Let our requirement considering unit test cases is to reorder test cases such that the one providing maximum coverage concerning nodes is selected first. Considering a previous example from module 35.




```
1,2 scanf ("%d, %d", &x, &y);
    3.  if (y < 0) {pow = -y;}
    4.  else {pow = y;}
    5.   z = 1.0;
    6.  while (pow != 0) {
    7.  z = z * x; pow = pow – 1;}
        8.  if (y < 0)
        9.  z = 1.0 / z ;
        10. printf("%f", z);
```

Test cases for paths 1,2,4,5, 6, 8,9,10 and 1,2,3,5, 6,7,6,8,9,10 give complete node coverage. Yet edge 8→10 is not covered. We need following additional test case 1, 2,3,5,6, 8, 10.

For edge coverage our test cases are:

-
  - 1,2,4,5, 6,8,9,10
  - 1,2,3,5, 6,7,6,8,9,10
  - 1,2,3,5,6, 8,10

If we prioritize them according to number of nodes covered, they would become

-
  - 1,2,3,5, 6,7,6,8,9,10
  - 1,2,4,5, 6,8,9,10
  - 1,2,3,5,6, 8,10

Test case prioritization techniques:

Different test case prioritization techniques are as follows:

1.
   I. Sort test cases in order of increasing cost per additional coverage
   II. Select the first test case
   III. Repeat the above two steps until n test cases are selected

Cost decisions can be based on individual test case cost and cumulative formula.

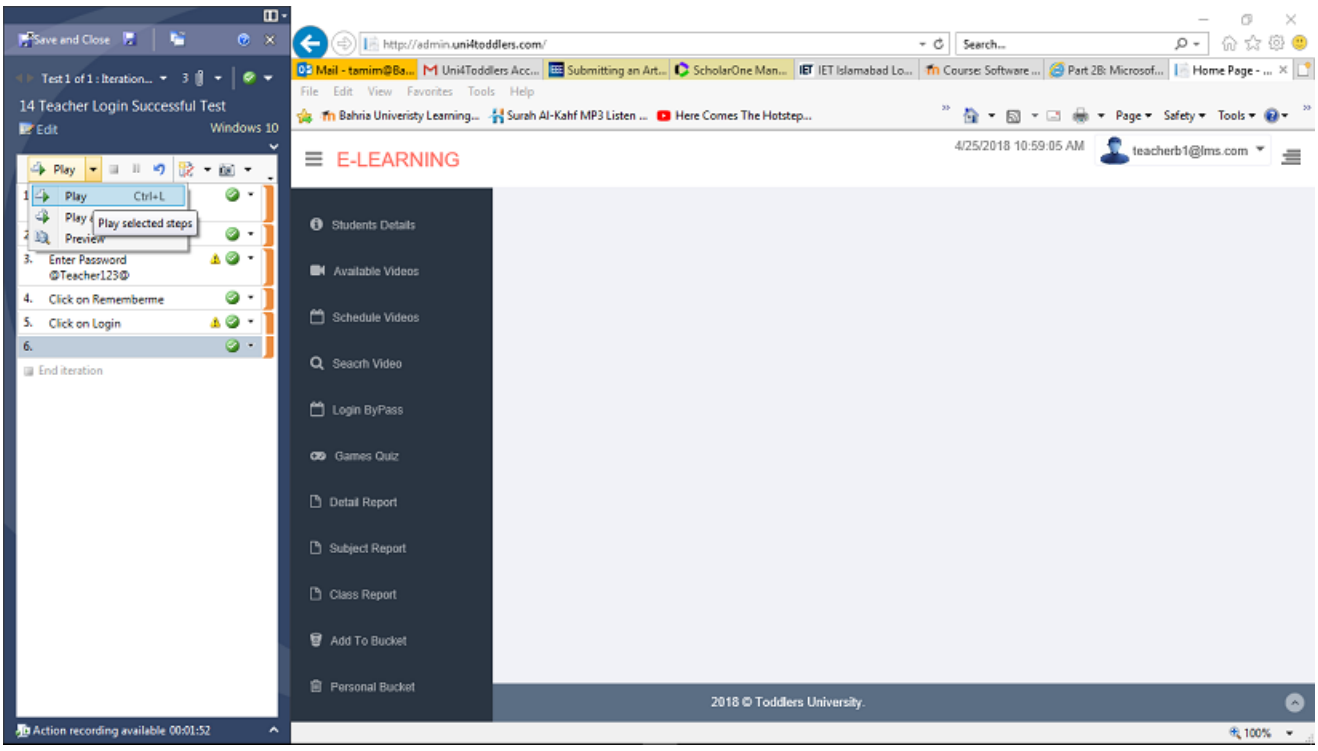
Test Case Recording For Regression Testing

Based on our knowledge about our test suite and our information about the affected area or the target area of the software under test i.e., SUT, we select test cases for regression testing. Now we consider the case where we do minimal effort on finding out the affected area of SUT and test case prioritization or minimization. Record test case execution, *somehow*, well! Why somehow? The details will be covered in the next few modules.

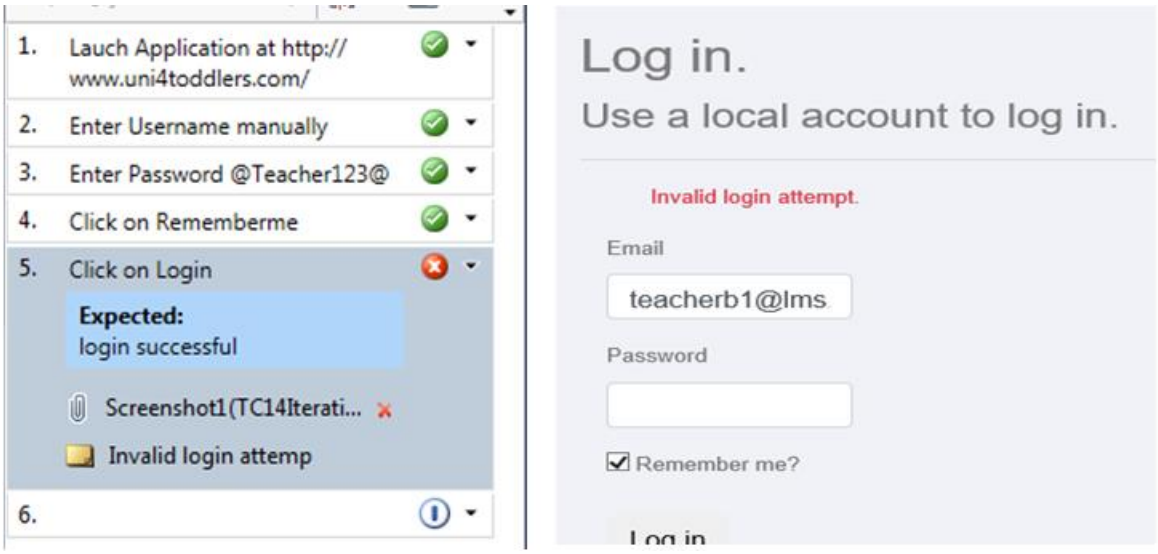
Test Case Execution Recording With MTM

Once a test case is executed and recorded for the first time, the results are stored as test results. Recording a test case helps testers reduce re-testing costs and make the testing process more efficient. To rerun a recoded test case, we need to perform the following steps in MTM.

Figure 1 shows the MTM interface. In the left pane, we have “Teacher login successful test”. To re-run this test for regression testing we press the play button as shown in figure 1.



We already have an executed test case recording with results. When a recorded test case is played, it will show step-by-step test results as pass/fail for the selected test module “Login” as shown in the figure below.



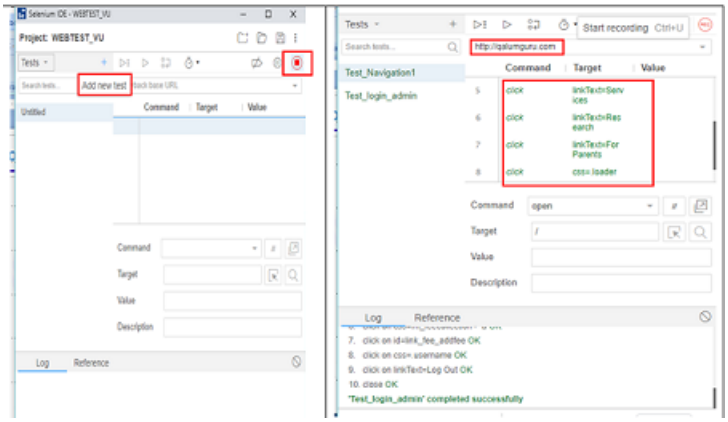
Test Case Recording With Selenium

Selenium is a set of tools that supports the rapid development of test automation for web-based applications. It works where JavaScript is supported. It supports languages e.g., Java, Ruby, Python. Selenium can simulate a user navigating through pages and then assert for specific marks on the pages.

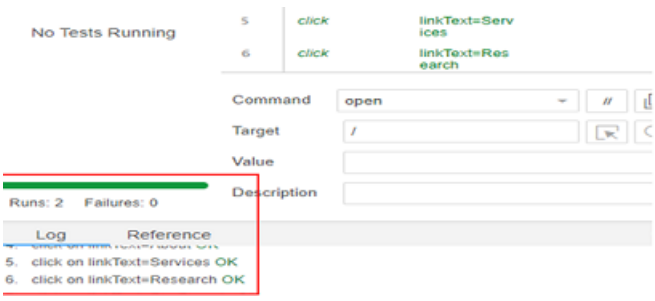
Test case recording with selenium:

Following steps are performed to record a test case execution with selenium

1. Selenium IDE is managed as an Add-ons in a web browser where we have to browse our web application. Once it is added, the first step is to create a new project and name it.
2. Inside a new project interface, add a new test case for a web application.
3. Once a test case is added, press the record button to start the test execution recording.
4. Selenium interface will display test execution steps and URL of the web application under test as shown below.



After execution of all steps, stop test recording. Selenium interface will display the test results with the test log as shown below.



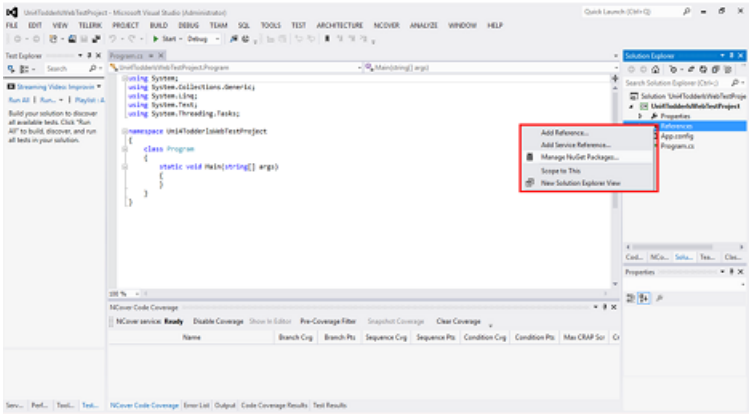
Now, these test cases are available as a test suite and they can be run with a click. We can rerun them anytime and see if there is any regression in the overall system quality.

Selenium as Add-on from IDE

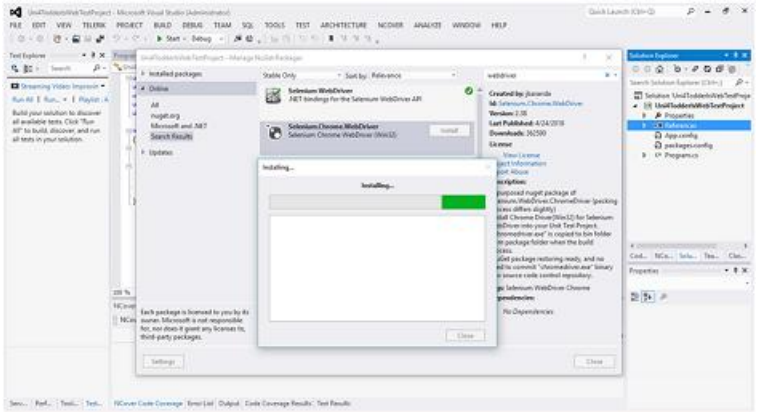
We can run recorded test cases using IDE e.g., Visual Studio, and develop a test suite. We can run these test cases using a selenium driver from IDE.

Example of test case written in C#:

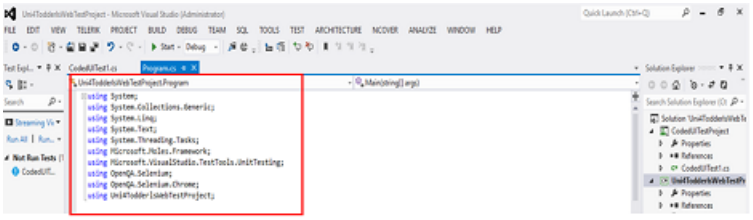
We take an example of a test case written in C# to perform unit testing using selenium IDE in visual studio. For this purpose, selenium IDE will be used as an add-on inside the visual studio using the "NuGet Packages" option as shown below.



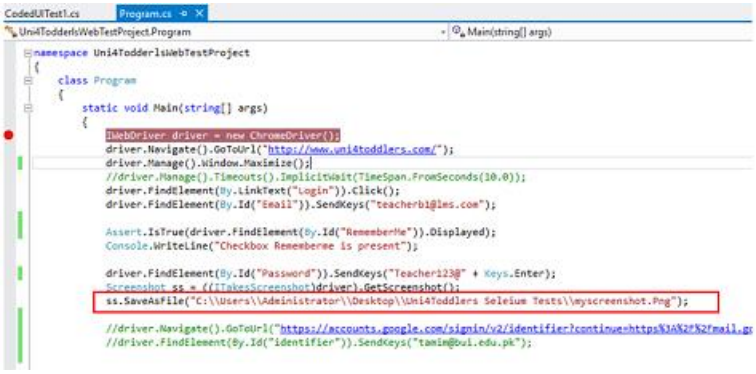
Inside the "NuGet Packages" window, install "Selenium WebDriver" and "Selenium.Chrome.WebDriver" as shown below.



After installation, the packages will be included in the unit test code so that it can be performed via selenium IDE instead of a visual studio testing framework.



C# code for unit testing of website module using selenium IDE packages is shown below in which the test case recording is stored in the form of a screenshot.



```
namespace Uni4ToddlersWebTestProject
{
    class Program
    {
        static void Main(string[] args)
        {
            IWebDriver driver = new ChromeDriver();
            driver.Navigate().GoToUrl("http://www.uni4toddlers.com/");
            driver.Manage().Window.Maximize();
            //driver.Manage().Timeouts().ImplicitWait(TimeSpan.FromSeconds(10.0));
            driver.FindElement(By.LinkText("Login")).Click();
            driver.FindElement(By.Id("Email")).SendKeys("teacherb1@lms.com");

            Assert.IsTrue(driver.FindElement(By.Id("RememberMe")).Displayed);
            Console.WriteLine("Checkbox Rememberme is present");

            driver.FindElement(By.Id("Password")).SendKeys("Teacher123@" + Keys.Enter);
            Screenshot ss = ((ITakesScreenshot)driver).GetScreenshot();
            ss.SaveAsFile("C:\\Users\\Administrator\\Desktop\\Uni4Toddlers Seleium Tests\\myscreenshot.Png");

            //driver.Navigate().GoToUrl("https://accounts.google.com/signin/v2/identifier?continue=https%3A%2F%2Fmail.g
            //driver.FindElement(By.Id("identifier")).SendKeys("tamin@bui.edu.pk");
```

The recorded screenshot is used for regression testing. Now, these test cases are available as a test suite. They can be run with a click. We can rerun them anytime and see if there is any regression in the overall system quality.

Web Testing

Difference between testing and web testing:

Testing is the process of exercising software with the intent of finding errors whereas **web testing** is the testing that focuses on the testing of web applications.

Web testing checklist:

1. **Functional testing part:** In functional testing, we need to check that each component is functioning as expected or not, so it is also called "Component Testing".
2. **Usability testing part:** In usability testing, we ensure that all possible test cases which target the audience of the web application are done regularly.
3. **Compatibility testing:** It ensures how an application is working in the supported environment.
4. **Database testing:** Web application is thoroughly tested to ensure data reliability.
5. **Crowd testing part:** Crowd testing is when a large group of perfect strangers try your product then give you phenomenally helpful feedback on usability, bugs, and features.
6. **Interface testing part:** In the interface testing mainly three areas should be covered: web server, application server, and database server.
7. **Performance testing:** Performance testing is to check that web application is working under heavy load.
8. **Security testing:** The security testing is carried out to ensure any information leakage in terms of encrypting the data.

Web testing scenarios:

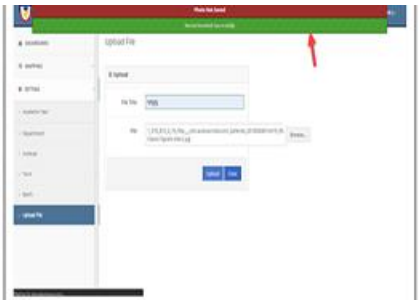
Different web testing scenarios are as follows:

- User navigation to different pages of the website and completing the end-to-end workflow
- If the user can select/deselect checkboxes
- If the user can select values from dropdown fields
- Different navigation buttons like submit, next, upload, etc. buttons are working well.
- Calendars are loading properly and allowing the user to select a date
- Calculations are happening as implemented
- Search functionality is working if any
- If the user can select/deselect radio buttons

"Qalum guru" test case example:

We are testing the "Qalum guru" website's upload video feature. The test case for "Upload Video" is as follows

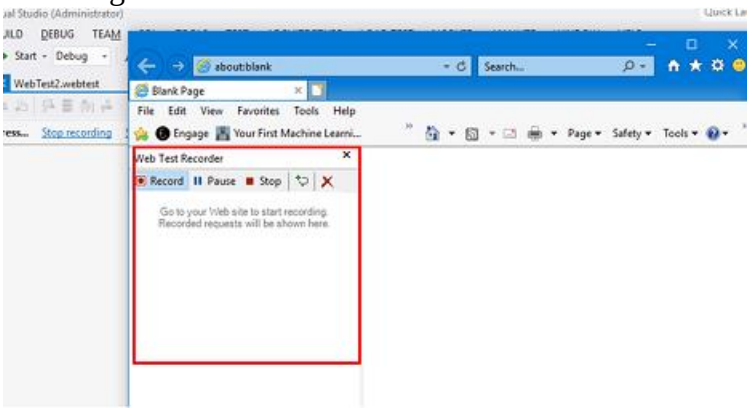
Test case	Verify File upload with valid Data
Test case steps	1- Enter URL http://qalumguru.com/index.html. 2. Log in as a student. 3. Click 'Setting' tab 4. Click Upload File button. 5. Enter File name. 6. Select File. 7. Click upload button. 8. Verify if video is uploaded.
Expected Output	Video should be uploaded
Actual Output	Video is not uploaded
Result	Fail



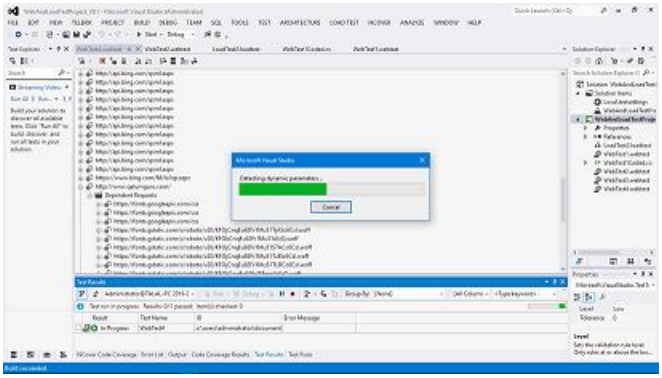
Web Test Recording With Microsoft Visual Studio

We can record a test case as a web test case using Microsoft Visual Studio. These test cases serve many purposes. One purpose is to record them for rerunning. Steps to record a web test in a visual studio are as follows.

1. Inside visual studio C# interface, create a new web performance and load test project and give it a name.
2. Inside the project created in step 1, add a web performance test case. For this purpose, right-click on the project name, from the "Add" option choose "Web Performance Test".
3. Once a test case is added, press starts recording. It will open a web browser for recording the desired web page.
4. The web browser interface has an add-on recording pane on the left side containing all recording controls as shown below.

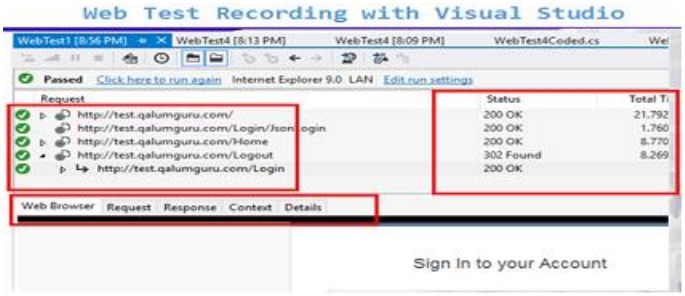


5. After pressing the start recording button, the first step is to write the URL of the web page under test. It is worth noting that while recording if the internet went off the explorer launched Bing and record it as a step until the problem is resolved. We remove these unwanted steps once we have all the recording steps.
6. The web page under test will open. Perform testing steps and remove unwanted steps and press stop recording.
7. Once the recording is stopped. Visual studio will load dynamic parameters of the test case and save it as a test case recording as shown below.

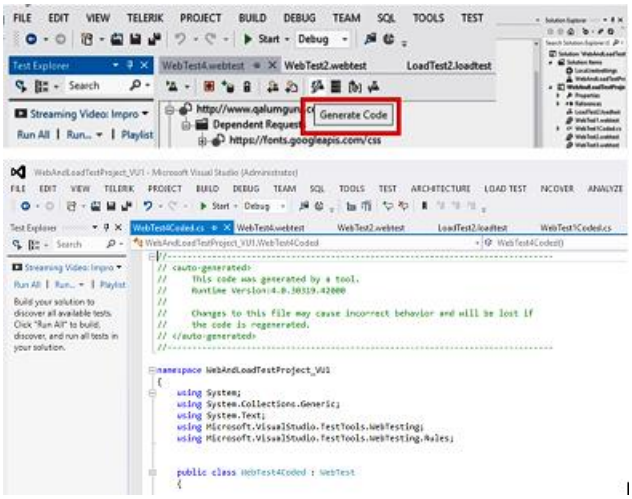


8. Right-click on the recorded test case and press "Run" to execute it. The visual studio interface will display test execution with test execution details and execution status as

shown below.



9. To maintain this test case recording and to use it as a module in other test cases, we can generate its code for editing from the "Generate Code" option. The generated code for the test case recorded in the above steps is shown below.



Load Testing

What is load testing?

Load testing is a type of software testing which is conducted to understand the behavior of the system under test (SUT) under a specific expected load. It is a type of non-functional testing which is performed to determine a system's behavior under both normal and at peak conditions.

Requirements to perform load testing:

Load Testing is performed considering requirements such as type of load test, user load estimation and distribution, user activity analysis, production environment analysis, database size, report generation, and simple metric collection.

We determine the goal of the test and the perspective in which we intend to find the synchronization problems or we intend to perform a load test or a stress test or a performance test.

Types of Load Tests

Different types of load test with examples are discussed as follows:

- **User load estimation:** We try to find out system behavior when subjected to expected load. For example, you are developing a system for Uber and you would like to see how many parallel transactions your SUT would support.
- **User load distribution:** Considering the same Uber application, in load distribution, you would like to see system behavior since the user load may be at peak in the morning or afternoon and very less in the evening. This may also help determine the concurrent users and simultaneous users.
- **User activity analysis:** There are a variety of functions that your SUT can perform. For example, in the morning there may be 200 forms submitted or in the evening most of the users may log in and perform results generation activity so in the morning scenario out of 1000 concurrent users, 70% may be performing this activity. You would like to see system behavior under different sets of activities performed by users.
- **Production environment analysis:** Each connection requires X amount of RAM and disk quota for your application. This may input valuable information such as analyzing if the environment has enough hardware or is it running in a clustered environment?
- **Database size analysis:** The database size does matter in the load test. Bulky the database, more the latency. Database administrators update PCT-Used, PCT-Free depending upon how frequent updates or addition/deletion activities are.

Load/Stress/Performance Testing

Load testing:

Load test determines how the system under test (SUT) performs under the concurrent user sessions for a typical user scenario. Load testing subjects system under a calculated load to see performance degradation. It is a process of exercising the system under test by feeding it the largest tasks it can operate with. In load testing, testers constantly increase the load on the system via automated tools to simulate real-time scenarios with virtual users. Examples of load testing include testing a word processor by editing a very large document or testing a web application in terms of concurrent users or HTTP connections.

Stress testing:

Stress testing examines how an application behaves under maximum load. In simple terms find the upper threshold for the application below which it can work normally. We try to break the system under test by overwhelming its resources or by taking resources away from it. Our purpose is to make sure that the system fails and recovers gracefully.

Performance testing:

Performance test indicates response time for the entire application from the user's perspective. It is the process of determining the speed or effectiveness of a computer, network, software program, or device. In performance testing, the following factors are considered:

- **Throughput:** The capability of a product to handle multiple transactions in a given period. It represents several requests/business transactions processed by the product in a specified time duration.
- **Response Time:** Finding out how much time each of the transactions took to complete. It is defined as the delay between the point of request and the first response from the product.
- **Tuning:** The procedure by which product performance is enhanced by setting different values to the parameters of the product, operating system, and other components. It improves the product performance without having to touch the source code of the product.
- **Benchmarking:** System boundaries concerning several transactions, parallel connections, etc.

Load Testing With Visual Studio

We can record a test case as a load test case using Microsoft Visual Studio. These test cases serve many purposes. One purpose is to record them for rerunning. Steps to record a load test in the visual studio are as follows:

1. Inside visual studio C# interface, create a new web performance and load test project and give it a name.
2. Inside the project created in step 1, add a load test case. For this purpose, right-click on the project name; from the "Add" option choose "Load Test".
3. A load test wizard will open containing all necessary steps in the left pane window. The steps include

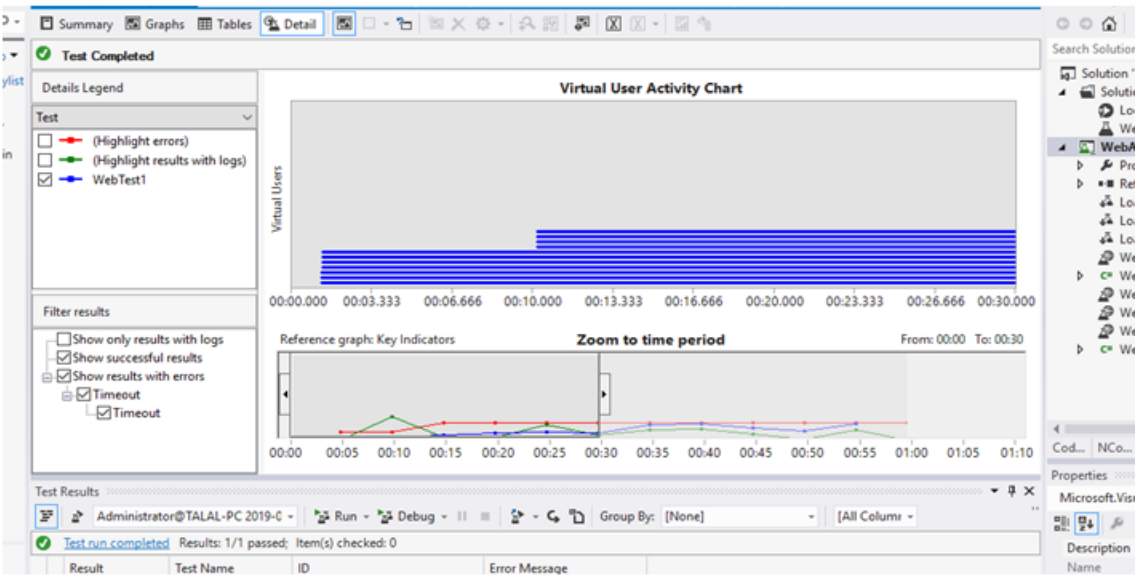
- Scenario
- Load pattern
- Test mix model
- Test mix
- Network mix
- Browser mix
- Counter sets
- Run settings

4. One by one, perform all necessary settings in each step according to the application perspective on which you are performing load testing.
5. Once the wizard is finished, right-click on the test case and click "Run Load Test".
6. As the load test starts executing, it will show a graphical representation of different aspects including

- Key indicators
- Page response time
- System under test

- Controller and Agents

7. Once the time duration of the load test is complete, it will display results in graphical form which includes a virtual user activity chart concerning the time as shown below.



Software Test Metrics

Introduction to metric:

Metric is a standard unit of measurement that quantifies results.

Software metric:

Metric used for evaluating the software processes, products, and services are termed as software metrics. A software metric is a measurement-based technique applied to processes, products, and services to supply engineering and management information and work on the information supplied to improve processes, products, and services if required.

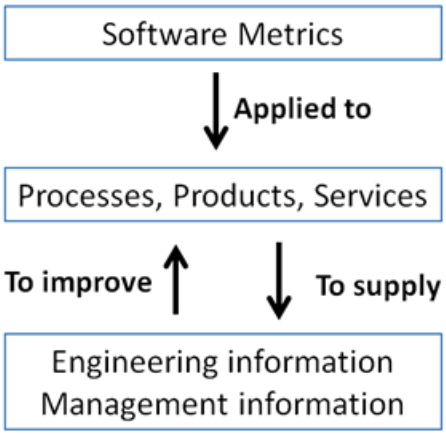
Aim of collecting test metric:

The aim of collecting test metrics is to use the data for improving the test process, rather than just showing fancy reports. This includes finding tangible answers to the questions like

-
  - How long will it take to test?
  - How much money will it take to test?
  - How bad are the bugs?
  - How many bugs found were fixed, reopened, closed or deferred?
  - How many bugs did the test team not find?

- How much of the software was tested?
- Will testing be done on time? Can the software be shipped on time?
- How good were the tests? Are we using low-value test cases?
- What is the cost of testing?
- Was the test effort adequate? Could we have fit more testing in this release?

Software metrics cycle diagram:



Information derived from a software test metric:

A software test metric provides information regarding the total number of test cases, test cases passed/failed/blocked, total defects found, defects accepted/rejected, defects deferred, critical defects, number of planned test hours and actual test hours, bugs found after shipping.

We derive important information to measure how effective was our test process and how it can be further improved?

Importance of Test Metrics

Why do we need test metrics?

We need test metrics to track and control phases of testing and to find out if the system is ready for release. Test metrics are required to improve our in-house process and to find out if our testing process is efficient and when to stop testing. We also need test metrics to do trend analysis and to meet client expectations and customer needs.

Purpose of data collection and analysis in software metrics:

While collecting data and while performing analysis, we need to find out if the whole process was meaningful within the context of our organization and system under test (SUT). Data collection and analysis help us understand the impact of the metrics and the variance. It improves the reliability and consistency of testing. Data analysis makes testing activity economical to measure

and hence we have means for comparing our results to some benchmark or process capability baseline.

Test metrics classification:

We need to classify the test metric for quality, productivity, cost, defect, coverage, effort, stability, and effectiveness.

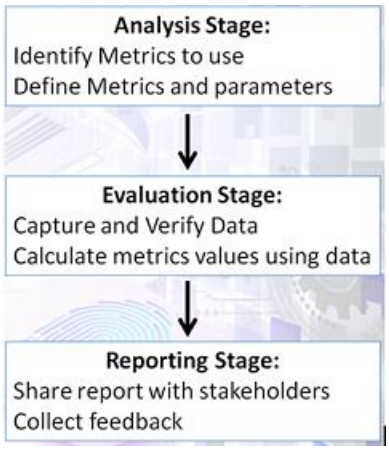
Test Metrics Lifecycle

Metrics are used to improve the quality and productivity of products and services thus achieving customer satisfaction. It is easy for management to digest one number and drill down if required. Different metric(s) trends act as monitor when the process is going out of control. Metrics provide improvement for the current process.

Test metrics lifecycle stages:

There are three stages of the test metrics lifecycle.

1. **Analysis stage:** In this stage, metrics are defined and identified to use and metrics parameters are defined.
2. **Evaluation stage:** In this stage, data is captured and verified. Metrics are calculated using captured data.
3. **Reporting stage:** In this stage, a report is shared with stakeholders and their feedback is collected.



Test Metrics Classification

There are three major classifications of test metrics.

1. **Manual testing metrics:** In manual testing metrics we analyze test case productivity, test execution summary and productivity, defect acceptance, rejection and severity, test efficiency, and bad fix defect.

2. **Performance testing metrics:** In performance testing metrics we analyze performance scripting productivity, performance execution summary, performance efficiency, and severity index.
3. **Automation testing metrics:** In automation testing metrics we analyze automation scripting and test execution productivity, automation coverage, and cost comparison.

General/Common metrics: Other metrics include analysis of effort and schedule variance and scope change.

Defect Related Metrics

Defect acceptance:

Defect acceptance helps determine the number of valid defects that the testing team identified during execution. The value of this metric is compared with previous releases for a clearer picture. The formula for calculation of defect acceptance is as follows:

$$Defect\ Acceptance = \left[\frac{Number\ of\ Valid\ Defects}{Total\ Number\ of\ Defects} * 100\right]\%$$



Defect rejection:

This metric determines the number of defects rejected during execution. It gives the percentage of the invalid defect the testing team has opened and one can control, whenever required. The formula for the calculation of defect rejection is as follows:

$$Defect\ Rejection = \left[\frac{Number\ of\ Defect(s)\ Rejected}{Total\ Number\ of\ Defects} * 100\right]\%$$

Bad fix defect:

Defects whose resolution gives rise to new defect(s) are bad fix defects. This metric determines the effectiveness of the defect resolution process. It gives the percentage of the bad defect resolution which needs to be controlled. The formula for calculation of bad fix defect is as follows:

$$Bad\ Fix\ Defect = \left[\frac{Number\ of\ of\ Bad\ Fix\ Defect(s)}{Total\ Number\ of\ Valid\ Defects} * 100\right]\%$$

Defect severity index:

This metric determines the quality of the product under test and at the time of release, based on which one can decide to release the product i.e. it indicates the product quality. The formula for calculation of defect severity index is as follows:

$$Defect\ Severity\ Index = \left[\frac{\sum (Severity\ Index * No.\ of\ Valid\ Defect(s)\ for\ this\ severity}{Total\ Number\ of\ Valid\ Defects} \right]$$

One can divide the Defect Severity Index (DSI) into two parts.

1. **DSI for All Status defect(s):** This value gives the product quality under test.
2. **DSI for Open Status defect(s):** This value gives the product quality at the time of release. For calculation of DSI for this, only open status defect(s) must be considered.

$$DSI(Open) = \left[\frac{\sum (Severity\ Index * No.\ of\ Open\ Valid\ Defect(s)\ for\ this\ severity)}{Total\ Number\ of\ Open\ Valid\ Defects} \right]$$

Test Efficiency Metrics

Test execution summary:

It gives the classification of the test cases concerning execution status along with the reason, if available, for various test cases. Test execution summary gives a statistical view of the release. It also provides data for the number of test cases executed and status for pass/fail with the reason for failure which includes postponed defect, setup issue, out of scope.

Test case productivity:

This metric gives the test case writing productivity based on which one can have a conclusive remark. Its formula is

$$Test\ Case\ Productivity = \left[\frac{Total\ Raw\ Test\ Steps}{Efforts\ (hours)} \right] Step(s)/hour$$

For example test case productivity will be 120 / 8 = TCP = 15 steps/hour for the data given in the table below.

Test Case Name	Raw Steps
1	30
2	32
3	45
4	13
Total Raw Steps	120
Total Hours	8

Test execution productivity:

This metric gives the test cases execution productivity which on further analysis can give a conclusive result. One can compare the productivity with the previous release and can have an effective conclusion.

$$Test\ Execution\ Productivity = \left[\frac{Total\ No.\ of\ TC\ executed\ (Te)}{Execution\ Efforts\ (hours)} * 8 \right] Execution(s)/Day$$

Test efficiency:

This metric determines the efficiency of the testing team in identifying the defects. It also indicated the defects missed out during the testing phase which migrated to the next phase.

$$Test\ Efficiency = \left[\frac{DT}{DT + DU} * 100 \right] \%$$

where

DT = Number of valid defects identified during testing

DU = Number of valid defects identified by the user after the release of the application i.e., post-testing defect.

Performance Testing Metrics

Performance testing is a software testing process used for testing the speed, response time, stability, reliability, scalability and resource usage of a software application under a particular workload. The main purpose of performance testing is to identify and eliminate the performance bottlenecks in the software application. It is a subset of performance engineering and is also known as “Perf testing”. The focus of performance testing is checking a software program’s

- Speed – Determines whether the application responds quickly
- Scalability – Determines the maximum user load the software application can handle.
- Stability – Determines if the application is stable under varying loads

Performance scripting productivity

This metric gives the scripting productivity for performance test script and has trend over a period of time.

Performance Scripting Productivity= [Operations performed/Efforts(hours)]Operation(s)/hour

where operations performed is: -

1.
    1. No. of click(s) i.e., click(s) on which data is refreshed.
    2. No. of input parameter
    3. No. of correlation parameter

Above evaluation process does include logic embedded into the script which is rarely used.

Performance execution summary

Performance execution summary gives classification with respect to a number of tests conducted along with status (Pass/Fail), for various types of performance testing. Some of the types of performance testing are as follows: -

1. **Peak volume test**: In this type of testing place a load on the server for a short time to emulate the heaviest demand that would be generated at peak user times

2. **Endurance/Soak test**: Involves testing the system to detect performance-related issues such as stability and response time by requesting the designed load on a system.

3. **Break-point/Stress test:** This is negative testing where a performance tester loads the software with a large number of concurrent users/processes and tries to fail the server resources. The break-point test has a gradually increased workload pattern without any upper limit.

4. **Fail-over test:** Fail-over testing is a technique that validates if a system can allocate extra resources and backup all the information and operations when a system fails abruptly due to some reason. This test determines the ability of a system to handle critical failures and handle extra servers.

Performance execution data - client-side

This metric gives detailed information on client-side data for execution. These are some of the data points of this metric, running users, response time, hits per second, and throughput, total transaction per second, time to first byte and error per second.

Performance execution data - server-side

This metric gives detailed information on the server-side data for the execution. These are some of the data points of this metric CPU utilization, memory utilization, heap memory utilization and database connections per second.

For more details go through the following link.

Ref: https://www.guru99.com/performance-testing.html

Performance Test Efficiency Metrics

In software engineering, performance testing is necessary before marketing any software product. It ensures customer satisfaction & protects an investor's investment against product failure. Costs of performance testing are usually more than made up for with improved customer satisfaction, loyalty, and retention.

Performance Test Efficiency

This metric determines the quality of the performance testing team in meeting the requirements which can be used as an input for further improvisation, if required.

Performance Test Efficiency = [(Req met during PT) - (Req met after signoff of PT)/Req met during PT * 100]%

To evaluate this one need to collect data point during the performance testing and after the sign-off of the performance testing.

Performance Severity Index

This metric determines the product quality based performance criteria on which one can make a decision for releasing the product to the next phase i.e. it indicates the quality of the product under test with respect to performance.

Performance Severity Index = [(Severity Index * No. Of Req. Not met for this severity)/Total no. Of Req. Not met]

If the requirement is not met, one can assign the severity for the requirement so that decision can be taken for the product release with respect to performance.

Automation Testing Metrics

Test automation makes software testing easier, faster and more reliable, and is essential in today's fast-moving software delivery environment. Usually seen as an alternative to time-consuming and labor-intensive manual testing, test automation uses software tools to run a large number of tests repeatedly to make sure an application doesn't break whenever new changes are introduced.

Implementing a test automation strategy within your organization is not for the faint of heart and you need to rely on well-chosen metrics that measure the performance (past, present, and future) of your automated testing process to determine if your company is getting an acceptable return on its test automation investment.

Implementing automated testing is a process, and any metrics chosen to measure improvement (e.g., The number of manual versus automated tests) need to take into account unique aspects of the organization, market, or environment they are being used in. There is no set of "universal metrics" reports that will work in every capacity all the time.

Automation Scripting Productivity

This metric gives the scripting productivity for automation test script based on which one can analyze and draw a most effective conclusion from the same.

Automation Scripting Productivity = [Operations Performed/Efforts(hours)]Operation(s)/hour

Where Operations performed is: -

-
  - No. of click(s) i.e., click(s) on which data is refreshed.
  - No. of input parameter
  - No. of checkpoint added

Above process does include logic embedded into the script which is rarely used.

Automation Test Execution Productivity

This metric gives the automated test case execution productivity.

Auto.Execution Productivity = [Total No. Of Automated TC executed (ATe)/Execution Efforts(hours) * 8] Execution(s)/Day

Where ATe is calculated as

ATe = Base Test Case + ((T(0.33)*0.33) + (T(0.66)*0.66)+(T(1)*1))

The evaluation process is similar to manual test execution productivity.

Automation Coverage

This metric gives the percentage of manual test cases automated.

Automation coverage= [Total No. Of TC Automated/Total No. Of Manual TC *100]%

Example

If there are 100 manual test cases and one has automated 60 test cases then automation coverage = 60%.

Cost Comparison

This metric gives the cost comparison between manual testing and automation testing. This metric is used to have conclusive ROI (return on investment). If the script is re-used the script development cost will be the script update cost.

Common Metrics for all type of Testing

Software testing metrics are a way to measure and monitor your test activities. More importantly, they give insights into your team's test progress, productivity, and the quality of the system under test. When we ask ourselves "What have we tested?" Metrics will give us better answers than just "we have tested it." Different teams measure various aspects depending on what they want to track and control or improve. A metric usually conveys a result or a prediction based on a combination of data.

Effort variance metric: This metric gives the variance in the estimated effort. The difference between the planned outlined effort and the effort required to actually undertake the task is called effort variance.

Effort variance = [Actual Effort - Planned Effort)/Planned Effort * 100]%

Schedule variance: This metric gives the variance in the estimated schedule i.e., the number of days. Any difference between the scheduled completion of an activity and the actual completion is known as schedule variance.

Schedule variance = [Actual calendar days – Planned calendar days) + Start variance)/ Planned calendar days x 100] %.

Scope change: This metric indicates how stable the scope of testing is. The variation metric helps us measure how much the requirements change during a sprint when compared with the original scope.

Scope Change = [Total Scope – Previous Scope)/ Previous Scope x 100] %

where

-
  - Total Scope = Previous Scope + New Scope, if scope increases
  - Total Scope = Previous Scope - New Scope, if Scope decreases

Different metrics comes with their defined analysis pattern. Therefore, it's crucial to select the right metrics as per the software requirement. Using software testing metrics is a productive and efficient way to track and fix defects early.

Penetration Testing

Penetration testing or pen testing is a type of security testing used to cover vulnerabilities, threats and risks that an attacker could exploit in software applications, networks or web applications. The purpose of penetration testing is to identify and test all possible security vulnerabilities that are present in the software application. Penetration testing is also called pen test. Our scope for this course is software applications under test (SUT).

Vulnerability is the risk that an attacker can disrupt or gain authorized access to the system or any data contained within it. Vulnerabilities are usually introduced by accident during the software development and implementation phase. Common vulnerabilities include design errors, configuration errors, software bugs etc. Penetration Analysis depends upon two mechanisms namely Vulnerability Assessment and Penetration Testing (VAPT). With the increased cyber-attacks, performing security testing is important. Penetration testing helps identify vulnerable areas of the system. It involves intentional attacks on SUT to identify weak areas, which might provide a passage to malicious or unauthorized users for attacking the system and altering its integrity and veracity. Helpful in fixing security-related defects/loopholes.

Benefits of Penetration Testing:

-
  - It exposes weaknesses before real hackers do
  - It can reveal which areas of security we need to invest in
  - It provides an outsider perspective on our security
  - It simulates a real attack scenario

Different forms of Penetration Testing:

1.
   1. Network vulnerability testing
   2. Web vulnerability testing
   3. Wireless war driving/walking
   4. Phone network testing
   5. Social engineering testing
   6. Walk-through and dumpster diving
   7. Physical security auditing.

Testers should act like a real hacker and test the application or system and needs to check whether a code is securely written. A penetration test will be effective if there is a well-implemented security policy. Penetration testing policy and methodology should be a place to make penetration testing more effective.

Penetration Testing Phases

The type of penetration test selected usually depends on the scope and whether the organization wants to simulate an attack by an employee, network admin (internal sources) or by external sources. There are three types of penetration testing and they are:

- Black box testing
- White box penetration testing
- Grey box penetration testing

In black-box penetration testing, a tester has no knowledge about the systems to be tested. He is responsible to collect information about the target network or system.

In a white-box penetration testing, the tester is usually provided with complete information about the network or systems to be tested including the IP address schema, source code, OS details, etc. This can be considered as a simulation of an attack by any internal sources (employees of an organization).

In grey box penetration testing, a tester is provided with partial knowledge of the system. It can be considered as an attack by an external hacker who had gained illegitimate access to an organization's network infrastructure documents.

Penetration Testing Steps/Phases:

1. **Scanning / Reconnaissance**

   Getting to know the target using passive methods. The idea of this phase is to gather as much info about the subject as we possibly can

   Methods:

- Search engine queries to gather data about the personnel, systems, or technologies of the client.
- Social Engineering to find out positions, technologies, and email addresses.
- Internet foot printing looking for email addresses, social accounts, names, positions.
- Dumpster diving to find valuable data that may be used for attacks or social engineering.
- Tailgating to get physical access or pictures with hidden cameras.

2. **Threat modelling**

   A description or model of all the security concerns and why they should be resolved. Once we feel we have sufficient info about the client's systems, we start modelling the threats that the client would realistically face and identify vulnerabilities that will allow for those attacks. It's kind of a pre-attack phase in which we get everything ready.

3. **Vulnerability analysis**

   Identifying vulnerabilities and determining their severity.

4. **Exploitation**

   Gaining access by breaching the security of a system or finding a bug to exploit in the software.

5. **Post-exploitation reporting**

   Detailing the vulnerabilities found and providing information on potential impact.

Types/Methods of Penetration Testing

A penetration test is a process where a skilled security tester attempts to find weaknesses, and breach the security of your systems. Ethical Hacking is also known as penetration testing or pen testing, is legally breaking into computers and devices to test an organization's computer security system.

Manual Penetration Testing:

Manual penetration testing is the ideal choice for deep app inspection as it detects bugs easily missed out by automated tests. MPT, carried out by a penetration tester or red-team, highlights the most challenging authorization or business logic flaws that automated software might fail to identify.

It involves a standard approach with different activities to be performed in a sequence as follows.

- Penetration Testing Planning
- Reconnaissance
- Vulnerability Analysis
- Exploitation
  - External Attacks
  - Internal Attacks
- Post Exploitation
- Reporting

Automated Penetration Testing: Automated penetration testing uses automated tools and the adoption of automated processes to execute pen tests. For example, rather than going through lines of codes to check for errors, a scanner can be deployed to scan through the codes in a short time.

Penetration Testing Tools

Some of the most commonly used tools include the following.

- **NMap**: This tool is used to do port scanning, OS identification, Trace the route and for Vulnerability scanning.
- **Nessus**: This is a traditional network-based vulnerabilities tool.

- **Metasploit**: The Metasploit framework is a ruby-based, modular penetration testing platform that enables you to write, test, and execute exploit code.
- **Wireshark**: The Wireshark free vulnerability scanner relies on packet sniffing to understand network traffic, which helps admins design effective countermeasures. If it detects worrisome traffic, it can help to determine whether it's an attack or error, categorize the attack, and even implement rules to protect the network.
- **Veracode**: Veracode is a modular, cloud-based solution for application security.

Read more at: https://www.appknox.com/blog/automated-penetration-testing-vs.-manual-penetration-testing

Penetration Testing Tools

Penetration testing tools are specific to either mimicking a subset of attacks. e.g., Sqlninja, as the name, indicates it is all about taking over the DB server using SQL injection in any environment. John the Ripper is a password cracker that works on most of the environments, although it's primarily for UNIX systems.

The wide division of these tools is as follows:

Network Vulnerability Assessment Tools

1. **Nmap**: Nmap is one of the well-known free and open-source network scanning tools among many security professionals. Nmap uses the probing technique to discover hosts in the network and for operating system discovery. This feature helps in detecting vulnerabilities in single or multiple networks. If you are new or learning with vulnerabilities scanning, then Nmap is a good start.
2. **OpenVAS**: An open-source tool, openvas can be used as a central service providing effective vulnerability assessment tools. These help in vulnerability scanning and in vulnerability management as well.
3. **Wireshark**: Widely used network protocol analyzer, a really powerful scanning tool. Checks networks at a microscopic level, captures issues online while the analysis is done offline, runs on all platforms- Windows, Linux, macOS etc.
4. **Metasploit**: The Metasploit framework is a Ruby-based, modular penetration testing platform that enables you to write, test, and execute exploit code.
5. **MBSA (Microsoft baseline security analyzer)**: An entirely free vulnerability scanner created by Microsoft, it's used for testing your windows server or windows computer for vulnerabilities.

Web Vulnerability Assessment Tools

1. **OWASP ZAP**: OWASP ZAP (short for Zed Attack Proxy) is an open-source web application security scanner. It is intended to be used by both those new to application security as well as professional penetration testers.

2. **Acunetix**: Acunetix is a professional tool for complete web security coverage. Automatic assessment of security vulnerabilities is part of its core vulnerability analysis functionality. Malware protection for web servers is not enough to ensure complete IT security.
3. **Burp Suite**: Burp Suite Professional is one of the most popular penetration testing and vulnerability finder tools, and is often used for checking web application security. "Burp," as it is commonly known, is a proxy-based tool used to evaluate the security of web-based applications and do hands-on testing.
4. **Nikto**: The Nikto web server scanner is a security tool that will test a website for thousands of possible security issues. Including dangerous files, misconfigured services, vulnerable scripts and other issues.
5. **Sqlmap**: SQLMAP is an open-source penetration testing tool written in python to detect and exploit SQL Injection flaws. SQL Injection is an injection flaw where user input could alter the SQL query being constructed and executed by the application. SQLMAP automates common SQL injection techniques like Error Based.

Types of Pen Testing/Testers

Black hat: With no prior knowledge of network, computer or application that needs to be tested.

White hat: With complete knowledge of network, computer or application that needs to be tested.

Grey hat: With limited knowledge of network, computer or application that needs to be tested.

Read more: https://phoenixnap.com/blog/best-penetration-testing-tools

Penetration Testing or Hacking

Difference between a Pen Tester and a Hacker?

Penetration testers have prior approval from senior management and penetration tester's social engineering attacks are there to raise awareness on the other hand; Hackers have prior approval from themselves and hackers social engineering attacks are there to trick systems into divulging sensitive information about the whereabouts of their estranged ex-spouse.

Ethical Hacking	Penetration Testing
"Ethical hacking focuses on using all techniques to find and exploit the vulnerabilities."	"Pen testing focuses on finding all vulnerabilities in the network, computer or application."
Main target is to break into the system	Main target is to defend the system against any threat
Ethical hacking is an offensive measure	Pen testing is a defensive measure

Vulnerability Assessment

A vulnerability assessment is a systematic review of security weaknesses in an information system. It evaluates if the system is susceptible to any known vulnerabilities, assigns severity levels to

those vulnerabilities, and recommends remediation or mitigation, if and whenever needed. Unreliable at times and high rate of false positives. Vulnerability assessment invites debate among system admins. Produces a report with mitigation guidelines and action items.

Penetration Testing

A penetration test, also known as a pen test, is a simulated cyber-attack against your computer system to check for exploitable vulnerabilities. In the context of web application security, penetration testing is commonly used to augment a web application firewall (WAF). Pen testing can involve the attempted breaching of any number of application systems, (e.g., application protocol interfaces (APIs), frontend/backend servers) to uncover vulnerabilities, such as unsanitized inputs that are susceptible to code injection attacks. Insights provided by the penetration test can be used to fine-tune your WAF security policies and patch detected vulnerabilities.

Benefits of Penetration Testing

1. Enables a proactive security approach.
2. Allows exploration of real risks and accurate representation of a company's IT infrastructure security posture at any given time.
3. Helps investigate data breaches or network intrusions to discover any leads to the leakage of data or theft of intellectual property.
4. Able to assess and validate the efficacy of one's defensive mechanisms

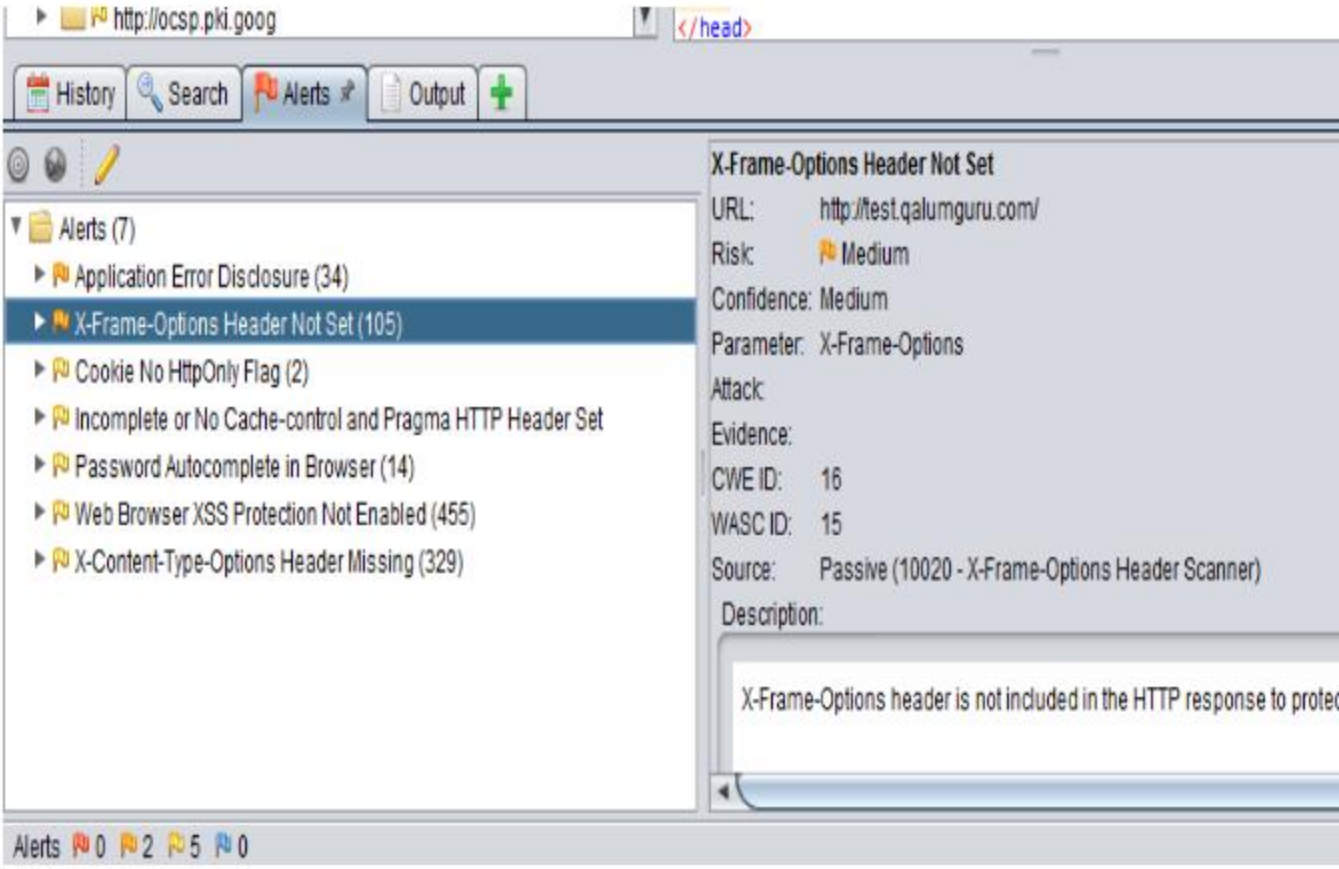
Disadvantages of Penetration Testing

1. It is a time consuming activity.
2. It is high-labor intensive and can therefore represent an increased cost
3. It might give a false sense of security. Being able to withstand most penetration testing attacks might give the sense that systems are 100% safe.

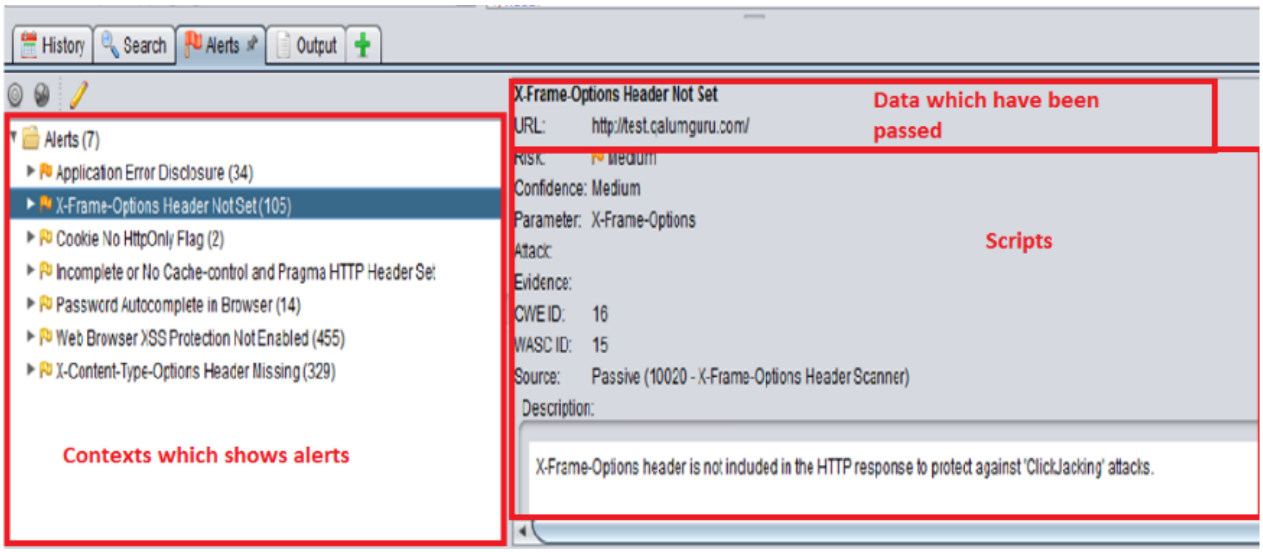
4. Penetration Testing: Worked Example

5. **Qalum Guru**
6. Zed Attack Proxy is an open-source web application security scanner. The application was taken and scanned against the ZAP OWASP scanner. As a result, scanner provides a list of potential vulnerabilities that can cause potential harm to the system, if exploited. The non-disclosure agreement (NDA) is one of the important documents to be signed before the penetration test to safeguard the interests of the concerned parties.

7. 



8. Now you can see; that we have performed automated penetration testing on the application qalumguru.com. Its subdomains are scanned automatically and provide the list of problems as alerts which are as follows:

9. 

10. On the leftmost side, these are the contexts that show 7 alerts.
11. The upper right side shows the data which has been passed and below this section shows scripts.
12. As you can see, the number of alerts application error disclosures is 34, and Password auto complete in the browser indicates that it may cause issues.

13. Exploratory Testing

Functionalities are checked in adhoc manners in Exploratory testing

14. Exploratory testing is a style of software testing that emphasizes personal freedom and responsibility of individual tester to continually optimize the quality of his/her work by treating test-related learning, test design, test execution, and test result interpretation as mutually supportive activities that run in parallel throughout the project." (Cem Kaner). The focus of exploratory testing is more on testing as a "thinking" activity. This testing is useful when requirement documents are not available or partially available.
15. It is about exploring and understanding what software can do or cannot do. Test cases are not created in advance but testers check the system on the fly. Both test design and test execution are performed simultaneously. There is no formal documentation for test scripts, test conditions, or test cases.
16. Test logging is performed at the same time as test execution. During test logging, we document the key aspects of what functionality is tested, any defects found, and any thoughts regarding further testing. It is not a technique but it is an approach. What actions you perform next are governed by what you are doing currently.
17. In software engineering, exploratory testing is performed to overcome the limitations of scripted testing. It helps in improving test case suite. It empathizes with learning and adaptability.

Most appropriate methods for regression testing test selection for general business application include?

Exploratory Testing Process

Following is a step by step process on how to do exploratory testing.

Four phases in exploratory testing:

1. **Learn**

   In this phase, we have to identify what to test, how it can be tested, and what factors to consider. Describe the starting point of testing and define how users will utilize the system.

2. Test Design

Brief test descriptions rather than test cases are prepared.

3. Test Execution

In this phase run the test execution, gather and compile the test results as well as check for needed additional testing.

4. Analysis

In this phase, we have been analyzing what has been learned about the product? What has been learned about the testing? And analyze coverage areas.

Exploratory Testing vs Structured Testing

Structured Testing:

Structural testing, also known as glass box testing or white box testing is an approach where the tests are derived from the knowledge of the software's structure or an internal implementation. The other names of structural testing include clear box testing, open box testing, logic driven testing, or path driven testing. In structured testing, we design test cases first and later proceed with test execution.

Exploratory Testing:

In exploratory testing, testers do not work based on previously created test cases. They check a system without a plan in mind to discover bugs that users may face when navigating a website or app without a specific aim or direction in mind. In exploratory testing, test design and test execution is a simultaneous process that is done in parallel.

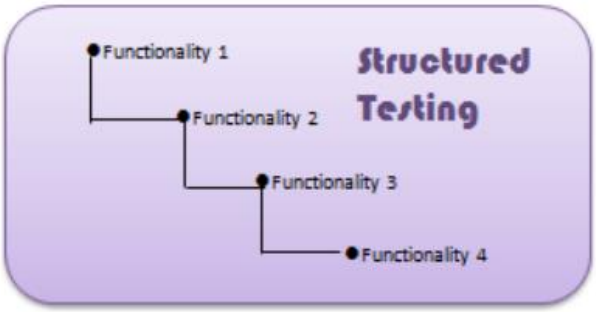
Most appropriate methods for regression testing test selection for general business application include? Choose the correct option.

both modification based and coverage based selection methods

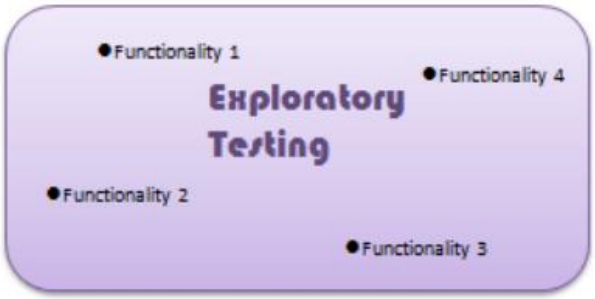
modification based method

coverage based method

re-test all



Functionalities are checked in a structured manner



Functionalities are checked in a ad-hoc manner

Structured Testing	**Exploratory Testing**
Directed from requirements	Directed from requirements and exploring during testing
Determination of test cases well in advance	Determination of test cases during testing
Confirmation of testing with the requirements	Investigation of system or application
Emphasizes prediction and decision making	Emphasizes adaptability and learning
Involves confirmed testing	Involves Investigation
Is about Controlling tests	Is about the Improvement of test design
Like making a speech – you read from a draft	Like making a conversation – it's spontaneous

Exploratory Testing

Exploratory testing is an approach to software testing that is often described as simultaneous learning, test design, and execution. It focuses on the discovery and relies on the guidance of the individual tester to uncover defects that are not easily covered in the scope of other tests. There are some benefits and drawbacks of exploratory testing which are as follows:

Benefits of Exploratory Testing:

- Requires a minimal amount of planning.
- Testers continuously decide on their next step of testing, and the decision depends upon the tester's thought process.
- It helps to provide feedback to developers as soon as possible.
- A larger number of bugs or defects can be uncovered using this way of testing.
- Exploratory testing helps in exploring critical issues.
- In the case of unstable requirements, exploratory testing helps in testing the new environment within a limited time.
- It helps to promote real-time thinking and also helps to uncover more and more defects.
- It promotes use cases as well as scenario-based testing.
- It converts the minimal documentation process into maximum testing in a form because test cases are written by exploring an application.

Drawbacks of Exploratory Testing:

- Purely depends on tester skills.
- Limited by domain knowledge of the tester.
- Not suitable for long execution time.
- Some important defects may be missed.
- Difficulties repeating the tests.

Can sometimes test what was implemented not what was supposed to be implemented

Exploratory Testing Worked Example

We use Microsoft Test Manager (MTM). We have already used this previously to explain system testing. We use the same application for explaining exploratory testing

Steps to do exploratory testing through MTM

1. Open MTM interface
2. Click Test >to start exploratory testing without any selected work item click do exploratory testing>Click explore>Click start>Click Login.
3. The system will allow you to create test cases so click create a test case to create test cases, test cases will automatically generated, enter user name password and click the sign-in button.
4. You can set some things manually according to your choice like you can edit test case name and select the tester whom you want to assign, you have seen the automatically generated test case steps. Select priority of test case and click the save button.
5. Run tests and you will see the test cases on the right side of the panel.
6. Click create a bug to report the bug then you can see the new bug
7. create the bug id and attach a screenshot with this bug report.

Usability Testing

Usability testing is an effort to ascertain the degree to which software has met the usability needs of the intended user base. Usability testing is an attempt to quantify software user-friendliness according to the skills needed to learn the software, and the time required to become efficient in using the software. The measured increase in user productivity and a subjective assessment of a user's attitude toward using the software.

Usability testing is a technique used to evaluate a product by testing it on users. Test users will try to complete typical tasks while observers watch, listen, and take notes, we intend to identify any usability problems, collect quantitative data on participants' performance,anddetermine participants' satisfaction with the product.. How well a product looks usually determines how well it works.

There are many software applications/websites, which miserably fail, once launched, due to reasons such as:

-
- Where do I click next?
- Which page needs to be navigated?
- Which Icon or Jargon represents what?
- Error messages are not consistent or displayed
- Session time not sufficient

Usability Heuristics by J Neilson Explanation

1. Visibility of system status.

Rationale: To make sure that users understand what information exists in the collection and how to find the information.

Question examples

- When searching, is it clear where you are?
- How clearly does the website describe its contents?
- Does formatting and organization of web site help understand where you are?

2: Match between the system and the real world

Rationale: The digital library should have clear language and follow website conventions.

Question examples

- Are the terms used throughout the System under Test (SUT) easy to understand?
- Is it clear when icons are links?
- Is it clear what the search and browse options are?

3: User Control and Freedom

Rationale: The digital library website should make it easy for users to return to the home page and to find pages within the site.

Question examples

- Can users easily move through search results?
- The navigation was visible. Users could easily find the ‘back’ and ‘home’ buttons.

4. Consistency and Standards

Rationale: Users should be able to clearly understand the format and metadata terminology should be understandable and consistent throughout.

Question examples

- Was the format consistent?
- Does the site comply with conventions (such as how links look)?

5. Error prevention

Rationale: To prevent users from encountering errors or problems.

Question examples

- Does the website adequately prevent problems?
- When no search results are found, the interface presents the user with different browsing options.

6. Recognition rather than recall

Rationale: The website should not require users to remember information from previous pages to perform tasks.

Question examples

- A menu or list is provided on each page highlighting the possible options available to users

7. Flexibility and Efficiency of Use

Rationale: Ease of searching and browsing methods ensures users will visit more than once.

select the r8 option from the following Usability heuristics by J Neilson.

Question examples

- Users can easily browse the SUT.

8. Aesthetic / Minimalist Design

Rationale: Users should be able to view readable text and relevant information.

Question examples

- SUT has a simple and clean design.
- SUT uses readable fonts in colors that are easy to read for all users.

9. Help Users Recognize, Diagnose, and Recover from Errors

Rationale: It is important to keep the user informed at all times

Question examples

- The search results indicate when no results are found.
- A message is displayed.

10. Help and Documentation

Rationale: Help allows users to continue using SUT with limited knowledge and with online documentation.

Question examples

- If the documentation is available as a link
- If the help is useful for making effective use of the SUT.

Usability Testing Process

The usability testing process has the following steps:

1. Plan and prepare
2. Choose participants
3. Perform tests
4. Analyze results
5. Document recommendations

Plan and prepare: Develop a test plan for simple testing, prepare a list of questions, and for more detailed testing, have a script prepared. Create lists of tasks or questions that a typical user should be able to complete in an hour. Tasks should not be too simple nor too difficult to accomplish e.g., find a video lecture you want to study, play an educational game online, start an assessment, etc.

Find Participants: A challenging aspect of usability testing is finding suitable participants. It is important to gather an ongoing user base. Test outside the team testing with people who are not associated with your company or your website.

Conduct the Test: The user will be asked to perform a set of pre-defined tasks (but do not tell them how many or how long each will take). Make the user feel comfortable and speak only to give a new task and take notes during the process.

Analyze results: Finally, after you've collected all your data, it's time to analyze the results and make conclusions. Try to do this as soon as possible after testing so that the observations are fresh in your mind. As you go over the data, pull out the most serious or frequent problems that users encountered for further examination. Don't address every single thing that went wrong instead prioritize the issues that were most problematic and need to be workshopped and resolved.

Read more: Read the chapter on analyzing and evaluating usability testing data for a straightforward 5-step approach.

Document recommendation: Once you are done with the tests, compile the results for analysis. Summarize your findings in preset formats. Refer to the test recordings to fill any gaps in your findings. Since you have covered the formats and processes for the documentation beforehand, this step will occur naturally throughout the process of testing.

Examples of usability testing test cases:

The goal of this testing is to satisfy users and it mainly concentrates on the following parameters of a system:

Example 1: Effectiveness

- Is the system easy to learn?
- Is the system useful and adds value to the target audience?
- Are content, color, icons, and images used aesthetically pleasing?

Example 2: Efficiency

- Little navigation should be required to reach the desired screen or webpage, and scrollbars should be used infrequently.
- Uniformity in the format of screen/pages in your application/website.
- Option to search within your software application or website.

Example 3: Accuracy

- No outdated or incorrect data like contact information/address should be present.
- No broken links should be present.

Example 4: User Friendliness

- The controls used should be self-explanatory and must not require training to operate.
- Help should be provided for the users to understand the application/website.

Topic 144-170

Methods for Usability Testing

There are two methods available to do usability testing:

- Laboratory usability testing
- Remote usability testing

Laboratory usability testing:

This testing is conducted in a separate lab/ room in presence of the observers. The testers are assigned tasks to execute. The role of the observer is to monitor the behavior of the testers and report the outcome of the testing. The observer remains silent during testing. In this testing, both observers and testers are present in the same physical location.

Remote usability testing:

Under this testing observers and testers are remotely located. Testers access the system under test, remotely and perform assigned tasks. The tester's voice, screen activity, and testers' facial expressions are recorded by automated software. Observers analyze this data and report the findings of the test. It's faster and cheaper to do quantitative usability testing, so you're more likely to get statistically significant results.

Primary goal of usability testing

The primary goal of this testing is to find crucial usability problems before the product is launched. Example checklist includes:

- Start the user experience or UX testing during an early stage of design and development
- Select appropriate users to test the system. They can be experts / non-experts users/mix of experts and non-experts users etc.

Benefits and Drawbacks of Usability Testing

Usability testing (UX testing) is a non-functional testing method that measures how easily end–users can use the software. This testing is crucial because even though the app designers know the product, they can't say if the website presents it properly to the final users. Besides, the project team may understand the software, but external customers might not. Seemingly straightforward steps for you can be extremely difficult for the user.

Usability tests are usually performed in the first stage of the design phase during the development cycle. When done early, you can start troubleshooting any problems sooner, ultimately reducing the cost to resolve them. The longer you wait to run a usability test, the more the problem "soaks" into the design, and the more difficult it will be to eliminate it over time. Also, the problems could become rather expensive to fix.

Benefits of usability testing include:

1. It helps uncover usability issues before the product is marketed.
2. It helps improve end-user satisfaction.
3. It makes your system highly effective and efficient.
4. It helps gather true feedback from your target audience who use your system during a usability test. You do not need to rely on "opinions" from random people.

Drawbacks include:

1. Test results don't prove that a product/design/UI "works".
2. Maybe have a usability expert look at it, to find gross violations.
3. It's possible that a user interface has a learning curve but subsequently becomes extremely powerful and usable. That isn't anything a usability test can detect.
4. Doesn't tell you if the market wants/needs a product like yours.

Usability Testing Worked Example

We first develop a checklist of questions following heuristics by J Neilson

Subset of Checklist

- **Website:** What it is about? What is the benefit? What is next?
- **Navigation:** Is your navigation clear?
- **Search:** Is the search function available?
- **Layout:** Are buttons consistent? Are menus consistent?
- **Look and feel:** Is there enough white space? Is information placed appropriately?
- **Help:** Is documentation / help available?

We select a testing method “Laboratory Usability Testing” and our application is Qalum guru which is available at http://www.qalumguru.com.We start the testing process and testers test where observes observe the process and ask questions one by one and report findings.

Our findings against our checklist analysis gathered by the observer are:

- The website is about children's basic education.
- Navigation is easy and every page is connected to every page.
- There are no search options.
- Buttons are consistent.
- Information is appropriately distributed amongst pages.
- Online help is not available.
- Documentation is not there.

User Documentation Testing

- Software documentation was a readme.txt file copied onto the software’s floppy disk, a 1-page insert put into the shrink-wrapped package containing the software comments in the

source code. Testers ran a spell checker on the readme.txt file and that was about the extent of testing documentation. Much of the non-code is the software documentation, which requires much effort to produce.

- Documentation is now a major part of a software system. It might exceed the amount of source code. It might be integrated into the software (e.g., help system). Testers have to cover the code and the documentation. Assuring documentation is correct is part of a software tester's job.
- Testing documentation is the documentation of artifacts that are created during or before the testing of a software application. Documentation reflects the importance of processes for the customer, individual, and organization. Projects which contain all documents have a high level of maturity. Careful documentation can save the time, effort, and wealth of the organization. There is the necessary reference document, which is prepared by every test engineer before stating the test execution process. Generally, we write the test document whenever the developers are busy writing the code. Once the test document is ready, the entire test execution process depends on the test document.
- The primary objective of writing a test document is to decrease or eliminate the doubts related to the testing activities. If the testing or development team gets software that is not working correctly and developed by someone else, so to find the error, the team will first need a document. Now, if the documents are available then the team will quickly find out the cause of the error by examining the documentation. But, if the documents are not available then the tester needs to do black box and white box testing again, which will waste the time and money of the organization. More than that, a lack of documentation becomes a problem for acceptance.
- **Documentation styles:**
- Loosely coupled to the code: e.g., user manuals, packaging fliers.
- Tightly coupled to the code: e.g., documents are an integral part of the software, such as a training system, user manuals, etc.

Important Aspects of Documentation Testing

Good documentation and planning always lead to better quality software testing and products. Most successful organizations give a lot of emphasis to software testing documents since it is key to a successful project. In many cases, the project even gets rejected in the proposal/acceptance phase for lack of documentation. This can result in a bad name for the organization in the market. It would not be wrong to say that documentation is a bridge between the organization and the client.

Documentation aspects:

Some important aspects of documentation testing are as follows

-
  - Packaging text and graphics
  - Marketing material, ads, and other inserts
  - Warranty/registration
  - End-ser License Agreement (EULA)

- Labels and stickers
- Installation and setup instructions
- User's manual
- Online help
- Tutorials, wizards, and computer-based training
- Samples, examples, and templates
- Error messages

Benefits of documentation testing include:

1. Improves usability: Not all software was written for the Mac.
2. Improves reliability: Testing the documentation is part of black-box testing. A bug in the user manual is like a bug in the software.
3. Lowers support cost: The exercise of writing the documentation helped debug the system.
4. Project documents can make project testing easy and systematic
5. It saves organization time as well as cost
6. You can deliver a quality product to the client within the specified time limits
7. It leads to client satisfaction
8. Understand the scope better
9. Increased business opportunities with the same client.
10. Improved transparency with the client.
11. Maintain a good relationship with the client.

Read more: https://performancelabus.com/importance-of-software-testing-documentation/

Documentation Testing Process

Test documentation is documentation of artifacts created before or during the testing of software. The degree of test formality depends on:

1) The type of application under the test

2) Standards followed by your organization

3) The maturity of the development process. Important types of test documents are Test policy, Test strategy, Test plan, Test case, etc.

QA team needs to be involved in the initial phase of the project so that test documentation is created in parallel. The main reason behind creating test documentation is to either reduce or remove any uncertainties about the testing activities. The cost of the documentation may surpass its value as it is very time-consuming.

We consider the following aspects of the documentation testing process:

Audience: Keep in mind the audience who will use documentation so make sure documentation is not too novice or too advanced.

Terminology: Terminologies used in the documentation must be suitable for the audience, terms used consistently, and provide proper abbreviations for acronyms.

Content and subject matter: It includes whether the appropriate subjects are covered or not? No subjects missing? Proper depth? and missing features described accidentally?

Just the facts: All information technically correct? Correct table of contents, index, and chapter references? As well as correct website URLs, phone numbers?

Step by step: Must check if there are any missing steps in documentation? Compared tester results to those shown in the documentation?

Figures and screen captures: Must be accurate and precise? Are they from the latest version of the software? Are the figure captions correct?

Samples and examples: Do all the examples work as advertised? Spelling and grammar must be checked.

GUI (Graphical User Interface) Testing

A GUI (Graphical User Interface) or simply UI is a hierarchical, graphical front end to a software system. The first GUI is introduced in 1975. GUI as a means to use/interact with software systems. GUI is nowadays almost ubiquitous, even in safety-critical systems. GUI interacts with the underlying code by method calls or messages.

GUI testing is a process of verifying the GUI for its compliance with specifications, general principles, and specific project requirements. It involves simulating user actions like clicking buttons, following links, and other similar actions. In this way, developers check the product for its correct operation, the mutual interaction of components, and the UI usability as a whole.

GUI can exercise remote code. GUI responds to user events (e.g., mouse clicks), GUIs are event-driven systems. Testing GUI correctness is critical for system usability, robustness, and safety. The whole system can be executed using the GUI.

by means of GUI.

Read more: https://www.guru99.com/gui-testing.html

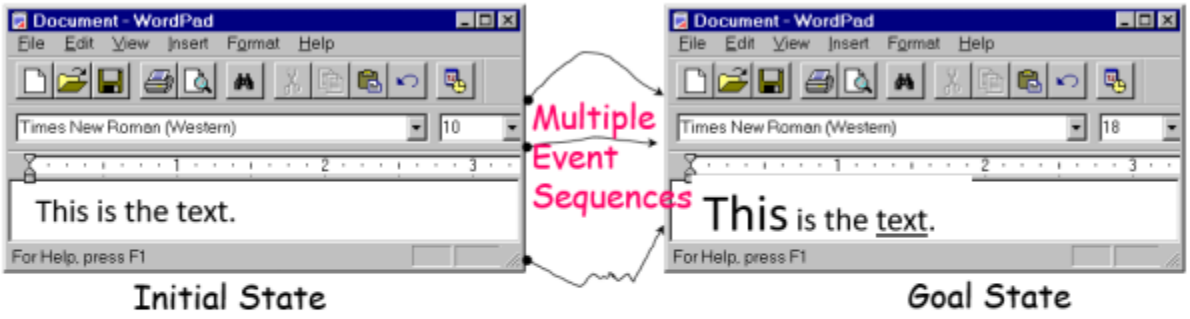
Graphical User Interface (GUI) Testing Challenges

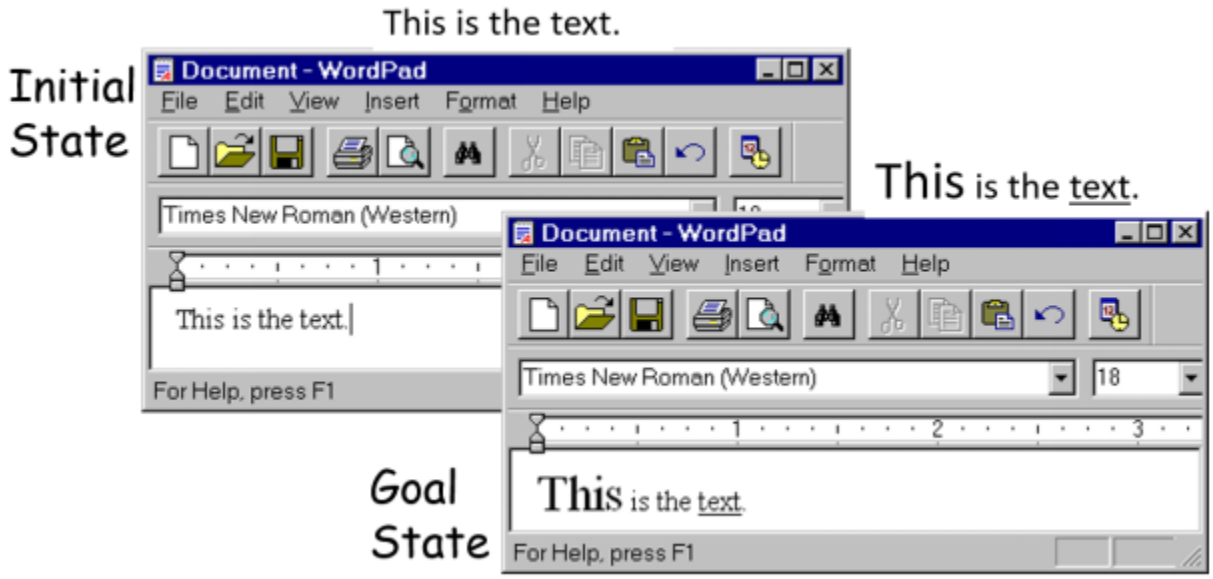
Testing the correctness of a GUI is difficult for several reasons. First of all, the space of possible interactions with a GUI is enormous. A large number of possible states results in a large number of input permutations requiring extensive testing.

Sequences can be infinitely many. Options could be:

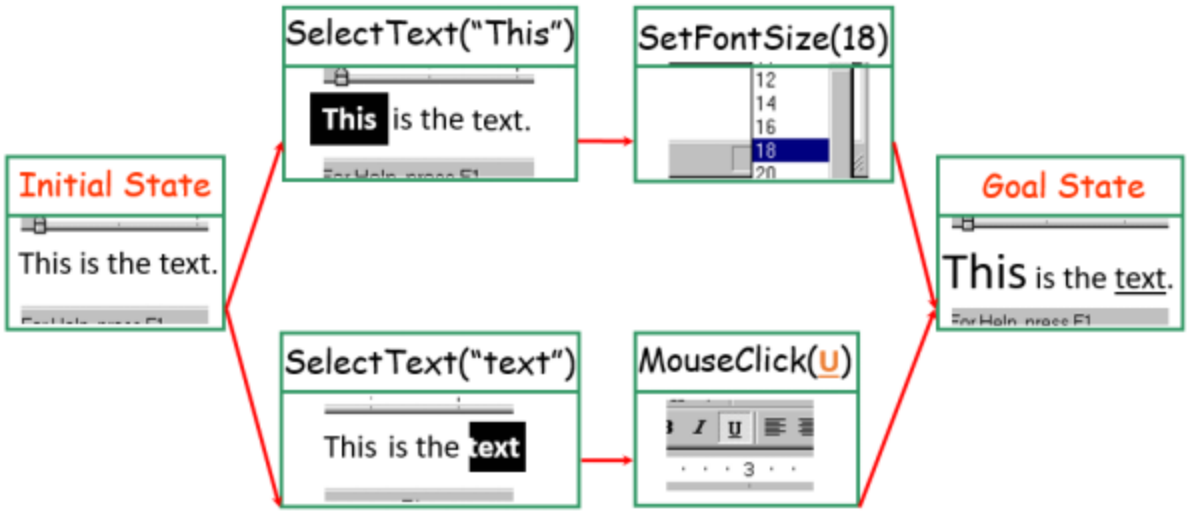
-
- Randomly choose sequences
- Expert chooses sequences
- Automatically generate events for commonly used tasks

Consider the following example having multiple event sequences.





Further elaboration of this example is as follows:



The problem of coverage is associated with it. For conventional software, coverage is measured using the amount and type of underlying code exercised. These measures do not hold good for GUI-based testing.

Another issue that we need to tackle is related to which level we are conducting the testing?

-
  -
    -
    - Unit level
    - Integration level
    - System level
    - Regression testing

- How to choose inputs?
  -
    -
    - using specifications or use cases etc
    - using code
    - only considering GUI (functionality, structure)

- How to identify the expected output?
  -
    -
    - test oracles

- How good test cases are?
  -

-
- when we can stop the testing activity

GUIs are event-driven, which has a large space of possibilities. The user may click on any pixel on the screen. Even the simplest components have a large number of attributes and methods. For example, JButton has more than 50 attributes and 200 methods. The state of the GUI is a combination of the states of all of its components.

Now we are summing up the challenges related to coverage. We cannot use code-based coverage due to the following reasons:

-
  -
    - Source code is not always available
    - Event-based input
    - Different levels of abstraction

We define an event-based graph for coverage known as an Event-flow graph.

Graphical User Interface (GUI) Testing Methods

Various methods are used for GUI testing. A few of them are discussed in this lecture.

1. Manual GUI testing:

In manual GUI testing, GUI components are identified and are manually tested by the tester and compared with the prototype screens or the test cases as prepared against the business requirement documents.

2. Record and replay (Test automation):

We can automate the GUI testing with the help of tools such as QTP, Selenium, Sikuli, etc.

3. Model-based GUI testing:

A graphical description of the behaviour of the system is known as a Model. A model helps us to determine the behaviour of the system under test. We use system requirements to generate efficient test cases.

The following needs to be considered for this model-based testing:

- Develop the model
- Determine various inputs for the model
- Determine the expected output for the model
- Execute the tests
- Compare the actual result against the expected output

- Make a decision on further action on the model

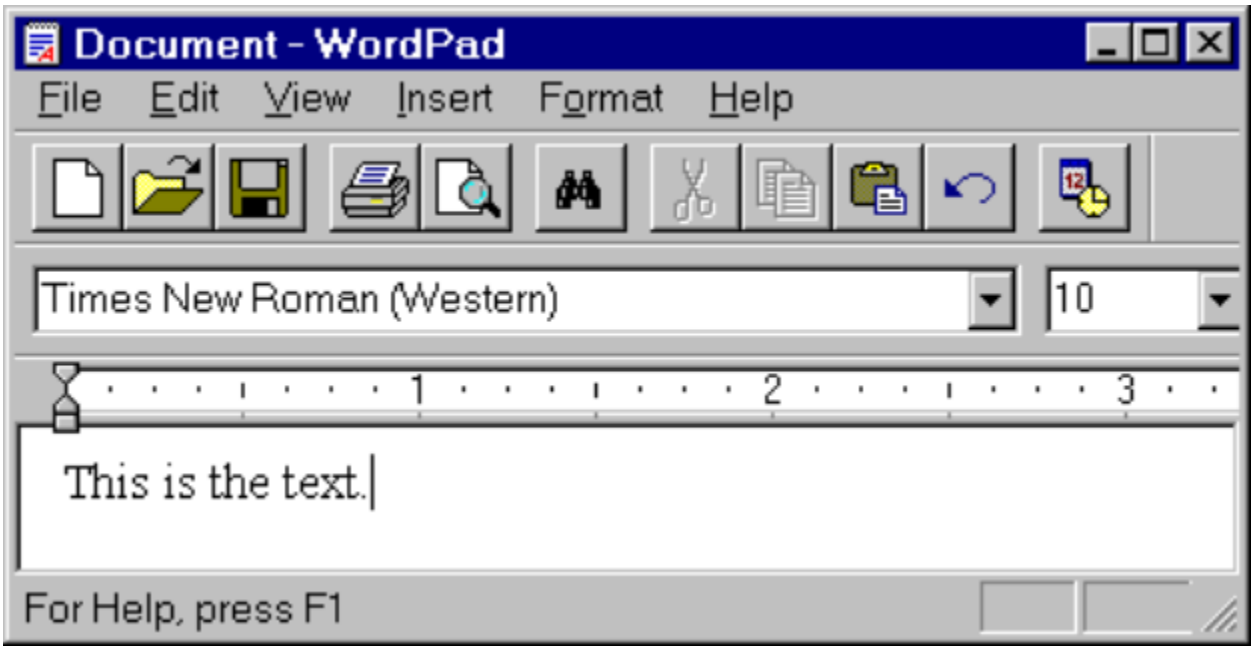
Since GUI is event-based, we develop the model in terms of an event-flow graph.

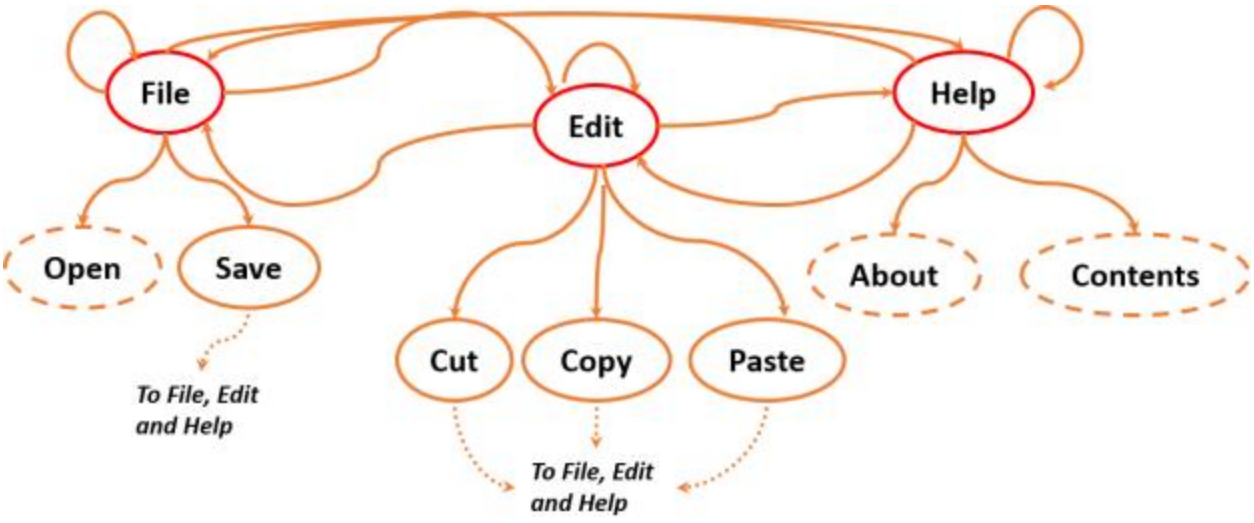
Definition: Event-flow graph is a 4-tuple

- <V, E, B, I>
- V is the set of vertices, representing events
- E is the set of directed edges, showing the follows relationship
- B is the set of events first available (shown in red)
- I is the set of events that invoke other components (dotted lines)

Considering our example of word processing application in the previous topic, we can find out:

-
- Set of vertices
- Set of edges
- Initial events
- Terminal events





Approaches to GUI testing:

Both black box and glass box (white box) approaches can be used.

- **Black box**
  - Launch the application
  - Simulate mouse and keyboard events
  - Compare the final look to an existing screen dump

- **Glass box**
  - Launch application in the testing code
  - Obtain references to the various components and send events to them
  - Assert the state of components directly

GUI Testing Coverage

Testing coverage criteria:

Each component is a unit of testing.

1. Test events within each component

   • Intra-component coverage criteria

2. Test events across components

• Inter-component coverage criteria

1. Intra-component coverage

-
- Event coverage
  - Individual events
  - Each node in the event-flow graph

-
- Event-interaction coverage
  - Each pair of events
  - Each edge in the event-flow graph

-
- Length-n event sequence coverage
  - Sequences of events
  - Bounded by length
    - Length-1, Length-2, length-6 event sequences
  - Paths in the event-flow graph

2. Inter-component Coverage

- Invocation coverage
  -
  - Invoke each component
  - Each restricted-focus event

- Invocation-termination coverage
  -
  - Invoke each component and terminate it
  - Restricted-focus event followed by a termination event

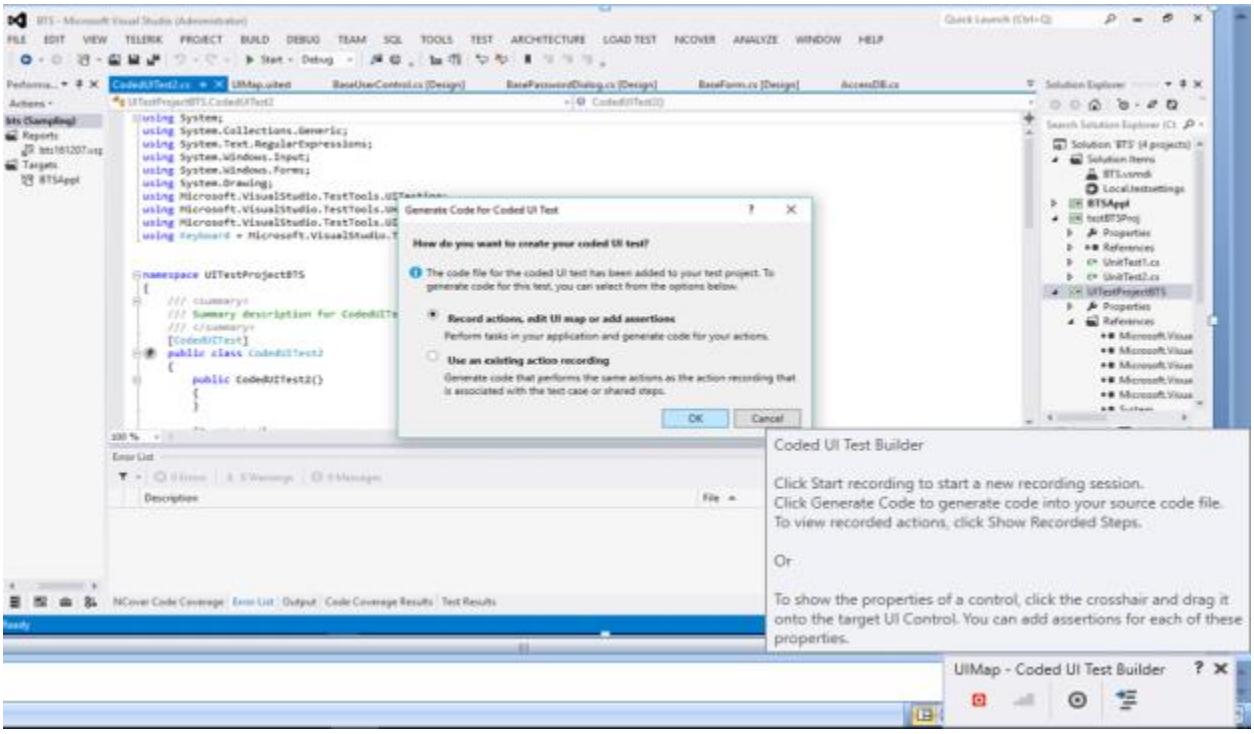
- Inter-component length-n coverage
  -
  - Longer sequences from one component to another
  - Bounded by length

GUI Testing Worked Example

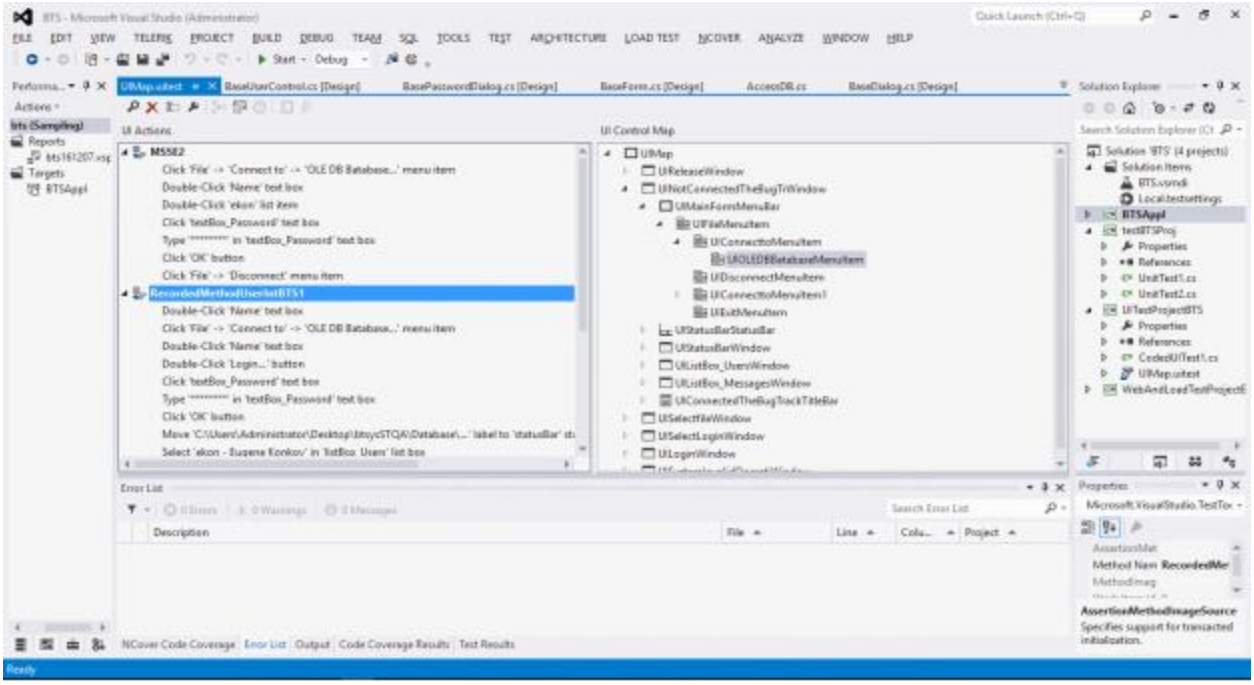
We are now writing GUI testing-based test case for a client-server/desktop application. We have already identified that we can write GUI based test cases using a variety of applications such as:

-
- Selenium
- Sikuli
- UFT
- Silk Test
- TestComplete

In this example, we use Microsoft Visual Studio to test a desktop application. The first step is to add a new “Coded UI Test” inside the “Coded UI Test Project”. Once it is created, visual studio will open the “Coded UI Test Builder” window having multiple editing options as shown in the figure below:



A test case can be recorded from the builder options. Microsoft visual studio will record tests in form of performed steps as shown in the figure below:



Once a test case is recorded, run the test case. It will show execution results as well as display all recorded steps in the same sequence in which they were performed.

Acceptance Testing

Acceptance testing is testing with respect to user needs, requirements, and business processes. It is conducted to determine whether (or not) a system satisfies the acceptance criteria and to enable the user, customers or any other authorized entity to determine whether or not to accept the system. Acceptance testing is a formal testing conducted to determine whether a system satisfies its acceptance criteria or not.

A few types of acceptance testing are as follows:

- User Acceptance Testing (UAT): It is conducted by the customer to ensure that the system satisfies the contractual acceptance criteria before being signed-off as meeting user needs.
- Business Acceptance Testing (BAT): It is undertaken within the development organization of the supplier to ensure that the system will eventually pass the user acceptance testing. It is also known as operational acceptance testing.
- Contract and regulation acceptance testing
- Alpha / beta (field) testing

Objectives of Acceptance Testing:

The main objectives of acceptance testing are as follows:

I. It confirms that the system meets the agreed upon criteria.
II. It identifies and resolves discrepancies (if there are any).
III. It determines the readiness of the system for cut-over to live operations.

The acceptance criteria is defined on the basis of the following aspects:

-
  - Functional Correctness and Completeness
  - Backup and Recovery
  - Usability
  - Performance
  - Start-up Time
  - Stress / Load / Volume
  - Reliability and Availability
  - Maintainability and Serviceability
  - Robustness
  - Timeliness, Compliance
  - Documentation
  - etc.

The acceptance criteria discussed are too many and very general. The customer needs to select a subset of the quality attributes. The quality attributes are prioritized according to the specific situation.

Involvement of in-house QA team:

The in-house QA team may be involved in the acceptance testing depending upon the scenario.

i. No involvement (UAT is performed by the client or their representative only)
ii. Assist in this testing (QA team and client both perform UAT)
iii. Perform UAT and present results(QA team perform UAT itself)

Acceptance Testing Phases

The acceptance test cases phases are:

Phase 1: Execute basic test cases

- If the test results are satisfactory then the second phase, in which the complex test cases are executed, is taken up.

  Phase 2: Complex scenarios or test cases

- A subset of the system-level test cases are executed to confirm test results independently.

Acceptance test execution activities are as follows:

-
  - First of all, train the customer (representative) on usage of the system.
  - Development team and customer representative team work on a pre-defined exit criteria and execute test cases.
  - Issues are discussed and resolved.

Some pre-requisites for acceptance testing are:

-
  - Business requirements must be available.
  - Application code should be fully developed.
  - Unit testing, integration testing and system testing should be completed.
  - No showstoppers, high, medium defects in system integration test phase.
  - Only cosmetic errors are acceptable before user acceptance testing (UAT).

-
  - Regression testing should be completed with no major defects.
  - All the reported defects should be fixed and tested before UAT.
  - Traceability matrix for all testing should be completed.
  - The UAT environment should be ready.
  - Communication from the System testing team that the system is ready for UAT.

Acceptance Testing Exit Criteria

There are a number of exit criteria for acceptance testing. A few of them are:

-
  - Traceability method
  - Formal method
  - Prototyping method
  - Etc.

Traceability Matrix:

- It works by creating a cross reference between requirements and acceptance test cases.
- Fabricating relationships between test cases and requirements.

- A straightforward way to match the test cases to the requirements:

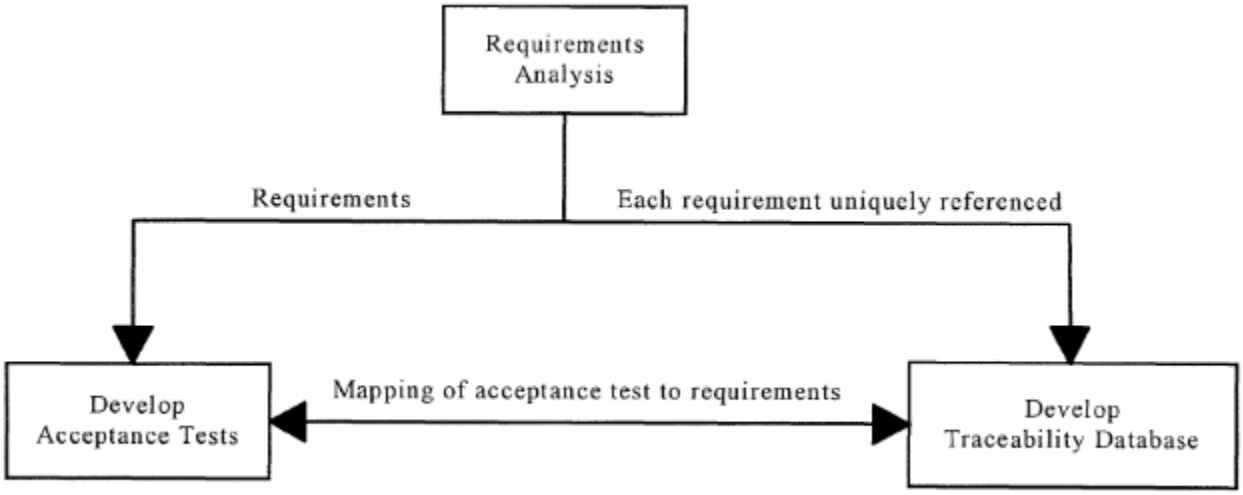
    - It allows testers to easily check the requirements not yet tested.
    - It shows the customer which requirement is being exercised by which test case(s).

- However, each cross reference is created by the tester subjectively.
- Requirements in SRS are tagged:

    - We have identifiers against each requirement, use-case, description, user-story, sprint, etc.
    - This is already done if we have requirement IDs in our test case specification.

- A database is used to automate traceability tasks.

    - A matrix or checklist ensures there is a test covering for each requirement.
    - Keep track of when tests are completed/failed.

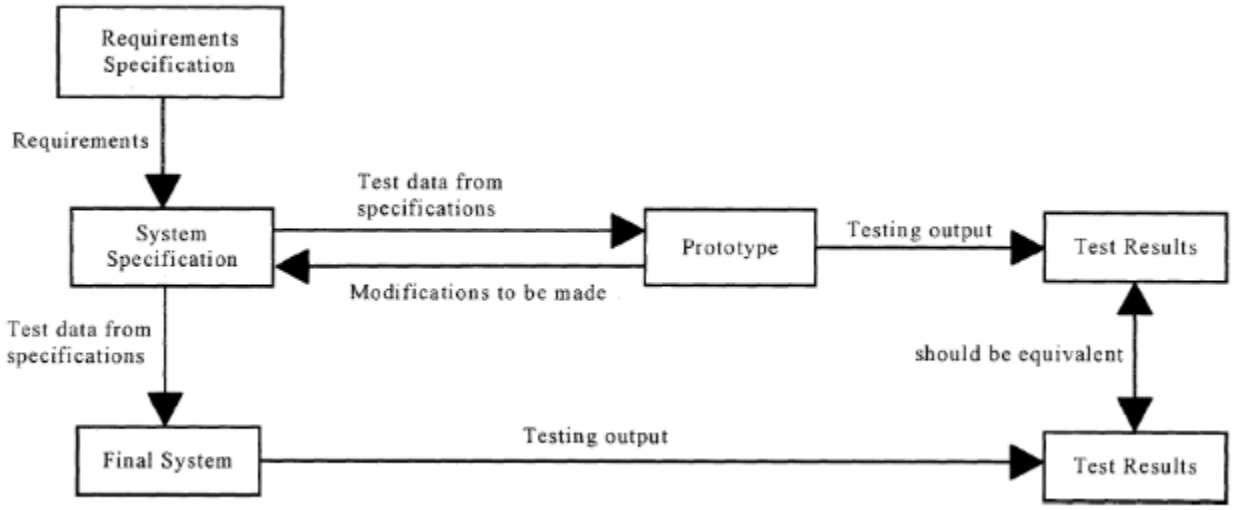


- Apply mathematical modeling to test case generation.
- They include:

    - application of formal languages

- finite state machines
- various graphing methods
- and others to facilitate the generation of test cases for acceptance testing

- Prototyping methods begin by developing a prototype of the target system.
- This prototype is then evaluated to see if it satisfies the requirements.
- Once the prototype does satisfy the requirements, the outputs from the prototype become the expected outputs from the final system.
- Therefore, prototypes are used as test oracles for acceptance testing.



- Once, we have some mechanism to arrive at a conclusion such as:

    - Accept the system as delivered.
    - Accept the system after modifications are made.
    - Do not accept the system.
    - If the test results are unsatisfactory, changes be made to the system before acceptance testing can proceed to the next phase.

- Finally, the process of acceptance testing is completed on the basis of the following exit criteria:

    - No critical defects remain open.
    - Business process works satisfactorily.
    - UAT Sign off meeting with all stakeholders.

Acceptance Testing Activities

User acceptance testing (UAT) involves the following activities:

- First of all, an effective UAT strategy is created.
- An effective collaboration between IT team and business analyst (BA) is ensured.
- Participation in requirements walk-through meetings with the client is ensured.
- Effort estimation, and test plan are reviewed.
- Requirements traceability is ensured.
- It is required to drive metrics collection to quantify the benefits derived out of the updated testing methodology, tools and environment usage.

UAT Team activities include the following:

- Verify & validate business requirements against business process
- Estimation for UAT
- Create & execute UAT test plan
- Prepare test scenarios, test cases and test data based on business process
- Maintain traceability
- Execute test cases and prepare test logs
- Report defects in test management tool and manage them throughout their life-cycle
- Produce UAT end-of-test report
- Provide business readiness support and live proving.

Reliability Testing

Reliability testing is "*the probability of **failure-free** operation of a computer program for a **specific time** in a specific environment*".

Example: Imagine, we are testing the software application of a very famous transport company Uber, and the probability that the system under test (SUT) is up and running for one year without crashing is 99%; this is referred to as reliability.

Whenever we say that a certain software application is not reliable, it means the system's output is not consistent. i.e. several runs of a system are not producing the same output. Inconsistency of a software depends upon a number of factors.

Again considering our previous example, we book a ride from Uber, however sometimes booking is confirmed and sometimes it is not. So it will be considered as an inconsistent system. Another example is if we book a seat from Lahore to Islamabad using Daewoo bus service, and sometimes we get seat confirmation SMS and sometimes not, then it will also be considered as an inconsistent system.

Reliability is a broad concept. It is applied whenever we expect something to behave in a certain way. Faults result in failure. Therefore, fault removal improves reliability. Failure is a dynamic system characteristic, since we come across faults only when executing them.

We have discussed two examples, and we have two different reliabilities. One for Uber (which is higher) and the one for Daewoo (which is lower). Reliability is not of a test (or test case) but is of a test score. It is a psychometric property (psychometric properties are characteristics of tests and other measures of human characteristics that identify and describe attributes of an instrument, such as its reliability or appropriateness for use in a particular circumstance).

It is important to note that we use the same settings and repeat a test, and we measure consistency, repeatability or stability of results. It is a correlation coefficient used as an index of reliability. Where correlation coefficient is used, in statistics, to measure how strong a relationship is between two variables.

Reliability depends upon:

-
  - Faults in a system
  - Operational profile of a system

Reliability is a user-oriented quality factor relating to system operation. If users rarely experience failure, the system is more reliable than one that fails more often. However, developing a 100% correct system is difficult (but not impossible). Even an incorrect system may be considered to be reliable if frequency of failure is acceptable.

Reliability Testing Characteristics

We have already defined reliability testing as"*the probability of **failure-free** operation of a computer program for a **specific time** in a specific environment*".

Reliability testing is dependent upon:

- Fault density: number of faults presents in the software
- Time:
  - Execution time: CPU time taken by a software application
  - Calendar time: normal view of time
  - Clock time: execution time + idle time

- Operational profile: the way users operate the system

We need to revisit definitions of error, fault and failure.

Error is the difference between actual observed output and the correct expected output . It refers to human action resulting in faults or defects.

Fault is the static condition that causes a system to fail in performing its required functionality.

Failure is the inability of a system to perform its required function according to its specification. Failure is a characteristic of program's behaviour and it is a dynamic attribute, whereas fault is a static attribute. Failure occur only when a fault is executed.

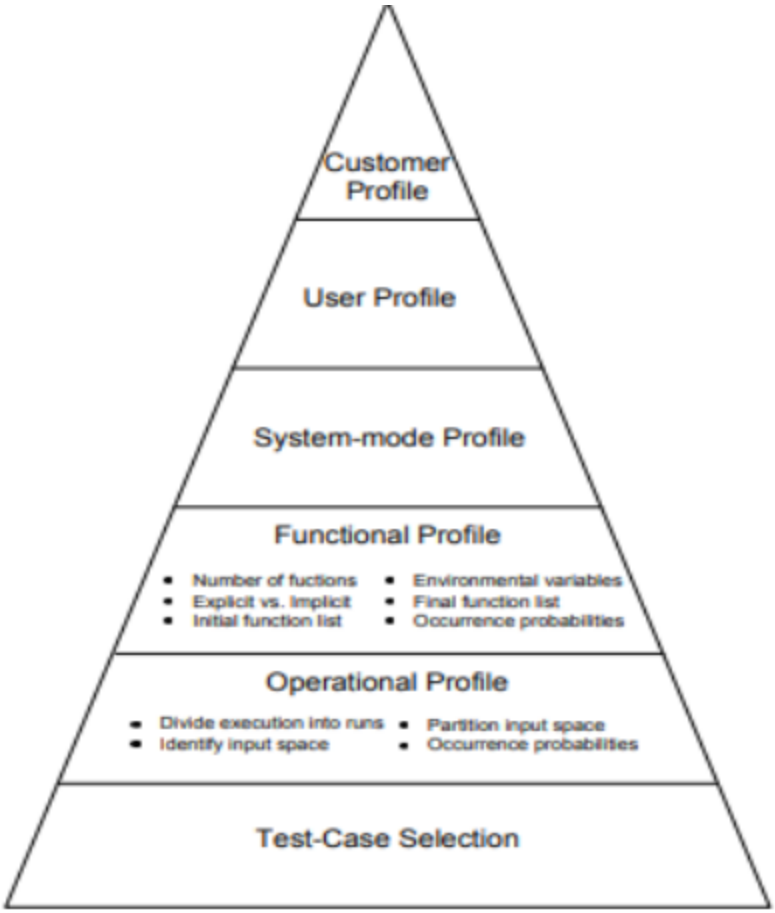
Main purpose of reliability testing is to check whether the system under test (SUT) meets requirement of customer's reliability. Reliability testing will be performed at several levels e.g., at unit, assembly, (sub-) system levels.

The objective behind performing reliability testing are to find out:

- structure of repeating failures.
- number of failures occurring is the specified amount of time
- main causes of failure
- performance testing of various modules of software application after fixing defect

Operational profile of a system

An operational profile describes how actual users operate a system. There are many ways to represent operational profiles e.g., Tabular , Graphical etc. This concept is developed by John Musa at AT&T Bell Labs. His company develops operational profiles to guide the testing of systems. With an operational profile, a system can be tested more efficiently because testing can focus on the operations most used in the field. It is a practical approach to ensure that a system is delivered with a maximized reliability, because the operations most used also have been tested the most.



Reference:

Operation profiles example from "Musa J.D., Operational profiles in software reliability engineering, In IEEE Software 10 (1993), No. 2, pp. 14-32"

Reliability Testing Types

Reliability testing types include:

- Feature testing
- Load testing
- Regression testing

1. Feature Testing:

It checks feature provided by software and is conducted such that:

- Each operation in SUT is executed at least once

- Interaction between two operations is reduced
- Each operation have to be checked for proper execution

2. Load testing:

- It checks how well the system under test (SUT) performs when compared to other systems. It is also based on the number of concurrent users who are using the SUT and behaviour of the SUT to users.
- The SUT must respond to user commands with better response time (than the system in competition) and meet user expectations.

3. Regression testing:

- It checks if the SUT is performing well and no bugs have been introduced as a result of the addition of new functionality in the software.
- It is also done when a bug has been fixed and the tester needs to test it again.

Fundamental reliability testing types include:

- Test-retest reliability
- Parallel or alternate forms reliability
- Inter-rater reliability, etc.

1. Test-re-test Reliability: A reliable test must show similar results over time. Consider the Daewoo bus service example, we do booking at 9:30 am and testing the same functionality at 1:00 pm again, and we compare both results, and if we are getting a high correlation in results, we find out that test is 'Reliable'.

2. Parallel or Alternate Forms Reliability: Different versions of the same test must produce similar behaviour. Consider the Daewoo bus service example again. We do booking at 9:30 am and then we test the same functionality at 1:00 pm again. We compare both results and if we are getting a high correlation in results, we conclude that the system is reliable.

3. Inter-rater Reliability: Rating by multiple judges should be similar and high. Inter-rater reliability is also known as inter-observer or inter-coder reliability. It is a special type of reliability which consists of multiple judges. It deals with the consistency of the rating put forward by different observers.

Models of Software Reliability

There are many types of software reliability models which are broadly categorized into two categories:

- Prediction models
- Estimation models

1. Prediction Modeling:

In predictive testing, we predict result with the historical data, statistics, machine learning data etc. We construct plots such as scatter plot and draw an extrapolate line to the existing historical data and we can predict the upcoming data.This type of modeling is performed before the development or testing stage itself.

2. Estimation Modeling:

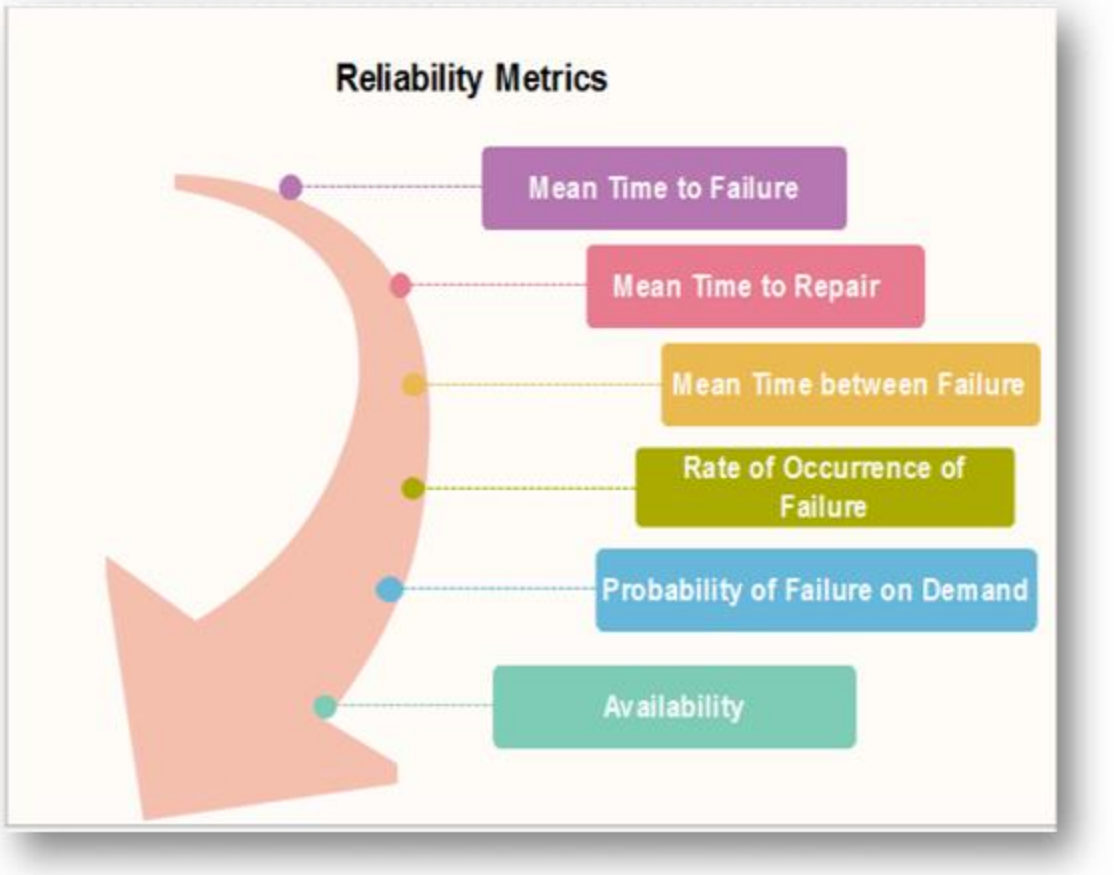
In addition to using historical data, we use current data in estimation testing.We predict reliability of a product in the present time or future time. This type of testing is performed during last stages of software development.

Key assumptions while developing these models are:

- Faults in the program are independent.
- Execution time between failures is large with respect to instruction execution time.
- Potential test space covers its use space.
- The set of inputs per test run is randomly chosen.
- The fault causing a failure is immediately fixed or else its re-occurrence is not counted again.

Software Reliability Metrics

Reliability metrics are used to quantitatively express the reliability of the software product. The option of which metric is to be used depends on the type of system to which it applies and the requirements of the application domain. Some reliability metrics which can be used to quantify the reliability of the software product are as follows:



1. Probability of failure:

   It is the likelihood that the system under test (SUT) would behave unexpectedly.

2. Rate of occurrence of failure:

   It is the frequency of occurrence of unexpected behaviour of the SUT.

3. Mean time to failure (MTTF):

   It is a measure of the time interval between observed failures. It is useful to find how long a system remains stable.

4. Availability:

   It is a measure of likelihood that the system would be available for use over a given period of time.

The mathematical formula of availability is:

Availability = [MTTF /(MTTF+MTTR)]*100

Where,

MTTF: Mean Time To Failure

MTTR: Mean Time To Repair

MTBF: Mean Time Between Failures

= MTTF + MTTR

Other important factors in the context of software reliability metrics are:

1. Intrinsic Quality:

It is measured in terms of number of "bugs" in the software. How long a system can run before encountering serious faults, e.g., crash.

It is measured in terms of:

- Fault density: it measure faults relative to software size
- Mean time to failure (MTTF): it measure time between failures

2. Customer Satisfaction:

It is measured in terms of customer survey data and usability metric. For measuring intrinsic quality, we calculate fault removal effectiveness in the following steps:

- Faults are injected during different phases
- Faults at development cycle m = faults escaped in (n – 1) + faults injected in (n) – faults removed in (n)
- For measuring reliability, we need to calculate fault removal ability
- When used for early phases, it is call early fault removal
- When used for a particular phase, it is called phase effectiveness
- Higher is the metric, better is the development process

Thus, fault removal effectiveness is calculated as:

= (fault removal in a development phase X 100) / fault latent in the product

Where

Fault latent (step i)=faults escaped (step i-1)+defects injected (step i)

Fault arrival pattern co-relates with faults rate in the production environment. Higher faults during testing show that software quality is low. Higher faults indicate the development process and developers' ability. It shows the fault density (i.e., faults per KLOC). Function point (FP) is also an indicator of produced software's quality.

Cleanroom Software Engineering

The name "cleanroom" is derived from the process used to fabricate semi-conductors. This philosophy focuses on defect avoidance rather than defect removal. It combines formal methods and software quality methods.

Cleanroom software engineering is a concept of shifting practice (from – to):

-
  - From individual efforts to peer reviewed
  - From sequential development to incremental
  - From informal coverage to statistical measures
  - From unknown reliability to measured reliability
  - From informal design to a formal design

Cleanroom software engineering involves integrated use of software engineering modeling, program verification, and statistical software quality assurance. It verifies design specification using mathematically-based proof of correctness. It relies on statistical use of testing to uncover high impact errors. It generally follows an incremental development process.

Benefits of cleanroom software engineering include:

-
  -
    - Failure reduction
    - Less development time
    - Better product life

The primary goal of cleanroom software engineering is to avoid dependence on defect-removal processes, by writing code increments right the first time and verifying their correctness before testing.

Thus, in cleanroom software engineering, our focus shifts from defect detection / correction to defect prevention, and zero-defect software is our goal.

Cleanroom Software Engineering Principles

The basic principles of cleanroom software engineering are as follows:

We have small team with independent specification, development, and certification sub-teams.

We do incremental development under statistical quality control. We assess performance during each increment using measures including:

-
    -
        - errors per KLOC(thousands of lines of code),
        - rate of growth in MTTF (mean time to failure),
        - or number of sequential error-free test cases
        - Etc.

Feedback used for process improvement and development plan is adjusted as needed.

The specification team develops and maintains the system specification. The development team develops and verifies software (software is not compiled or executes during verification). The certification team develops a set of statistical test to exercise software after development, and reliability growth models are used to assess reliability.

The software development is based on mathematical principles as under:

- The box principle is used for specification and design.
- Formal verification is used to confirm correctness of implementation of the specification.
- The program correctness is verified by team reviews using questionnaires.

Testing is based on statistical principles as under:

- Operational usage profiles are needed.
- Test cases are randomly generated from the usage model.
- Failure data is interpreted using statistical models.

Cleanroom Software Engineering Process

Cleanroom Software Engineering Process:

The cleanroom software engineering process works in an increment manner, and the following steps are involved:

- **Increment Planning:** The project plan is built around the incremental strategy.

- **Requirements Gathering:** Customer requirements are refined for each increment using the traditional requirements elicitation techniques.
- **Box-structure Specification:** After requirements gathering, a box structure specification is developed. Box structures isolate and separate the definition of behavior, data, and procedures at each level of refinement.
- **Formal Design:** Specifications (black-boxes) are iteratively refined to become architectural designs (state-boxes) and component-level designs (clear-boxes).
- **Correctness Verification:** At each level of refinement, correctness questions are asked and answered and followed by formal mathematical verification when required.
- **Code generation, Inspection, and Verification:** Box structures are translated into code; inspections are used to ensure conformance of code and boxes, as well as syntactic correctness of code; followed by correctness verification of the code.
- **Statistical Test Planning:** A suite of test cases is created to match the probability distribution of the projected product usage pattern.
- **Statistical Use Testing:** A statistical sample of all possible test cases is used rather than exhaustive testing.
- **Certification:** Once verification, inspection, and usage testing are completed and all defects are removed, the increment is certified as ready for integration.

Certification Steps:

The following steps are involved in the certification phase:

- Usage scenarios must be created.
- Usage profile is specified.
- Test cases are generated from the usage profile.
- Tests are executed and failure data are recorded and analyzed.
- Reliability is computed and recorded.

Cleanroom Software Engineering - Benefits

Benefits of cleanroom software engineering include the following:

- Our focus is on developing the right systems the first time.
- It requires customer involvement and feedback throughout the development process.
- It requires management control of resources and technical control of complexity.
- Product quality requires process measurement and control throughout the software development cycle.
- It supports concurrent engineering by scheduling parallel development and certification.
- It involves stepwise integration through testing cumulative increment (incremental delivery).
- It provides continuous quality feedback (of each increment) from statistical process control.
- It provides continuous customer feedback from actual use.

- It supports change management by systematic accommodation of change.
- It represents the first practical attempt to put software development under statistical quality control with a well-defined strategy for continuous process improvement.

Cleanroom Software Engineering - Box Principles

We need to understand box structure associated with cleanroom software engineering. We need to understand this to get to testing focus.

Box are essentially a design concept:

- The development team carries out a design and correctness verification cycle for each increment.
- The certification team works in parallel, using the usage specification to generate test cases that reflect expected use of the accumulating increments.

Box structures are used to systematically move from an abstract specification to a detailed design providing implementation detail.

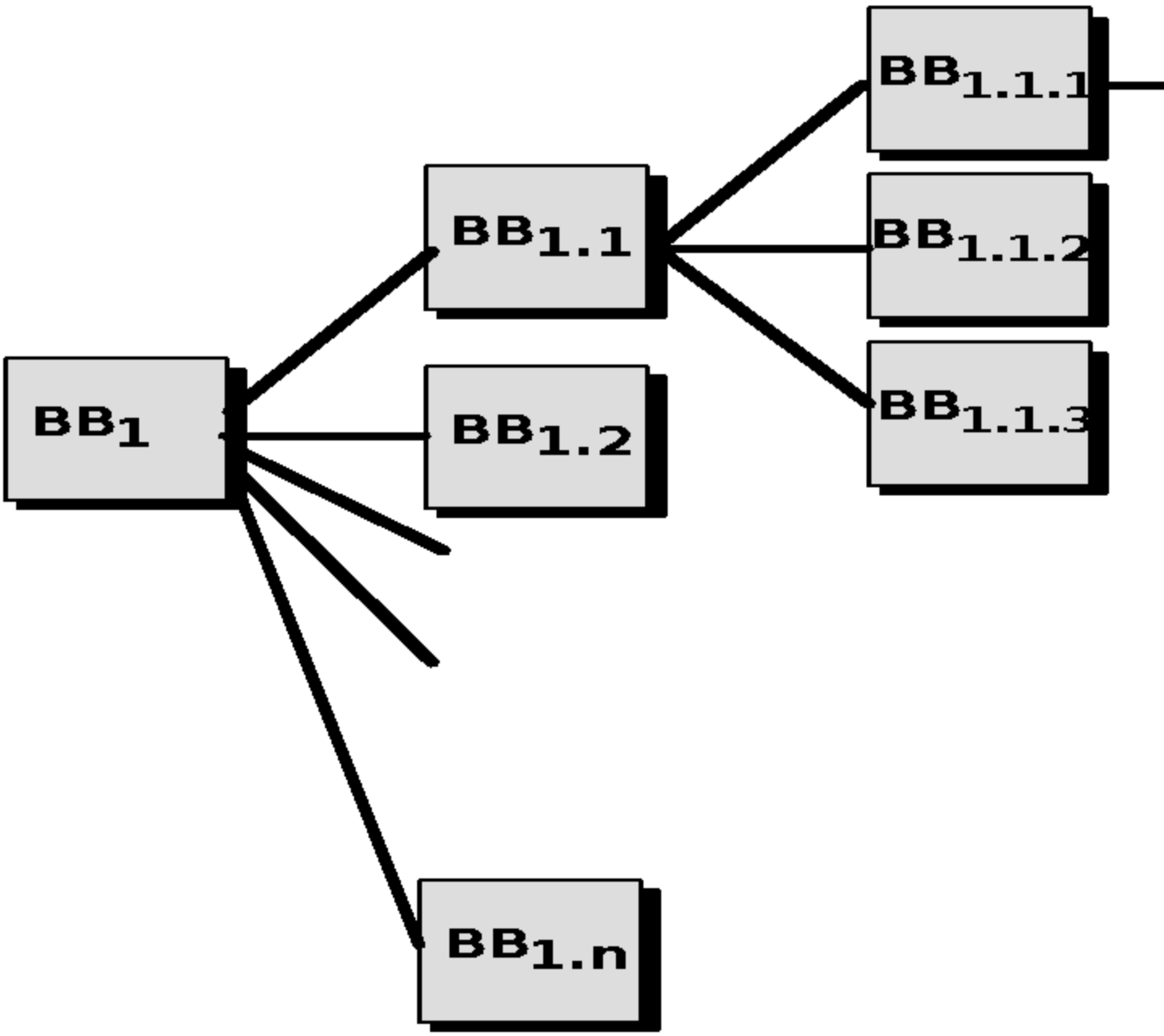
Box structures model system components as abstractions in three detailed forms:

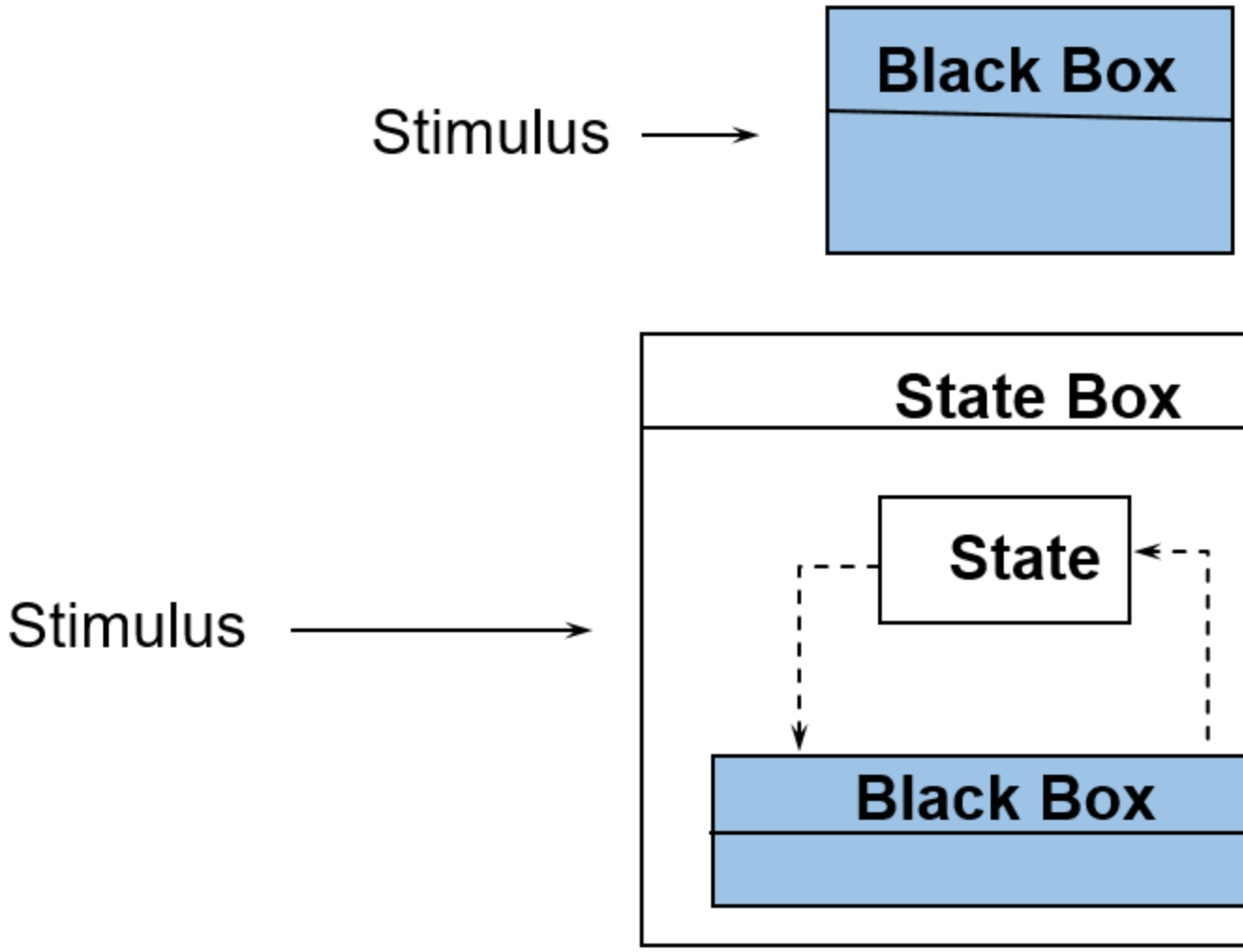
1. Black Box
    - It gives an external view of the component.
    - It provides description of functional requirements without details on the internal structure and operations.
    - It describes the user-visible system inputs and responses.

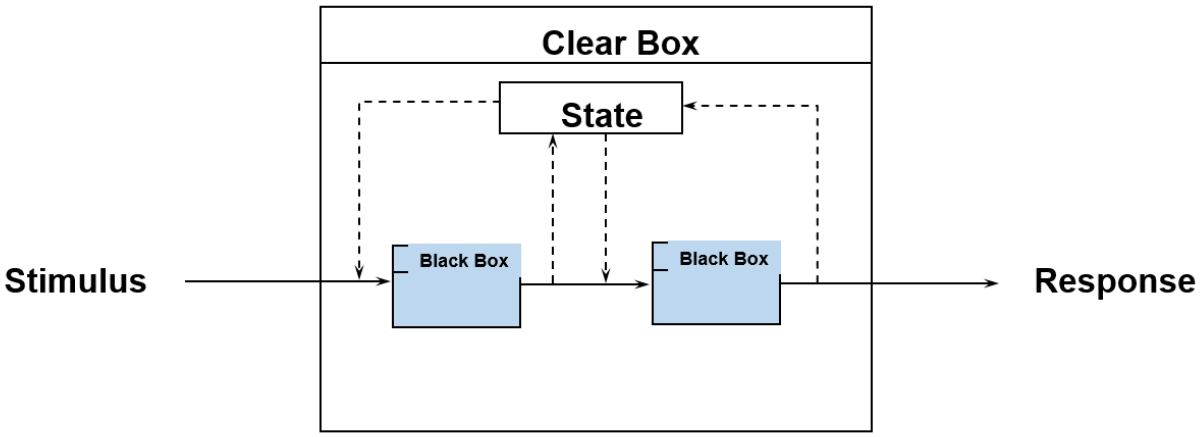
2. State Box
    - It gives an intermediate view of the component.
    - It decomposes the black box into an internal state representation and an internal black box.

3. Clear Box
    - It gives a detailed view of the component.
    - It replaces the internal black box with detailed design using structured programing constructs.

System development is a process of stepwise box decomposition.


BB1
BB1.1
BB1.2
BB1.n
BB1.1.1
BB1.1.2
BB1.1.3


Stimulus
Black Box
Stimulus
State Box
State
Black Box



Box Structure Principle:

- Referential Transparency
  - The behavior of a black box is the same regardless of where in the system it is referenced.
  - The implementation of a black box is independent of the implementation of other parts of the system.
  - Referencing a black box is equivalent to referencing its corresponding clear box representation throughout the system.
- Transaction Closure
  - It ensures that a sound and complete set of transactions is identified to achieve the required system behavior.
  - Black box level - system stimuli are necessary and sufficient to generate the required responses.
  - State box level - defined transactions must be necessary and sufficient for the acquisition and preservation of all state data
  - State data must be necessary and sufficient for the completion of all transactions
  - Clear box level - procedural design and the internal black boxes must include all transactions.
  - We can mark a start-state and an end-state.
  - This is helpful while writing test cases.
- State Migration
  - State data should be stored at as low a level as possible, but as high as necessary or expedient.
- Since we are using the concept of state and state transition, we can co-relate this directly with our previous knowledge of extracting test cases from systems represented as a state machine. Referring to our model-based testing lecture.

- Common Services
  - Reusable boxes
  - May be created or referenced from a library of reusable components.

- If from a library, the common service is a pre-certified component.
- Can reduce system size and complexity.

Cleanroom Software Engineering - Testing Focus

In this lecture, we will discuss the testing focus in cleanroom software engineering.

- Generation of test cases

- SUT is treated as a state-machine.
- Each test case begins in a start state and represents a random walk through the usage model ending at a designated end state.
- Our test sequences correspond to paths through the state-machine.
- Control of statistical testing
- A well-defined procedure is performed under specified conditions.
- Each performance is a trial and can be used as part of an empirical probability computation.
- Stopping criteria for testing
- When testing goals or quality standards are achieved.
- When the difference between the predicted usage chain and the actual testing chain becomes very small.

There are five major activities involved in a Cleanroom process.

- Specification
- Increment Planning
- Design and Verification
- Statistical Testing
- Certification

Two to three independent teams may exist and work concurrently.

- Development Team
- Testing (or Certification) Team
- Documentation Team

Two specifications are produced: functional and usage.

- Functional Specification
    - Defines the required external system behavior in all circumstances of use.
    - Forms the basis for incremental software development.
- Usage Specification
    - Defines usage scenarios considering:
        - *User* - person, hardware device or other software; subclasses may exist

- *Use* - a particular work session or transaction; bounded by specific start and end events
- *Environment* - platform, OS environment, system load, etc.

- Forms the basis of statistical testing and quality certification.

The procedural control structures of structured programming are single-entry, single-exit structures. Therefore, it produces no side-effects in control flow.

When it executes, a given control structure simply transforms data from an input state to an output state. This transformation is called as the structure's *program function*.

Example: For integer x >= 0, the program function of the iteration control structure below is, in English, "Set odd x to 1, even x to 0."

In designing clear box procedures, we define:

- an *intended function*,
- We then refine it into a control structure and
- new intended functions.

Intended functions are recorded in design and are attached to the corresponding control structure refinements. So, clear boxes are composed of a finite number of control structures, each of which can be checked for correctness against its intended function.

To verify the correctness of each control structure,

- We derive its program function (the function it actually computes) and
- Compare it to its intended function, as recorded in the design.

This corresponds to the test case structure we have already studied. A correctness theorem formally defines how to do this for each control structure in terms of language independent correctness conditions.

- During a team review, every correctness condition of every control structure is verified in turn.
- Each team member must agree that each condition is correct. Thus, an error is possible only if every team member incorrectly verifies a condition.
- If an informal approach cannot produce a unanimous decision, formal proof of correctness can be employed.
- This is more efficient and produces better code than unit testing.
- Testing software according to the way users intend to use it.

- The entire focus is on external system behavior, not the internals of the design or implementation.

- The certification team's goal is not to debug but to certify the software's quality. This requires deep knowledge of expected usage but no knowledge of design or implementation information.

- Three steps:
    - Specify usage probability distributions
    - Derive test cases that are randomly generated from usage probability distributions.
    - Execute test cases, assess results, and compute quality measures.
- Based on the data gathered during statistical testing, the software can be given certified reliability.
- Reliability is expressed as MTTF and is computed according to specific mathematical reliability models
    - Sampling model
    - Component model
    - Certification model

Software Testing Support for Java related IDE

We have already seen unit test development using Microsoft Visual Studio. We now have a look at how to use Eclipse for the same purpose. We intend to see how we can make use of Java related environments for the same purpose.

Unit test development in eclipse IDE for Java code:

In order to test java based application code in eclipse IDE, the first step is to create a new java project inside eclipse IDE. Once project is created, create a new package from project items option. Now add a new class inside package. Write the following code inside class and compile the code.

```
package src;

public class MyBSearch {

   public int search(int key, int[] elemArray) {

      int bottom = 0;  int top = elemArray.length - 1;

      int mid;   int index = -1;  boolean found = false;

      while (bottom <= top && found == false)  {

         mid = (top + bottom) / 2;

         if (elemArray[mid] == key) {

            index = mid;  found = true;     return index;       }
```

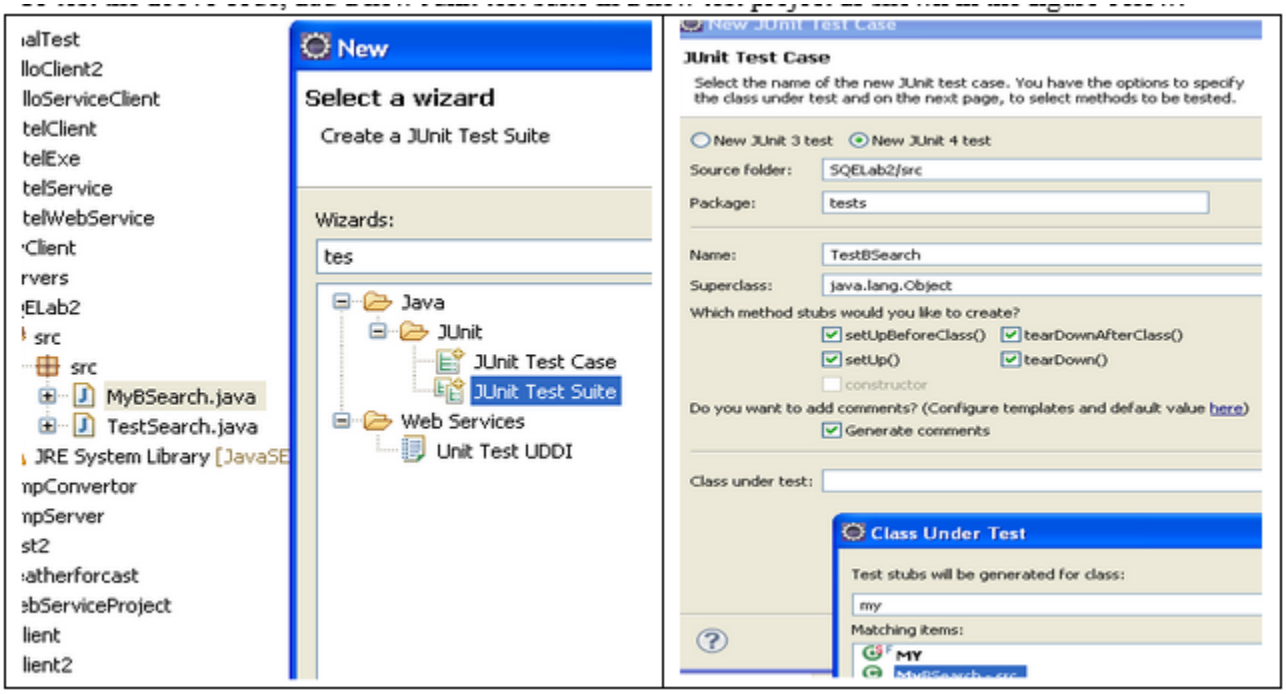
```
    else  {          if (elemArray[mid] < key)

                bottom = mid + 1;

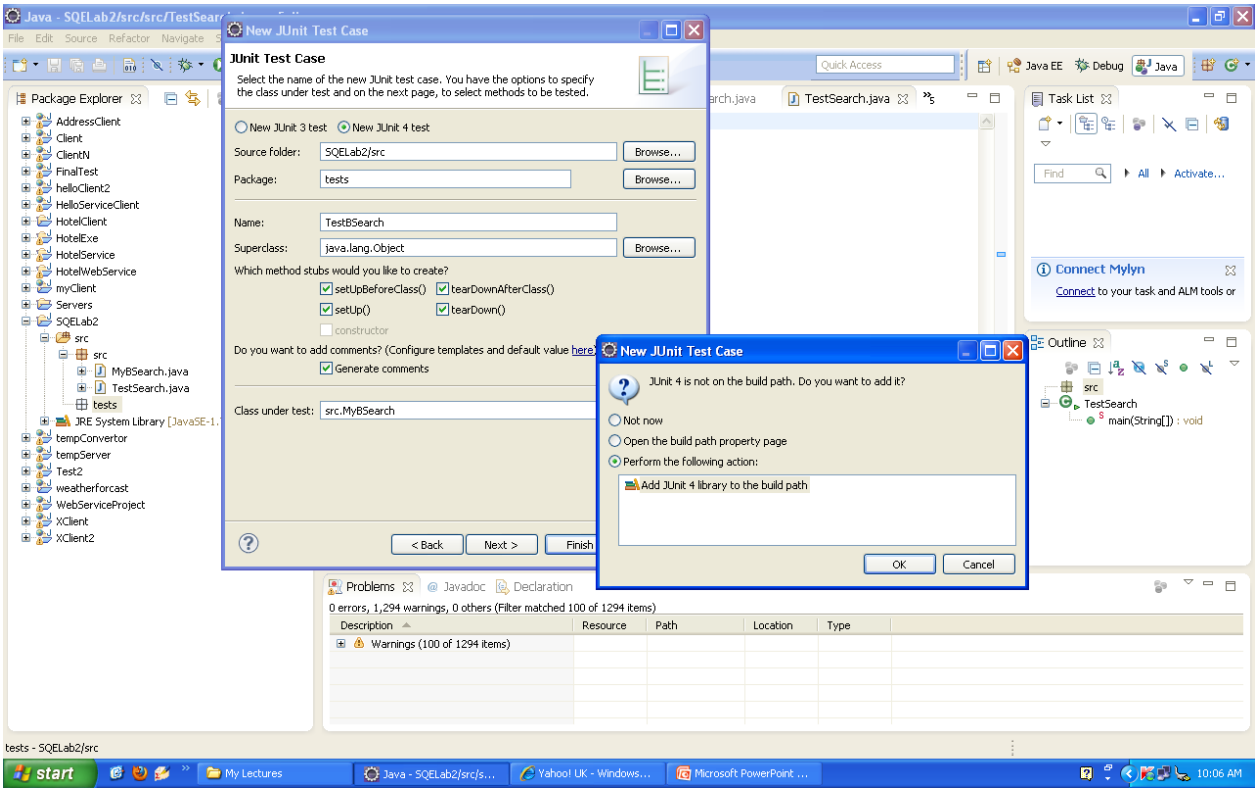
        else              top = mid - 1;

    } }      return index; }}
```

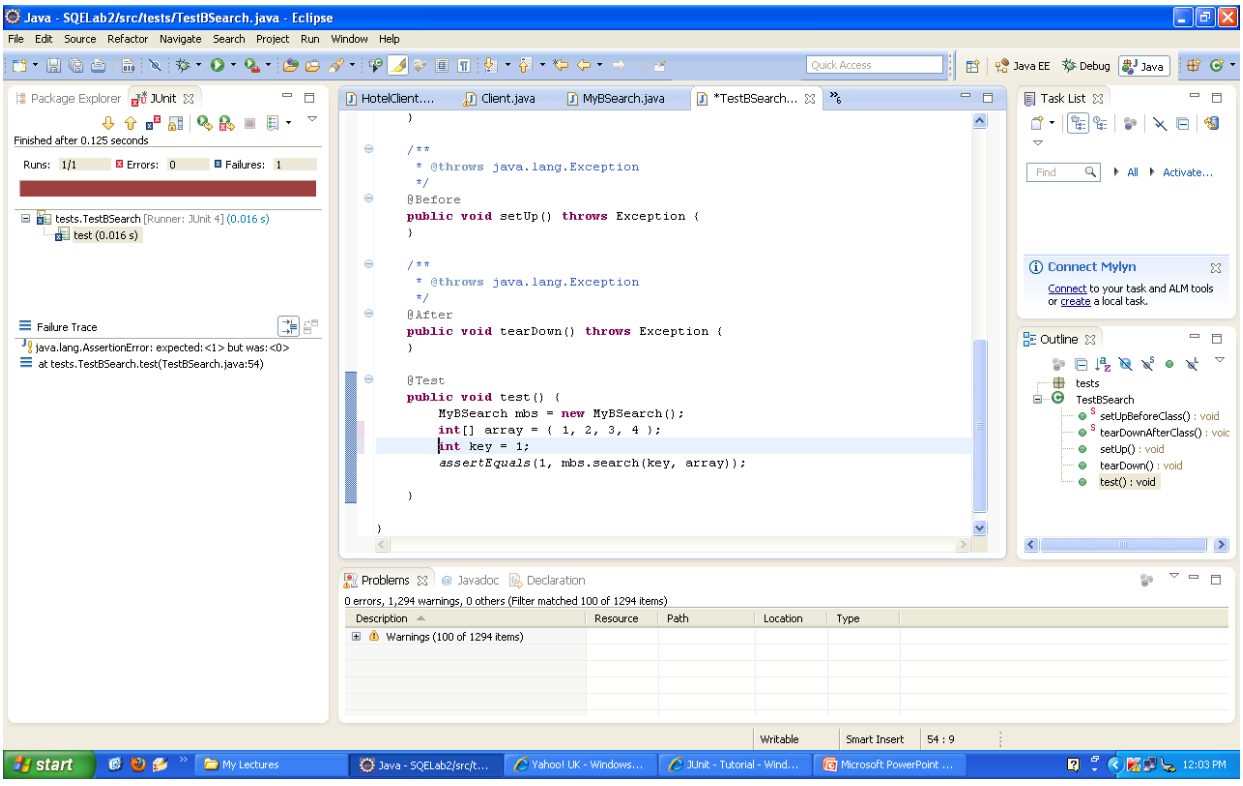
To test the above code, add a new Junit test suite in a new test project as shown in the figure below:



Now add a reference to the SUT project as shown in the figure below. It will provide built-in testing functions. Also, make sure that the code project and test project should be in two different folders so we can reuse the code project in other IDE as well.



The next step is to write a unit test code inside a test class for the given search function under test. The test code will be as follows

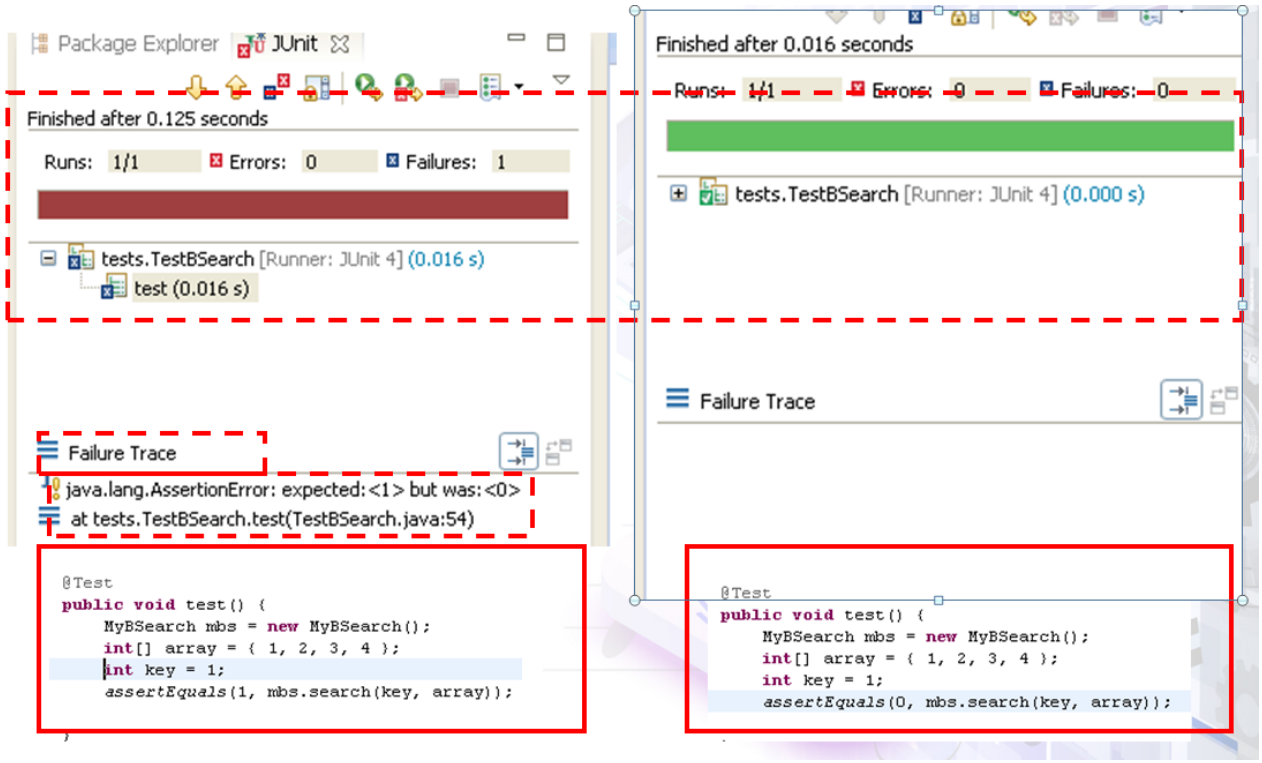


The search function search element from an array based on the key index given as input. This test code will test the search function by comparing the expected output with the actual output. An “assertEquals” function is used with two parameters described as

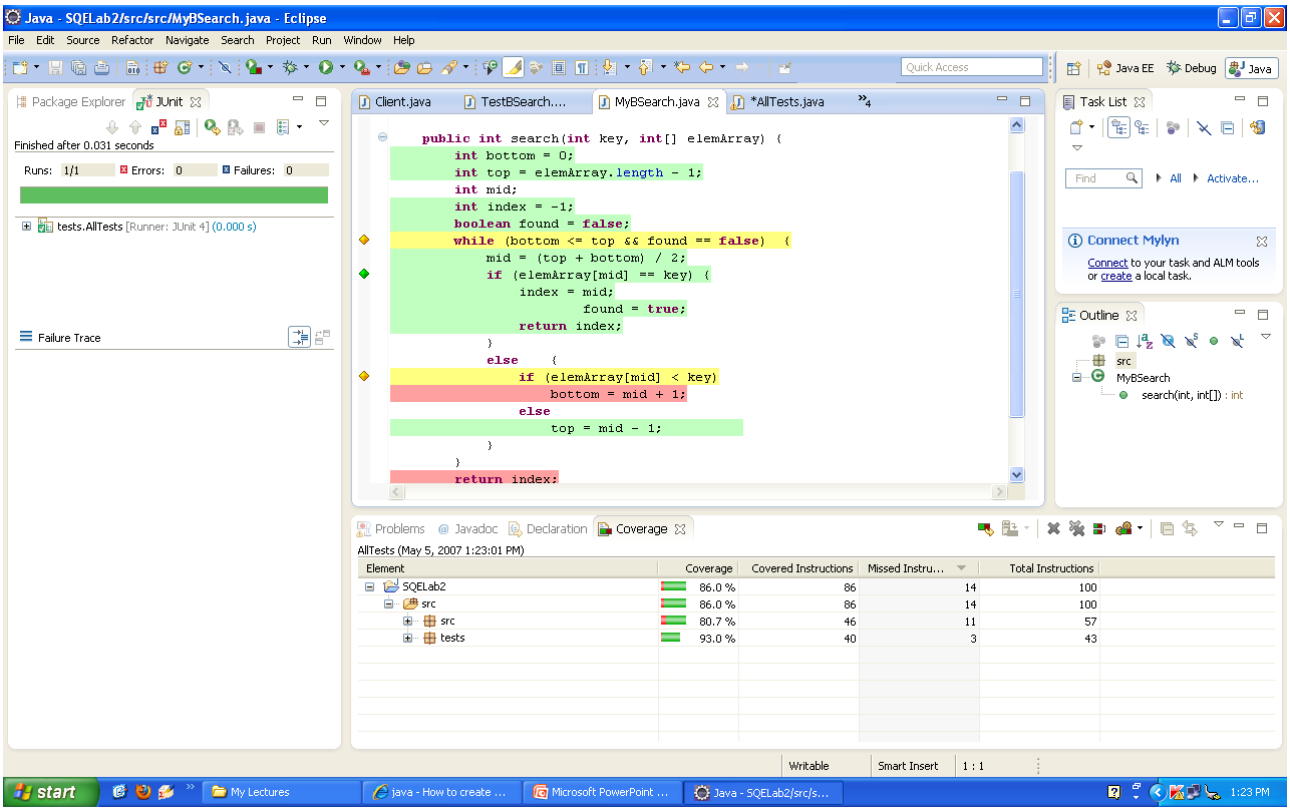
(1)-> Expected output

(mbs.search (key,array)) -> Actual output

For key=1, the actual output should be ‘0’. This test case will fail because the actual output does not match with the expected ‘1’. The test case will pass in case the expected output should be ‘0’. The results are shown in the figure below:



Similarly we can add multiple unit test codes to test different aspects of Java code under test. To get test coverage and code analysis we should add “Eclemma Plugin” inside eclipse. It will provide complete coverage details as well as analyze the code. The test coverage and code analysis results are highlighted in the figure below:


Java - SQELab2/src/src/MyBSearch.java - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer JUnit
Finished after 0.031 seconds
Runs: 1/1 Errors: 0 Failures: 0
tests.AllTests [Runner: JUnit 4] (0.000 s)
Failure Trace
Client.java TestBSearch.... MyBSearch.java *AllTests.java
public int search(int key, int[] elemArray) {
int bottom = 0;
int top = elemArray.length - 1;
int mid;
int index = -1;
boolean found = false;
while (bottom <= top && found == false) {
mid = (top + bottom) / 2;
if (elemArray[mid] == key) {
index = mid;
found = true;
return index;
}
else {
if (elemArray[mid] < key)
bottom = mid + 1;
else
top = mid - 1;
}
}
return index;
Task List
Connect Mylyn
Connect to your task and ALM tools or create a local task.
Outline
src
MyBSearch
search(int, int[]) : int
Problems Javadoc Declaration Coverage
AllTests (May 5, 2007 1:23:01 PM)
Element Coverage Covered Instructions Missed Instru... Total Instructions
SQELab2 86.0 % 86 14 100
src 86.0 % 86 14 100
src 80.7 % 46 11 57
tests 93.0 % 40 3 43
Writable Smart Insert 1 : 1
start
My Lectures
java - How to create ...
Microsoft PowerPoint ...
Java - SQELab2/src/s...
1:23 PM

THE END

State Machine for Testing

State-machine is one such example of system represented as a model. Finite State Machine is defined formally as a 5-tuple, (Q, Σ, T, q0, F) consisting of a finite set of states Q, a finite set of input symbols Σ, a transition function T: Q x Σ →Q, an initial state q0 ∈ Q, and final states F ⊆ Q .

FSM can be described as a state transition diagram. For example, figure 1 depicts state transition diagram where Q= {s0, s1} and Σ = {0, 1}. For complex problems, the difficulty in representing the system as FSM is how to deal with the state explosion problem. A system with n variables that can have Z values can have Zn possible states.

FSM is further distinguished by Deterministic Finite Automata (DFA) and Nondeterministic Finite Automata (NFA). In DFA, for each pair of state and input symbol there is only one transition to a next state whereas, in NFA, there may be several possible next states. Often NFA refers to NFA-epsilon which allows a transition to a next state without consuming any input symbol. That is, the transition function of NFA is usually defined as T: Q x (ΣU{ε}) → P(Q) where P means power set. Theoretically, DFA and NFA are equivalent as there is an algorithm to transform NFA into DFA.

State-machine for testing:

State Transition testing is a Black-box testing technique, which can be applied to test 'Finite State Machines'. A 'Finite State Machine (FSM)' is a system that will be in different discrete states (like "ready", "not ready", "open", "closed",…) depending on the inputs or stimuli. The discrete states that the system ends up with, depends on the rules of the transition of the system. That is, if a system gives a different output for the same input, depending on its earlier state, then it is a finite state system. Further, if every transaction is tested in the system, it is called "0-switch" coverage. If testing covers 2 pairs of valid transactions, then it is "1-switch" coverage, and so on. State transition technique is a dynamic testing technique, which is used when the system is defined in terms of a finite number of states and the transitions between the states are governed by the rules of the system. or in other words, this technique is used when features of a system are represented as states which transform into one another. The transformations are determined by the rules of the software.

So here we see that an entity transitions from State 1 to State 2 because of some input condition, which leads to an event and results in action and finally gives the output.

To explain it with an example:

You visit an ATM and withdraw $1000. You get your cash. Now you run out of balance and make exactly the same request of withdrawing $1000. This time ATM refuses to give you the money because of insufficient balance. So, here the transition, which caused the change in state is the earlier withdrawal.

State transition testing definition:

Having understood what State Transition is, we can now arrive at a more meaningful definition for State Transition testing. So, it is a kind of black-box testing in which the tester has to examine the behavior of AUT (Application under Test) against various input conditions given in a sequence. The behavior of the system is recorded for both positive and negative test values.

When to use state transition testing?

State Transition testing can be employed in the following situations:

- When the application under test is a real-time system with different states and transitions encompassed.
- When the application is dependent upon the event/values/conditions of the past.
- When the sequence of events needs to be tested.
- When the application needs to be tested against a finite set of input values.

When not to use state transition testing?

You should not rely upon State Transition testing under the following situations:

- When testing is not required for sequential input combinations.
- When different functionalities of the application are required to be tested (more like exploratory testing).

Fault Model for FSM:

Output fault the machine provides an output, different from the one specified by the output function. Transfer fault, the machine enters a different state than that specified by the transfer function. Transfer faults with additional states; number of states of the system is increased by the presence of faults, additional states is used to model certain types of errors. Additional or missing transitions; one basic assumption is that the FSM is deterministic and completely defined (fully specified). So the faults occur when it turns out to be non-deterministic and/or incompletely (partially) specified

Fault Types:

1. Missing or incorrect
    a. Transitions – new state is legal but incorrect.
    b. Events – valid message ignored.
2. Extra, missing or corrupt state.
3. Sneak path: messages are accepted when they should not.
4. Illegal message failure.
5. Trap door: system cannot get out of that state.

Coverage Criteria:

We can choose one or more of the following test selection criteria:

1. All states – testing passes through all states.
2. All events – testing forces all events to occur at least once.
3. All actions – testing forces all actions to be produced at least once.

Test Strategies:

Possible strategies are as follows:

-
- All round-trip paths.
- All transition sequences beginning and ending in the same state.
- All simple paths from initial to final state.

These strategies will help you to find all invalid or missing states, some extra states and all event an action faults.

State-Machine as Model – Testing

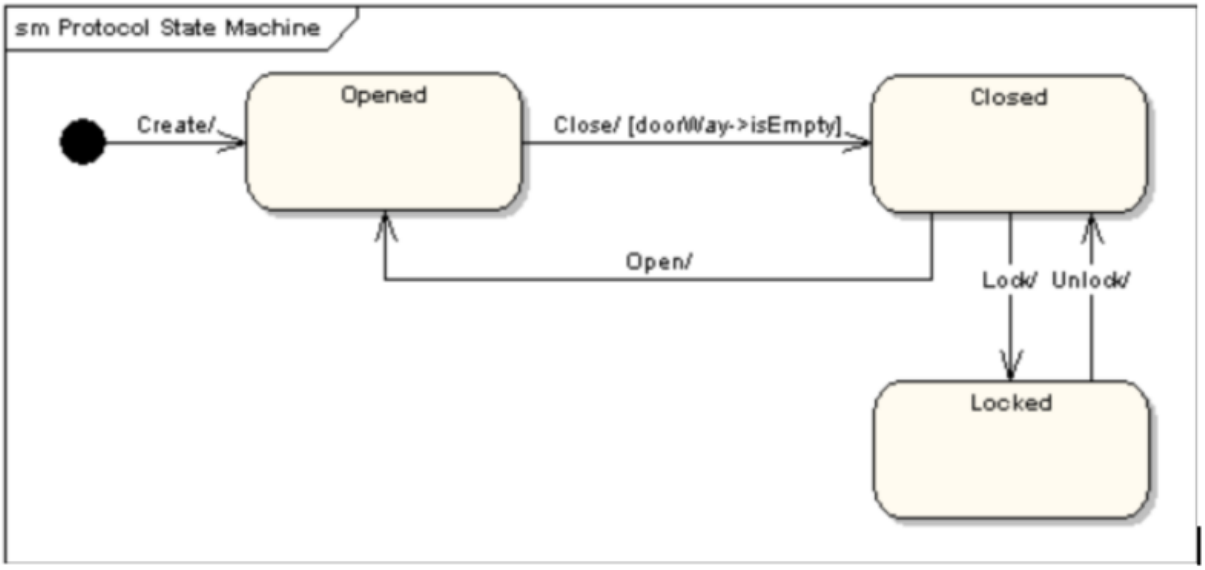
A state machine is a concept used in designing computer programs or digital logic. There are two types of state machines: finite and infinite state machines. The former is comprised of a finite number of states, transitions, and actions that can be modeled with flow graphs, where the path of logic can be detected when conditions are met. The latter is not practically used.

State machines are represented using state diagrams. The output of a state machine is a function of the input and the current state. State machines play a significant role in areas such as electrical engineering, linguistics, computer science, philosophy, biology, mathematics, and logic. They are best used in the modeling of application behavior, software engineering, design of hardware digital systems, network protocols, compilers, and the study of computation and languages.

While doing state-machine based testing, we represent the system as a state-machine where the stimulus is provided by inputs and the result is a state transition from A to B. We write test cases that are sets of inputs and expected outputs and our intention is to see if a transition takes place as intended. This gives rise to coverage criteria required specific to this form of model-based testing and a fault model.

State-Machine as Model – Example

A state machine diagram models the behavior of a single object, specifying the sequence of events that an object goes through during its lifetime in response to events.



As an example, the following state machine diagram shows the states that a door goes through during its lifetime.

The door can be in one of three states: "Opened", "Closed" or "Locked". It can respond to the events Open, Close, Lock and Unlock. Notice that not all events are valid in all states; for example, if a door is opened, you cannot lock it until you close it. Also notice that a state transition can have a guard condition attached: if the door is opened, it can only respond to the Close event if the condition doorway->isempty is fulfilled.

Advantages of the State Transition Technique

-
- Model can be as detailed or as abstract as you need it to be.
- In case a part of the system is more important (that is, requires more testing) a greater depth of detail can be modeled.
- In case the system is less important (requires less testing), the model can use a single state to signify what would otherwise be a series of different states.

State Machine-Based Testing – Example

In the practical scenario, testers are normally given the State Transition diagrams and we are required to interpret it. These diagrams are either given by the Business Analysts or a stakeholder and we use these diagrams to determine our test cases.

Let’s consider the below situation:

Software name – Manage_display_changes

Specifications – The software responds to input requests to change display mode for a time display device.

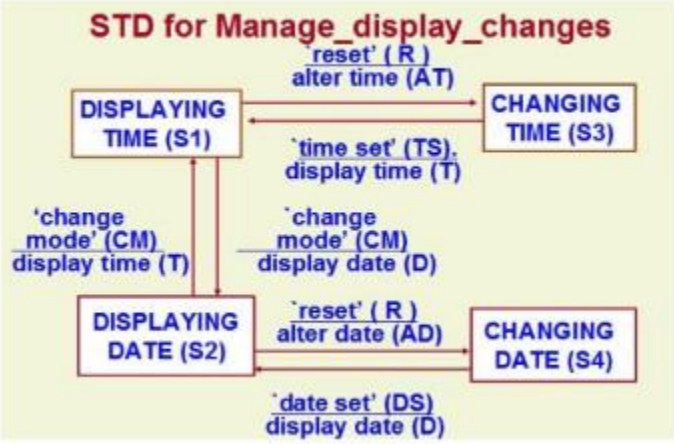
The Display mode can be set to one of the four values:

- Two corresponding to displaying either the time or date.
- The other two when altering either the time or the date.

The different states are as follows:

- Change Mode (CM): Activation of this shall cause the display mode to move between "display time (T)" and "display date (D)".
- Reset (R): If the display mode is set to T or D, then a "reset" shall cause the display mode to be set to "alter time (AT)" or "alter date (AD)" modes.
- Time Set (TS): Activation of this shall cause the display mode to return to T from AT.
- Date Set (DS): Activation of this shall cause the display mode to return to D from AD.

State Transition Diagram



Now, let's move to interpret it: Here:

#1) Various States are:

- Display Time (S1),
- Change Time (S3),
- Display Date (S2), and
- Change Date (S4).

#2) Various Inputs are:

- Change Mode (CM),

- Reset (R),

- Time Set (TS),

- Date Set (DS).

#3) Various Outputs are:

- Alter Time (AT),

- Display Time (T),

- Display Date (D),

- Alter Date (AD).

#4) The Changed States are:

- Display Time (S1),

- Change Time (S3),

- Display Date (S2) and

- Change Date (S4).

Step 1: Write all of the start states. For this, take one state at a time and see how many arrows are coming out from it.

- For State S1, there are two arrows coming out of it. One arrow is going to state S3 and another arrow is going to state S2.
- For State S2 – There are 2 arrows. One is going to State S1 and other going to S4
- For State S3 – Only 1 arrow is coming out of it, going to state S1
- For State S4 – Only 1 arrow is coming out of it, going to state S2

Let's put this on our table:

Start State	S1	S1	S2	S2	S3	S4

Since for state S1 and S2, there are two arrows coming out, we have written it twice.

Step -2: For each state, write down their final transitioned states.

- For state S1 – The final states are S2 and S3
- For State S2 – The final states are S1 and S4

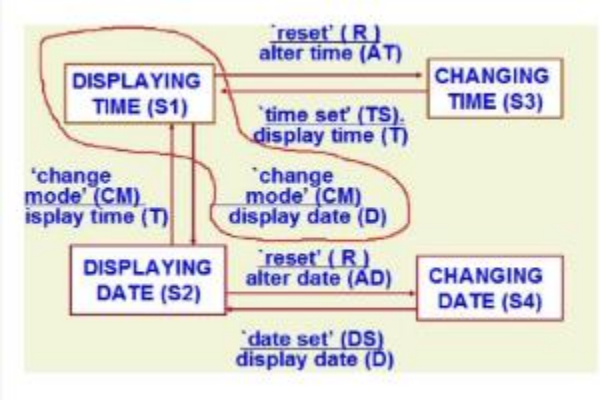
- For State S3 – The final state is S1
- For State S4 – Final State is S2

Put this on the table as an Output/Resultant state.

Start State	S1	S1	S2	S2	S3	S4
Finish State	S2	S3	S4	S1	S1	S2

Step 3: For each start state and its corresponding finish state, write down the input and output conditions

For state S1 to go to state S2, the input is Change Mode (CM) and output is Display Date(D) shown below:



In a similar way, write down the Input conditions and its output for all the states as follows:

Start State	S1	S1	S2	S2	S3	S4
Input	CM	R	R	CM	TS	DS
Output	D	AT	AD	T	T	D
Finish State	S2	S3	S4	S1	S1	S2

Step 4: Now add the test case ID for each test shown below:

Test Case	TC1	TC2	TC3	TC4	TC5	TC6
Start State	S1	S1	S2	S2	S3	S4
Input	CM	R	R	CM	TS	DS
Output	D	AT	AD	T	T	D
Finish State	S2	S3	S4	S1	S1	S2

Now let's convert it to formal test cases:

TCID	Description	Steps	Expected Results
TC1	Validate that the system is able to do the transition from Display time to Display date with output as Display date.	1. Open the application. 2. Enter the Input state as Change mode.	Output should be Display Date and the final state is Display Date.
TC2	Validate that the system is able to do the transition from Display time state to Change time state with output as Alter time	1. Open the application. 2. Enter Input as Reset.	Output should be "Alter Time" and the state is Alter Time.

In this way, all the remaining test cases can be derived. I assume the other attributes of the test cases like preconditions, severity, priority, environment, build, etc. are also included in the test case.

Confidence Based Testing Techniques

In confidence-based testing methods, we test usefulness of test cases or comprehensiveness of our test suite through a measure of confidence. This means we would have a set of test cases upfront and we would do some form of fault seeding to artificially sow faults. In case, we are doing this one-by-one, we call it mutation testing where we have an original and a mutant version of the program after a fault is introduced into the program. We then run after test suite and see if that introduction makes one of our test cases with "pass" status to fail or report a deviation in the results. In case this happens, we say that our test suite is capable of finding faults and we add a test case to find this sown fault otherwise. If we add a single fault per run, this is called mutation testing and if we do a number of faults in one go, we call it fault seeding.

Confidence Based Methods:

In coverage-based testing, we take the structure of the artifact to be tested into account. In fault-based testing, we do not directly consider this artifact instead we look for a test set with high ability to detect faults, there are two techniques as follows:

1. **Fault Seeding**: Fault seeding and mutation testing are fault-oriented techniques, applicable to all levels of testing. In Fault Seeding Technique, a predefined number of artificially generated errors is "sown" in the program code.
2. **Mutation Testing:** In mutation testing, we sow one fault at a time leading to original program and mutant version of that program**.** Test runs are used to detect these faults**,** we examine ratio between actual and artificial errors based on the total number of detected errors. The testers do not know the artificially generated errors.

Mutation Testing

Mutation Testing is a testing technique that focuses on measuring the adequacy of test cases. Mutation Testing is NOT a testing strategy like path or data-flow testing. It does not outline test data selection criteria. Mutation Testing should be used in conjunction with traditional testing

techniques, not instead of them. Faults are introduced into the program by creating many versions of the program called mutants. Each mutant contains a single fault. Test cases are applied to the original program and to the mutant program. The goal is to cause mutant program to fail, thus demonstrating effectiveness of the test case.

How to use Mutants in Testing:

If a test produces different results for one of the mutants, that mutant is said to be dead**.** If a test set leaves us with many live mutants, that test set is of low quality**.** If we have M mutants, and a test set results in D dead mutants, then the mutation adequacy score is D/M**.** Larger mutation adequacy score means better test set.

Strong vs Weak Mutation Testing:

Suppose we have a program P with a component T, we have a mutant T' of Since T is part of P, we then also have a mutant P' of P. In weak mutation testing, we require that T and T' produce different results, but P and P' may still produce the same results. In strong mutation testing, we require that P and P' produce different results.

Assumptions Underlying Mutation Testing:

- Competent Programmer Hypothesis: competent programmers write programs that are approximately correct.
- Coupling Effect Hypothesis: tests that reveal simple fault can also reveal complex faults.

Test and Mutation Adequacy

Test Adequacy:

A test case is adequate if it is useful in detecting faults in a program. A test case can be shown to be adequate by finding at least one mutant program that generates a different output than does the original program for that test case. If original and all mutants generate same output, test case is inadequate.

Mutant Programs:

Mutation testing involves the creation of a set of mutant programs of the program being tested. Each mutant differs from the original program by one mutation. A mutation is a single syntactic change that is made to a program statement.

Mutation Adequacy:

Mutation adequacy uses a similar concept to fault seeding to evaluate the effectiveness of a test suite. Assume we have a test suite TS with confidence C with total test cases c(j). Assume that the program under test P passes all the test cases c(j) for 1 <= j <= C. Can we stop testing? That is, have we tested P adequately?

The mutation adequacy criterion provides one answer that we might use. The mutation adequacy approach differs from fault seeding in that it is applied at a particular point in the testing process and also in that faults are not directly inserted into. Instead, a series of mutants m(i) are created. Each mutant m(i) differs from P by the injection of exactly one fault.

Let M be the total number of mutants m(i). The test suite TS is applied to each mutant m(i). If a particular mutant m(i) fails any test in c(j), then it is said to be killed. All mutants that are not killed are said to remain live at this point. The ratio of killed to total mutants (K/M) can be considered a measure of adequacy of TS.

Mutation Score:

Mutation score for a set of test cases is the percentage of non-equivalent mutants killed by the test data.

- **Mutation Score = 100 * D / (N - E)**
  - **D** = Dead mutants
  - **N** = Number of mutants
  - **E** = Number of equivalent mutants
- A set of test cases is mutation adequate if its mutation score is 100%.

Mutation Operator – Example:

Manually creating mutants is time-consuming. A collection of mutants m(i) created from P at some point in time will no longer be representative of P after it has undergone many changes. Mutation can be automated by through the concept of mutation operators. Mutation operators are simple changes that can be made at various program locations.

Mutation Operator	Meaning	Original Code	Mutated Code
Add 1	Add 1 to a constant	`q = 0;`	`q = 1;`
Replace Variable	Replace a variable with a different one of the same type	`r = x;`	`r = y;`
Replace Operator	Replace an operator with a compatible one	`q = q + 1`	`q = q - 1`

The following table shows a program P to perform integer division. Given inputs x and y, P computes the integer division of x divided by y producing quotient q and remainder r. Three mutants m(1), m(2), and m(3) are shown resulting from application of the mutation operators in the previous table.

P	*m*(1)	*m*(2)	*m*(3)

`q = 0;`
`r = x;`
`while r >= y {`
`  r = r - y;`
`  q = q + 1;`
`}`
	`q = 1;`
	`r = x;`
	`while r >= y {`
	`  r = r - y;`
	`  q = q + 1;`
	`}`
		`q = 0;`
		`r = y;`
		`while r >= y {`
		`  r = r - y;`
		`  q = q + 1;`
		`}`
			`q = 0;`
			`r = x;`
			`while r >= y {`
			`  r = r - y;`
			`  q = q - 1;`
			`}`

Mutation Testing – Evaluation:

Theoretical and experimental results have shown that mutation testing is an effective approach to measuring the adequacy of test cases. The major drawback of mutation testing is the cost of generating the mutants and executing each test case against them.

Fault Seeding

Fault Seeding, is used to estimate the "number of errors" remaining in a program. In Fault Seeding Technique, a predefined number of artificially generated errors is "sown" in the program code. Fault Seeding, also known as error seeding, defect seeding or be bugging is a method in the software development life-cycle to evaluate the Fault Detection Rate of software tests and the number of remaining faults in the software. These two metrics provide a deeper understanding about the test quality and software quality. Fault seeding as a technique introduced by Mills (1972).

If S is the total number of faults seeded, n is the number of seeded faults detected, and N is the number of non-seeded errors detected, the number of remaining (non-seeded) faults in program is: $N(S/n - 1)$. One or more faults are deliberately introduced into a code base, without informing the testers. The discovery of seeded faults during testing can be used to calibrate the effectiveness of the test process. Let S is the total number of seeded faults, and s(t) is the number of seeded faults that have been discovered at time t. S(t)/S is the seed-discovery effectiveness of testing to time t.

Process

The idea behind Fault Seeding is to simply insert artificial faults into the software and after testing count the number of artificial and real faults discovered by the software tests. The complete process of Fault Seeding consists of the following five steps:

- Fault Modeling
- Fault Injection
- Test Execution
- Calculation
- Interpretation

Fault Modeling:

The first step is to model some artificial faults. You can e.g. replace arithmetic, relational or logical operators, remove function calls or change the datatype of variables. There are many types of faults, be creative.

Fault Injection:

These artificial faults are then seeded into the software. The number of seeded faults is self-determined and depends on the size of the software. For example, if your software has 1,000 lines of code five artificial faults might be insufficient. An exact rule to determine the number of artificial faults does not exist, sorry folks.

The injection of artificial faults into the master branch should be avoided. It is recommended to create a new branch and do the fault seeding activities in this branch.

Test Execution

Now your manual or automated tests are executed. The aim of the tests is to discover as much faults as possible.

Calculation

Your tests hopefully discover a proper number of real and artificial faults. Now, the number of discovered seeded faults, the number of seeded faults and the number of discovered real faults are known. In the following mathematical equations you can see the calculation of the Fault Detection Rate and the number of the remaining faults. The following mathematical equation represents the basic relationship between seeded and real faults which the Fault Seeding method is based on.

$$\frac{Discovered\ \ seeded\ \ faults}{Seeded\ \ faults} = \frac{Discovered\ \ real\ \ faults}{Real\ \ faults}$$

The other mathematical equations shows the calculation of the Fault Detection Rate and the number of remaining faults which are derived from the first equation.

$$Fault\ \ Detection\ \ Rate\ = \frac{Discovered\ \ seeded\ \ faults}{Seeded\ \ faults}$$

$$Real\ \ faults\ = Seeded\ \ faults \times \frac{Discovered\ \ real\ \ faults}{Discovered\ \ seeded\ \ faults}$$

$$Remaining\ \ faults\ = Real\ \ faults\ - Discovered\ real\ \ faults$$

Interpretation

The main objective of Fault Seeding is to evaluate both the software and test quality. On the basis of this information you can identify deficiencies of your tests and define and execute improvement activities to reduce the identified deficiencies.

Fault seeding provides an alternative to structural testing techniques providing us with a measure of sufficiency of testing or effectiveness of our test suite.

Fault types:

1. Domain faults
2. Computational faults

Mutation Testing

Mutation testing is a type of software testing in which certain statements of the source code are changed/mutated to check if the test cases are able to find errors in source code. The goal of Mutation Testing is ensuring the quality of test cases in terms of robustness that it should fail the mutated source code.

The changes made in the mutant program should be kept extremely small that it does not affect the overall objective of the program. Mutation Testing is also called Fault-based testing strategy as it involves creating a fault in the program and it is a type of White Box Testing which is mainly used for Unit Testing.

Mutation was originally proposed in 1971 but lost fervor due to the high costs involved. Now, again it has picked steam and is widely used for languages such as Java and XML.

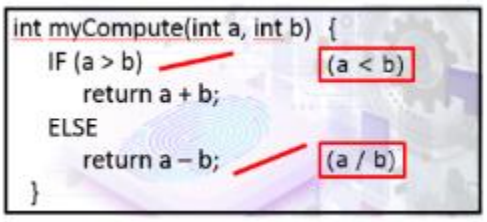
Objective of Mutation Testing:
The objective of mutation testing is:

- To identify pieces of code that are not tested properly.
- To identify hidden defects that cannot be detected using other testing methods.
- To discover new kinds of errors or bugs.
- To calculate the mutation score.
- To study error propagation and state infection in the program.
- To assess the quality of the test cases.

Mutation Testing Example

Here we will add mutation operators. We will induce faults and try to find out faults as well as we will try to explain that what confidence is.

Example Code: We do two mutations one by one. Here we replace a>b with a<b and a-b with a/b



Therefore there are three versions of the same program, two mutants and one original code

- Three versions:

```
int myCompute(int a, int b) {
    IF (a > b)  return a + b;
    ELSE        return a – b;    }
```

```
int myCompute(int a, int b) {
    IF (a < b)  return a + b;
    ELSE        return a – b;    }
```

```
int myCompute(int a, int b) {
    IF (a > b)  return a + b;
    ELSE        return a / b;    }
```

Now we will perform testing on these three versions. Following are the three versions of the same program along with their test cases are shown

(ORIGINAL CODE)

```
int myCompute(int a, int b) {
    IF (a > b)  return a + b;
    ELSE        return a – b;    }
```

(MUTATION 1 (M1)

```
    IF (a < b)  return a + b;
    ELSE        return a – b;    }
```

(MUTATION 2 (M2)

```
    IF (a > b)  return a + b;
    ELSE        return a / b;    }
```

- Test Output

S #	Test Cases	Expected	Actual
1	T_1 = [{(2, 1), 3}]	3	3
2	T_2 = [{1, 2), -1}]	-1	-1

S #	Test Cases	Expected	Actual
1	T_1 = [{(2, 1), 3}]	3	1
2	T_2 = [{1, 2), -1}]	-1	3

S #	Test Cases	Expected	Actual
1	T_1 = [{(2, 1), 3}]	3	3
2	T_2 = [{1, 2), -1}]	-1	0.5

Mutation Score = 100 * D / (N - E)

-
  - D = Dead mutants
  - N = Number of mutants
  - E = Number of equivalent mutants
- Total dead mutants = 2
- Number of mutants = 2
- Equivalent mutants = 0
- Mutation score = 100 * 2/2
- Mutation score = 100%

Since there are no equivalent mutants if there are any equivalent mutants than we will subtracts from total number of mutants.

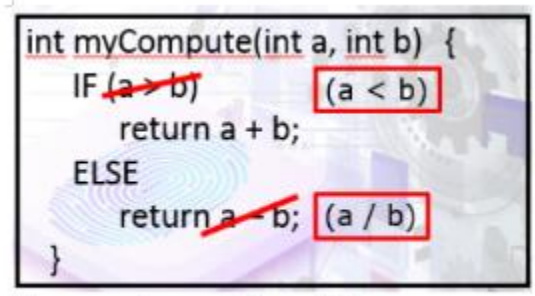
Fault Seeding

Fault seeding is a technique for evaluating the effectiveness of a testing process. One or more faults are deliberately introduced into a code base, without informing the testers. The discovery of seeded faults during testing can be used to calibrate the effectiveness of the test process. In the fault seeding technique, no mutants (a small change in code) are created. In this, the program code is tested within itself for finding the errors.

Let's explain this concept with an example consider *S* is the total number of seeded faults, and *s*(*t*) is the number of seeded faults that have been discovered at time *t*. *S*(*t*)/*S* is the seed-discovery effectiveness of testing to time *t*. If seeded faults are assumed to be representative of actual faults, then seed-discovery effectiveness can be assumed to be representative of overall testing effectiveness.

Fault Seeding Example

Let's take an example Code, we do fault seeding in which N number of faults seeded in one go,

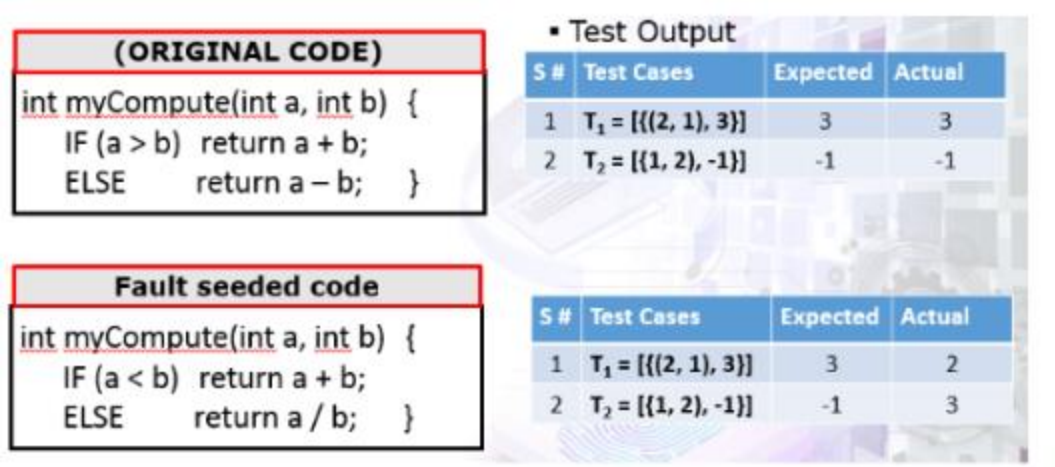


There are two versions of the same program, one fault seeded and one original code

```
int myCompute(int a, int b) {
    IF (a > b)  return a + b;
    ELSE        return a – b;    }
```

```
int myCompute(int a, int b) {
    IF (a < b)  return a + b;
    ELSE        return a / b;    }
```

Now we will perform testing on these two versions



Calculating confidence C

C = 1 if n > N and **C = S/(S - N + 1)** if n ≤ N

Where

S is the number of seeded faults,

N is the total number of non-seeded (indigenous) faults which can be found

We calculate N by

N = S*n / s

Where **n** is actual number of faults detected during testing and **s** is the number of seeded faults detected during testing.

C = 1 since N=0, n=2 => n > N

Topic 86

What is test plan document?

According to IEEE, a document describing the scope, approach, resources, and schedule of intended testing activities is known as test plan document. It identifies test item, the features to be tested, the testing tasks, who will be doing each task, and any risks requiring contingency planning.

Kaner et.al, 1999 defines test plan as a valuable tool to the extent that it helps you manage your testing project and find bugs. Beyond that, it is a diversion of resources.

IEEE Standard for Software Documentation

-
  - Test Plan Identifier
  - Introduction
  - Test Items
  - Features to be tested
  - Features not to be tested
  - Approach
  - Item pass / fail criteria
  - Suspension criteria and resumption criteria
  - Test deliverables
  - Testing tasks
  - Environmental needs
  - Responsibilities
  - Staffing and training needs
  - Schedule
  - Risks and contingencies
  - Approvals
- **Test Items**

It identifies items that are to be tested within the scope of this test plan. It includes

-
  - Functions of the software
  - Requirements stated in the Design stage
  - This is important while reporting test results, outputs etc.
- **Specifications**
  - Test Design Specification
    - Test conditions, test items, high level test cases
  - Test Case Specification

- **Test Case Ingredients**
  - For different aspects of the test plan, we have different test case ingredients
  - ID : unique identifier of a test case
  - steps / input values
  - Expected result / output values
  - Actual result: what you really get from application
  - Pass / Fail
  - Verbal description indicative of test case objective
  - Goal / objective : primary verification point of the test case
  - Functional area
  - Bug numbers for Failed test cases
  - Positive / Negative class: for test execution planning
  - Manual / Automatable / Automated parameter etc : for planning purposes

- **Test approach:** This section identifies strategy for test plan, differing depending on the level of test plan (Unit, Integration and Acceptance). The level of detail of this section differs depending on the level of test plan. For example, a Unit test plan will go into much detail on individual unit tests and test data.
- **Risks:** All risks associated with the software and its testing need to be identified in this section. This could include complex functions and new versions of cooperating software. Test planners should be aware of vague, unclear or un-testable requirements and misunderstanding of requirements.
- **Features to be tested:** It identifies the features to be tested from a particular development phase point of view. For example, if we are working on an enhancement in our product ABC, where only HR module is added and finance module is updated. We only test:
    -
    - HR Module
    - Finance module

- **Features not to be tested:** Features not to be tested lists the features not to be included in the testing process. We also identify reasons behind its exclusion.

Test Plan
(Test Plan Objectives and Motivation)

- **Test Plan Objectives:**
- Test objectives and success criteria should be developed based on client's business, technical goals and service level agreements (SLAs) associated with applications or services. Test objectives should simply measure the outcome of the test case. This helps test plan developer in defining relevant test cases with clearly identifiable success or failure metrics that can be agreed upon by tester / client.
- **Test Plan Motivation:**
- Testers' motivation is to find faults or defects and to provide a mechanism to identify all aspects of testing that are essential to test SUT considering application domain and SRS document.
- A matrix containing test cases segregated according to their type and nature. For example, we need separate sections containing test cases pertaining to functionality, load, stress, admin pages, user pages, content managers etc. A test result reporting mechanism is used to satisfy key stakeholders, financer, management and development teams etc.